

Assessment Cover Sheet

Assessment Title	Project		
Assessment Type	Uncontrolled	Group	Not must-pass
Due Date	19-5-2026	Course Code	IT8510
Course Title	Threat Intelligence and Threat Hunting		
Internal Moderator's Name	Dr. Ayman Alani		
External Examiner's Name	Dr. James Egan		

Instructions:

1. This cover sheet must be completed (section in blue below) and attached to your assessment before submission in hard copy/soft copy.
2. Submit the project report on Sunday, 19-May-2026, before 11:55 PM via Moodle.
3. If an assessment is submitted late, the maximum result the student can achieve is 60% in that assessment. The cut-off time for submitting an assessment will be 3 calendar days after the assessment is due. After this, students will get 0%.
4. Students will have to work in as a group project, and each group can have max four members.
5. Each group must prepare a report and submit it on Moodle. Only the group leader is required to submit the report on behalf of the group.

Learner ID		Date Submitted	20/5/2026
Learner Name	Abdulelah slais,		
Programme Code	IT8510		
Programme Title	Threat Intelligence and Threat Hunting		
Lecturer's Name	Sehrish Shahid		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Learner Signature	Abdulelah slais
--------------------------	-----------------

Do not write beyond this line. For assessor use only.

Assessor's Name			
Marking Date		Maks Obtained	

Comments:

Contents

Introduction.....	3
Part A	3
Task 1:.....	3
Task 2:.....	4
1. VirusTotal.....	4
2. AbuseIPDB	5
3. AlienVault OTX.....	6
Part B	8
Task 1:.....	8
Introduction.....	8
Task 2 - XDR Detection and Automated Response	18
Attack 1 Evidence Package: SMB Brute Force Detection and IP Blocking	19
Attack 2: Malware Activity Detection and Automated Response (EICAR + VirusTotal)	29
Attack 3: Suspicious PowerShell Activity Detection and Response.....	42
Task 3 - SOAR Automation Using Shuffle.....	54
Introduction.....	54
Task 3 Result.....	63
Task 4: Threat Intelligence Integration into Wazuh.....	63
Source 1: VirusTotal	63
Source 2: AbuseIPDB	71
Conclusion	84
References.....	85
Group Reflection & Peer Review Table	86

Introduction

This project was undertaken as part of the IT8510 Threat Intelligence and Threat Hunting course, where the group members assumed the role of SOC consultants tasked with researching, designing, and then implementing a functional Security Operations Centre environment. The main platform that was used throughout the whole project is Wazuh, an open-source SIEM solution, which was extended with threat intelligence and threat hunting capabilities to collect, monitor, and analyze security events across multiple systems.

The project is structured across two main parts. Part A covers the design phase, including a full SOC architecture that integrates eight components alongside defined roles and workflows, and also a structured analysis of three external threat intelligence sources: VirusTotal, AbuseIPDB, AlienVault OTX.

Part B covers the complete technical implementation, which includes deploying and configuring the Wazuh SIEM environment with Windows Server 2022, Bodhi Linux, and Snort IDS as integrated log sources; also simulating and detecting three cyber threats using XDR capabilities, with automated response actions, automation alert-driven responses by using a Shuffle SOAR playbook, and integrating all three threat intelligence sources directly into Wazuh for enriched detection.

The goal of this project is to demonstrate practical competency in deploying a real-world SOC environment that combines threat detection, threat intelligence enrichment, and automated response within a unified security monitoring framework.

Part A

Task 1:

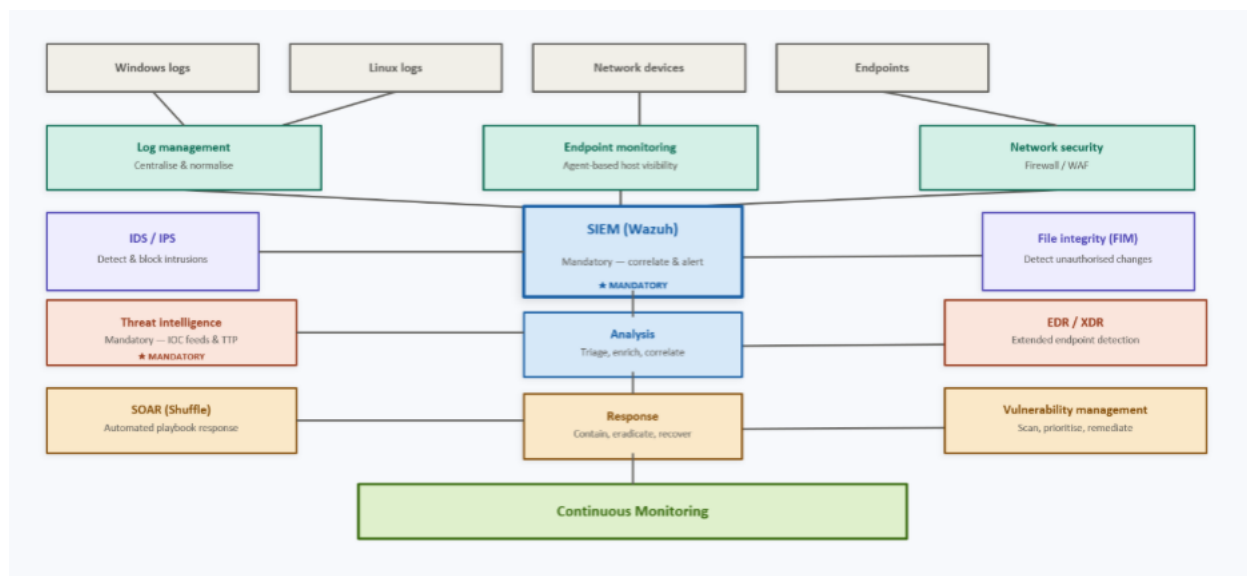


Figure 1: SOC architecture

This SOC architecture is a combination of eight components that span the entire security operations lifecycle. The two components required are the central correlation and alerting engine (SIEM - Wazuh) and the Threat Intelligence, that will provide external IOC feeds and TTP data from VirusTotal, AlienVault OTX, AbuseIPDB, etc.

Supporting components selected are: Log Management (centralizes and normalizes logs from Windows, Linux, and network devices); Endpoint Monitoring (offers agent-based visibility of hosts); Network Security (Firewall/WAF for perimeter security); IDS/IPS (network-level detection/blocking of intrusions); File Integrity Monitoring (flags unauthorized changes to files); EDR/XDR (extended endpoint detection and response); SOAR (Shuffle, for automated playbook execution); Vulnerability Management (continuous scanning and remediation prioritization).

The SOC workflow is a continuous cycle: Data Collection -> Detection -> Analysis -> Response -> Continuous Monitoring and returns findings back to enhance detection rules.

Five SOC Roles:

Role	Responsibilities
SOC Manager	Oversees Operations, staffing, reporting
L1 SOC Analyst	Monitors Alerts and performs initial triage
L2 SOC Analyst	Investigates escalated Incidents
Threat Hunter	Proactively searches for hidden threats
Threat Intelligence Analyst	Curates and operationalizes IOC/TTP feeds

Task 2:

This task talks about the three external threat intelligence sources chosen in details and structured analysis. The sources selected to support SOC operations are: VirusTotal, AbuseIPDB, and AlienVault OTX. For each source, the following will be explained in detail: purpose and overview of the platform, types of intelligence the platform provides, and how the selected source supports SOC operations, including detection, investigation, and response activities, which will include practical examples to demonstrate how the source is used.

1. VirusTotal

1.1 Purpose and Overview

VirusTotal is a free online threat intelligence platform, it is a multiscanner interface created by google. The primary objective of this platform is that it scans files,URLs using different antivirus engines, website security scanners, and blacklist databases to detect potential threats. It acts as a centralized platform that combines data from over 70 different security engines to deliver an overall evaluation on whether the given digital asset, whether a file or link, is malicious or not. By using this platform, it improves the probability of detecting various types of threats, including zero-day attacks and polymorphic malware that might be able to bypass traditional signature-based detection.

1.2 Types of Intelligence Provided

Type of Intelligence	Descriptions
IP Addresses	Reputation, geolocation, ASN, detected URLs, communicating samples.
Domains	WHOIS data, DNS records, subdomains, and related malware/phishing indicators.
URLs	Malicious domains, phishing, or suspicious URLs that are assessed against blocklists and scanners.
File Hashes	MD5, SHA-1, and SHA-256 hashes analyzed against over 70 AV engines to determine maliciousness.
Malware Details	Behavior analysis report, sandbox execution results, and YARA rule.
TTPs	MITRE ATT&CK technique mappings in sandbox reports explain how known malware families' behaviors react to specific tactics and techniques in the ATT&CK framework.

1.3 SOC Application

Threat Detection

When a file is created, Wazuh fires an alert containing a file hash (such as FIM), this hash gets auto-sent to VirusTotal's API to instantly determine whether that hash is flagged as malicious or not by any of its detection engines. This enriches the raw alert with actionable threat data, that enables faster triage.

Investigation

During any incident investigations, SOC analysts send suspicious domains, files, or IPs that have been encountered in logs to VirusTotal to claim detailed reports about the investigation submitted such as sandbox results, behavioral analysis, etc. This helps the analysts to further understand the full of a threat without the need for execution malware in a local sandbox environment.

Response

VirusTotal supports controlled decisions. In the case that the file hash has returned to be malicious and confirmed with a high detection ratio, the analysts can immediately escalate to isolation, hash-based blocking via EDR, or firewall rule updates to target specific malicious IPs and domains that have been identified in the report.

Practical Example

A SOC analyst receives a Wazuh alert indicating that a new executable file has been created on a Windows server, that was triggered by FIM (File Integrity Monitoring) rule. The alert sent contains the hash SHA-256. The first thing the analyst does is submit the hash to VirusTotal using API. VirusTotal then returns with 48/72 malicious, which means that 48 out of 72 antivirus engines flag the file as a known ransomware (e.g., Lynx, LockBit). The analyst also receives the associated IP addresses and domains from the VirusTotal report. Having this much evidence from the report and the intelligence, the analyst isolates the endpoint, blocks identified indicators at the firewall and escalates the incident to Tier 3 for full forensic analysis.

2. AbuseIPDB

2.1 Purpose and Overview

AbuseIPDB is a threat intelligence platform that is publicly accessible that is focused on IP address reputation. The main purpose of the platform is it acts as a central blacklist, where users are able to check report histories of any IP addresses to see if it has been associated with harmful actions. The way that this works is different users such as analysts, webmasters, and system administrators use this platform to report IP addresses that engage in abusive behavior, such as spamming, hacking, port scanning, brute force attacks, and other malicious activities.

2.2 Types of Intelligence Provided

Type of Intelligence	Description
Confidence Score	Abuse confidence score (0-100%) reflecting how likely an IP is to be abusive.
Total Report	The total number of abused report that have been filed against an IP by users.
ISP	The Internet Service Provider that is associated with the IP address

Abuse Categories	Attack types classified: port scan, web spam, DDoS, brute-force, hacking, phishing, etc
Geolocation	Information about the geographical location of a specific IP address
Blocklists	Downloadable list of the most commonly reported IP addresses for proactive blocking

2.3 SOC Application

Detection

AbuseIPDB improves Wazuh alerts by adding IP reputation scores to events that involve source IP addresses. When Wazuh detects activity, such as failed login attempt, it can automatically ask AbuseIPDB to check whether the IP address has a record of malicious behavior or abusive activities. If the confidence score returns to be significantly high (for example, 80%+), this provides strong evidence of a potential threat activity and elevates the alert severity.

Investigation

During investigations, analysts can examine the full abuse report history of a suspicious IP they have on AbuseIPDB, in this way, they can look at the types of attacks that have been previously associated with this IP, how many different users reported it, and the timeframe that the activity occurred in. This context helps the analysts decide whether an alert represents a targeted attack by a known threat actor from what they've seen, or just a random automated scan.

Response

AbuseIPDB intelligence helps guide analysts to response actions. In the case that an IP address is confirmed to be malicious and has a high abuse confidence score, the SOC analysts can automatically act using Wazuh. For example, it can block the IP on the host's firewall by running an active response script, or by adding the IP to a network-level blocklist via integration with a firewall or web application firewall (WAF).

Practical Example

A SOC analyst notices a Wazuh alert showing 200 failed SSH login attempts within the time period of five minutes targeting a Linux server, and it all comes from the same external IP address.

The first step that happens is that Wazuh automatically checks the IP address with AbuseIPDB to check the history of this IP, and it receives a 95% abuse confidence score, along with more than 1,500 reports from 100+ different users. The reports classify the activity as SSH brute-force attack and port scanning.

Based on the information received, Wazuh triggers an active response rule that will immediately block the IP address on the servers' firewall by using iptables, which stops the attack in real-time.

Finally, the analysts document the incident and submit a new abuse report to AbuseIPDB to contribute and strengthen the shared threat intelligence database.

3. AlienVault OTX

3.1 Purpose and Overview

AlienVault OTX (Open Threat Exchange) is the world's first open threat intelligence community which was developed by AT&T Cybersecurity. Its primary objective is to enable global sharing of threat intelligence through a collaborative model where threat researchers and security professionals, analysts, and organizations worldwide contribute over 19 million threat indicators daily known as Pulses. Each pulse represents a malware family or threat campaign and contains associated indicators

or compromise (IOCs). OTX allows anyone in the security community to discuss, research, validate, and share the latest threat trends, data, and techniques, which help one another strengthen the cyber defenses and helps also raise awareness of emerging threats globally. OTX provides a free API and supports direct integration with Wazuh through threat feed ingestion, which makes it one of the most feature-rich and freely accessible threat intelligence sources available to SOC teams.

3.2 Types of intelligence Provided

Type of Intelligence	Description
IP Addresses	Ips that have been used as source/ destination for servers suspected of malicious activity.
CVE	Standards-based identification of common vulnerabilities and exposures that are linked to threat pulses
Domain	Domain names that are suspected of hosting or engaging in malicious activity, which may also include a series of hostnames.
File Hashes	MD5, SHA256, SHA1. Hashes used to identify whether a file is malicious or not, it is often used to identify malwares such as trojans, viruses, ransomware, etc.
Mutex	Mutual Exclusion Object. A mechanism used by malware to detect whether a system has already been infected.

3.3 SOC Application

Detection

Wazuh integrates with AlienVault OTX to make it possible for the system to compare the telemetry information with threat intelligence feeds, allowing user to detect potential threats based on the latest threat intelligence data. Whenever a Wazuh alert includes an IP address, domain name, or file hash, it is compared against OTX IOCs, flagging any matches as malicious.

Investigation

If any IOC is discovered for a particular domain or indicator, the alert generated contains information that can help further analyze the IOCs found, giving the analyst additional threat context, such as the associated pulse, threat actor, and related indicators.

Response

A Wazuh alert triggered by an OTX IOC will result in the activation of an active response script, enabling automation for actions like blocking a malicious IP address or isolating an affected host.

Practical Example

A SOC analyst receives an alert through Wazuh regarding a Windows endpoint where Sysmon Event ID 22 triggers on the agent, meaning that the endpoint performed a DNS query to an unfamiliar domain. Wazuh identifies the queried domain as one that has never been seen before within the organization and triggers an alert, which automatically triggers an active response script on the agent to perform an API call to AlienVaultOTX with the queried hostname as a parameter. OTX then confirms the domain as a match and identifies it as an IOC in an OTX pulse. The alert generated contains relevant information about this malicious IOC, including its associated threat campaign, IP addresses, and the pulse that flagged it. With all this information, the analyst can immediately find out which endpoint connected to this malicious domain, review related IOCs from the same pulse to check for further exposure across the network, and take containment actions such as blocking the domain at the firewall level.

Part B

Task 1:

Introduction

The purpose of this task was to deploy centralized SIEM via Wazuh. To achieve this, various agents were integrated into several systems including Windows Server 2022, Bhodi Linux, and snort. Using these machines, the host, server, and network monitoring were provided. Agents were deployed and tested in the Wazuh dashboard to ensure they were communicating with the Wazuh manager. After integration, attacks were simulated from Kali Linux, such as brute force, port scanning, and ICMP attacks. This resulted in events that were logged and monitored in Wazuh, demonstrating the capability to aggregate logs and identify threats. This report's screenshots demonstrate the deployment of agents, log collection, and detection of attacks, meeting the criteria for Part B Task 1.

1. Wazuh Deployment

Wazuh was deployed using the OVA appliance. The OVA stores the Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard into a single machine ensuring all components were configured and compatible with the version being used.

```
[root@wazuh-server ~]# sudo systemctl status wazuh-manager
wazuh-manager.service - Wazuh manager
Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
Active: active (running) since Mon 2026-04-27 22:39:14 UTC; 2min 19s ago
Process: 3960 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
Tasks: 232 (limit: 9464)
Memory: 791.1M
CPU: 45.588s
CGroup: /system.slice/wazuh-manager.service
├─4022 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─4023 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─4024 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─4027 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─4030 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─4072 /var/ossec/bin/wazuh-authd
├─4088 /var/ossec/bin/wazuh-db
├─4124 /var/ossec/bin/wazuh-execd
├─4143 /var/ossec/bin/wazuh-analysisd
├─4205 /var/ossec/bin/wazuh-syscheckd
├─4223 /var/ossec/bin/wazuh-remoted
├─4224 /var/ossec/bin/wazuh-remoted
├─4260 /var/ossec/bin/wazuh-logcollector
├─4270 /var/ossec/bin/wazuh-monitord
└─4290 /var/ossec/bin/wazuh-modulesd
Apr 27 22:39:09 wazuh-server enu[3960]: Started wazuh-analysisd...
Apr 27 22:39:10 wazuh-server enu[3960]: Started wazuh-syscheckd...
Apr 27 22:39:11 wazuh-server enu[3960]: Started wazuh-remoted...
Apr 27 22:39:11 wazuh-server enu[3960]: Started wazuh-logcollector...
Apr 27 22:39:12 wazuh-server enu[3960]: Started wazuh-monitord...
Apr 27 22:39:12 wazuh-server enu[4288]: 2026-04-27 22:39:12 wazuh-modulesd:router: INFO: Loaded router module.
Apr 27 22:39:12 wazuh-server enu[4288]: 2026-04-27 22:39:12 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Apr 27 22:39:12 wazuh-server enu[4288]: 2026-04-27 22:39:12 wazuh-modulesd:inventory-harvester: INFO: Loaded inventory harvester module.
Apr 27 22:39:12 wazuh-server enu[3960]: Started wazuh-modulesd...
Apr 27 22:39:14 wazuh-server enu[3960]: Completed.
```

Figure 2

```
[root@wazuh-server ~]# sudo systemctl status wazuh-dashboard
wazuh-dashboard.service - wazuh-dashboard
Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: disabled)
Active: active (running) since Mon 2026-04-27 22:26:43 UTC; 15min ago
Main PID: 1767 (node)
Tasks: 11 (limit: 9464)
Memory: 347.0M
CPU: 23.323s
CGroup: /system.slice/wazuh-dashboard.service
└─1767 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandled-rejections=warn /usr/share/wazuh-dashboard/2
Apr 27 22:40:00 wazuh-server opensearch-dashboards[1767]: {"type":"response","timestamp":"2026-04-27T22:40:00Z","tags":["pid":1767,"method":"post","statusCo
Apr 27 22:40:00 wazuh-server opensearch-dashboards[1767]: {"type":"response","timestamp":"2026-04-27T22:40:00Z","tags":["pid":1767,"method":"post","statusCo
Apr 27 22:40:00 wazuh-server opensearch-dashboards[1767]: {"type":"log","timestamp":"2026-04-27T22:40:00Z","tags":["error","opensearch","data":{"pid":1767,"me
Apr 27 22:40:00 wazuh-server opensearch-dashboards[1767]: {"type":"log","timestamp":"2026-04-27T22:40:00Z","tags":["info","plugins","wazuh","cron-scheduler"]}
Apr 27 22:40:03 wazuh-server opensearch-dashboards[1767]: {"type":"response","timestamp":"2026-04-27T22:40:03Z","tags":["pid":1767,"method":"post","statusCo
Apr 27 22:40:04 wazuh-server opensearch-dashboards[1767]: {"type":"response","timestamp":"2026-04-27T22:40:04Z","tags":["pid":1767,"method":"post","statusCo
Apr 27 22:40:05 wazuh-server opensearch-dashboards[1767]: {"type":"response","timestamp":"2026-04-27T22:40:05Z","tags":["pid":1767,"method":"post","statusCo
Apr 27 22:40:07 wazuh-server opensearch-dashboards[1767]: {"type":"response","timestamp":"2026-04-27T22:40:07Z","tags":["pid":1767,"method":"post","statusCo
Apr 27 22:40:13 wazuh-server opensearch-dashboards[1767]: {"type":"response","timestamp":"2026-04-27T22:40:13Z","tags":["pid":1767,"method":"post","statusCo
Apr 27 22:40:17 wazuh-server opensearch-dashboards[1767]: {"type":"response","timestamp":"2026-04-27T22:40:17Z","tags":["pid":1767,"method":"post","statusCo
lines 1-20/20 (END)
```

Figure 3

```

[root@wazuh-server ~]# sudo systemctl status wazuh-indexer
● wazuh-indexer.service - wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-04-27 22:27:39 UTC; 15min ago
     Docs: https://documentation.wazuh.com
   Main PID: 2125 (java)
    Tasks: 102 (limit: 9464)
   Memory: 4.76
      CPU: 2min 10.042s
   CGroup: /system.slice/wazuh-indexer.service
           └─2125 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cache.ttl=60 -Dopensearch.networkaddress.cache.negative.ttl=5000

Apr 27 22:27:18 wazuh-server systemd-entrypoint[2125]: WARNING: A terminally deprecated method in java.lang.System has been called
Apr 27 22:27:18 wazuh-server systemd-entrypoint[2125]: WARNING: System::setSecurityManager has been called by org.opensearch.bootstrap.OpenSearch (file:/usr/share/
Apr 27 22:27:18 wazuh-server systemd-entrypoint[2125]: WARNING: Please consider reporting this to the maintainers of org.opensearch.bootstrap.OpenSearch
Apr 27 22:27:18 wazuh-server systemd-entrypoint[2125]: WARNING: System::setSecurityManager will be removed in a future release
Apr 27 22:27:19 wazuh-server systemd-entrypoint[2125]: Apr 27, 2026 10:27:19 PM sun.util.locale.provider.LocaleProviderAdapter <clinit>
Apr 27 22:27:19 wazuh-server systemd-entrypoint[2125]: WARNING: COMPAT locale provider will be removed in a future release
Apr 27 22:27:19 wazuh-server systemd-entrypoint[2125]: WARNING: A terminally deprecated method in java.lang.System has been called
Apr 27 22:27:19 wazuh-server systemd-entrypoint[2125]: WARNING: System::setSecurityManager has been called by org.opensearch.bootstrap.Security (file:/usr/share/
Apr 27 22:27:19 wazuh-server systemd-entrypoint[2125]: WARNING: Please consider reporting this to the maintainers of org.opensearch.bootstrap.Security
Apr 27 22:27:19 wazuh-server systemd-entrypoint[2125]: WARNING: System::setSecurityManager will be removed in a future release

```

Figure 4

Figures 2,3,and 4 verify that the Service Status is active and running.

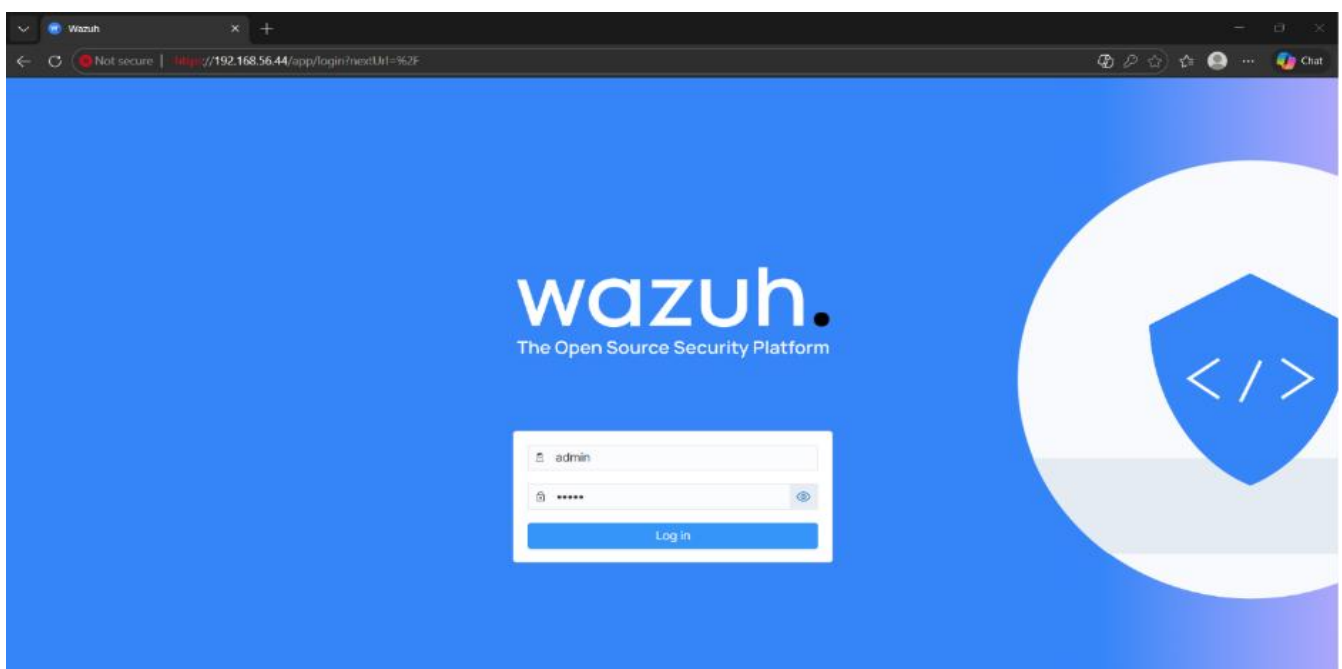


Figure 5: shows that the Wazuh Dashboard is running and accessed via the HTTPS web interface.

2. Agent and Log Source Integration

During this process, T=three log sources were integrated into the Wazuh Manager: Windows Server (2022), BodhiLinux, and Snort IDS running on BodhiLinux via syslog. This section documents this process.

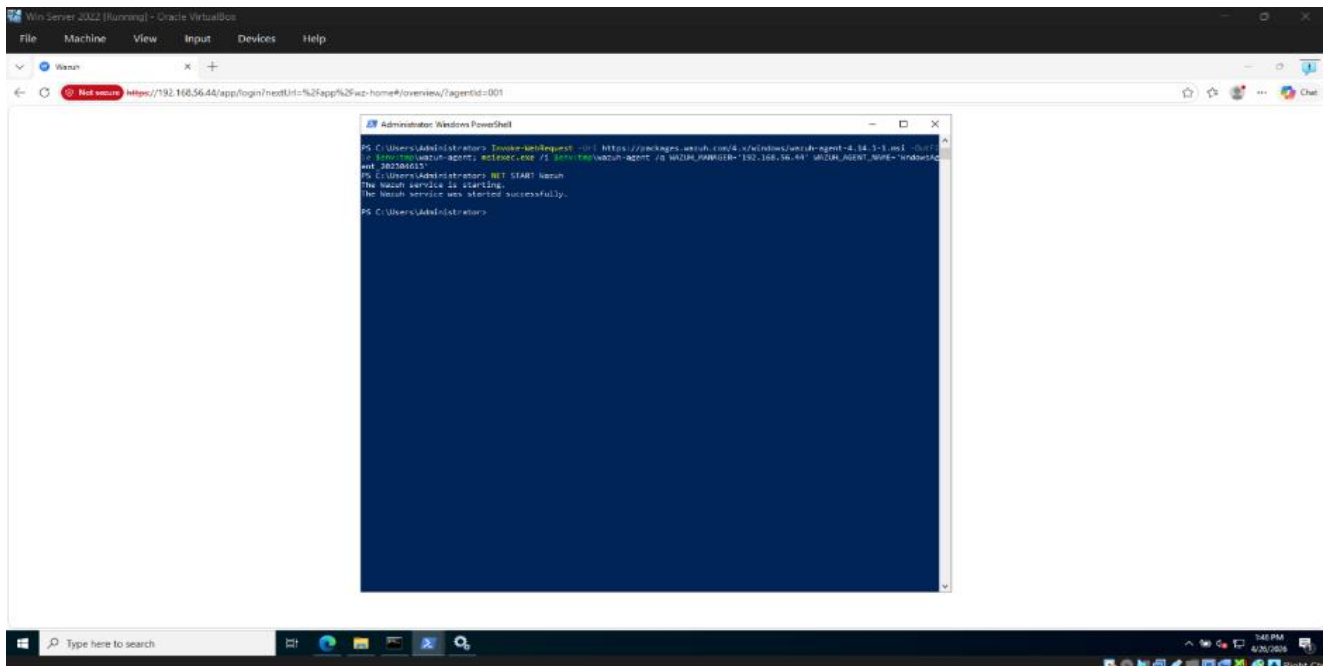


Figure 6: shows the installation of the Agent on Windows Server 2022

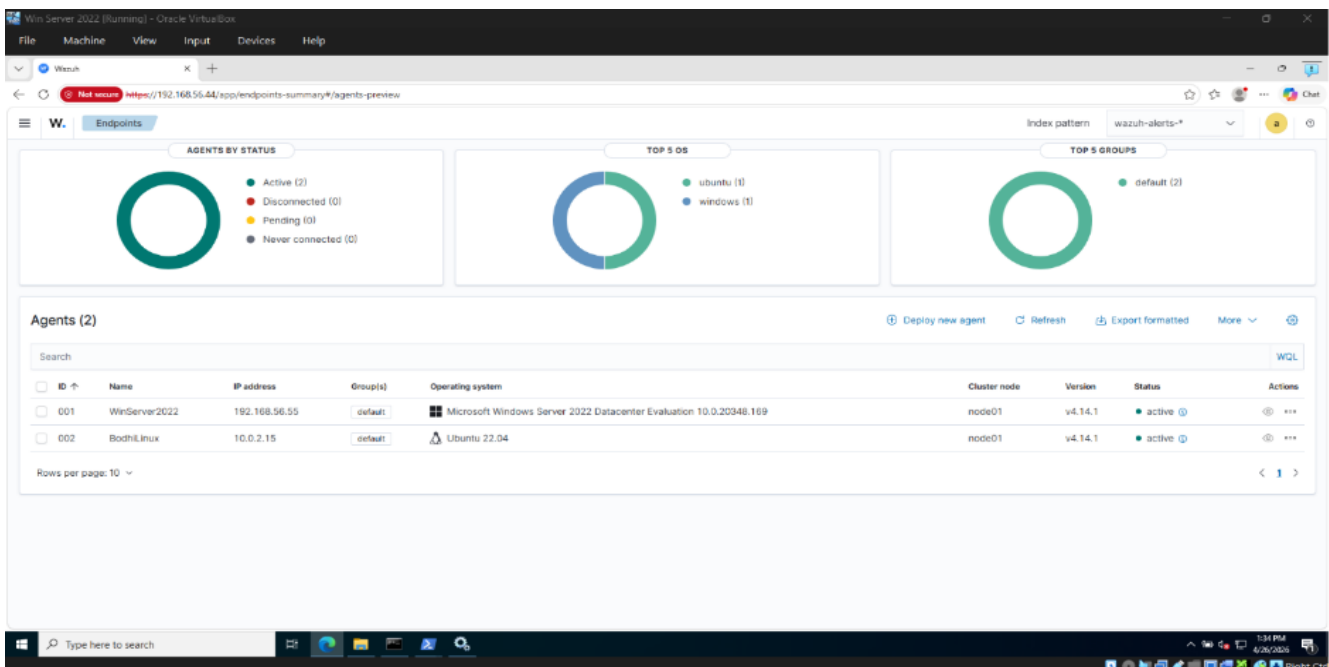


Figure 7: the installed agent on Windows Server is shown in Wazuh dashboard



Figure 8: Shows that the wazuh agent is enabled and started in Bodhi Linux using systemctl.

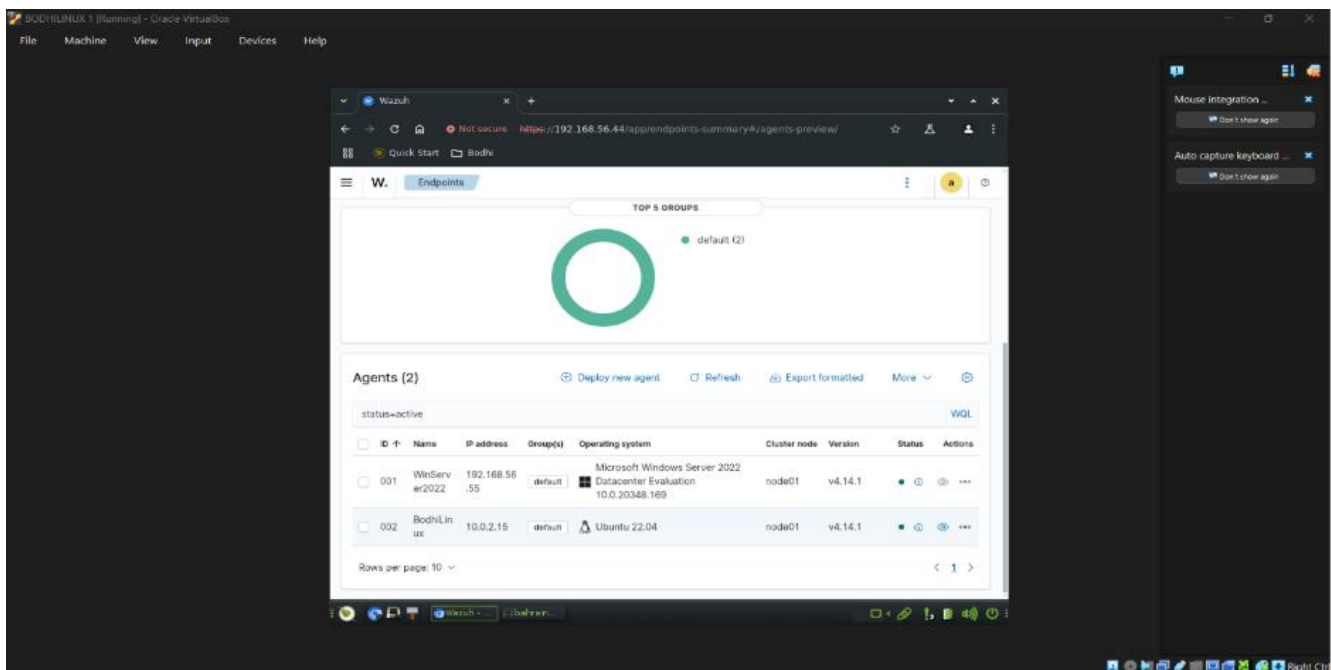


Figure 9: verifies that the agent is active and running.

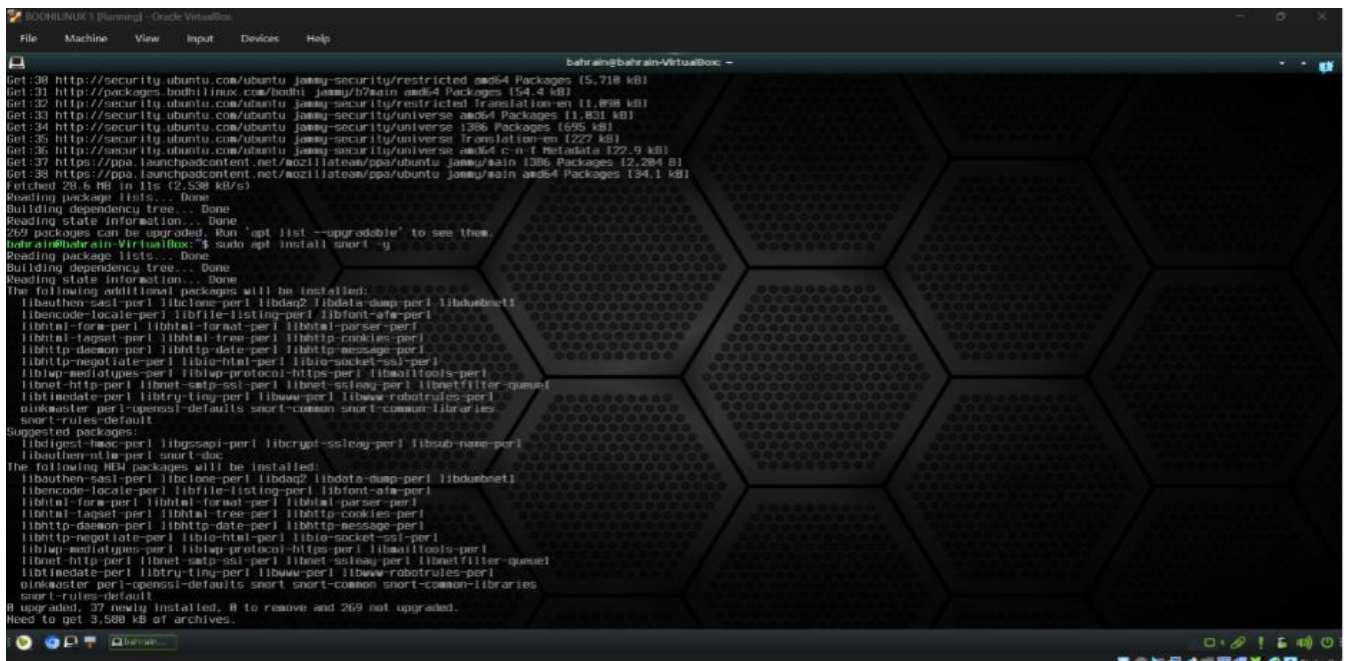


Figure 10: shows that snort is installed on BodhiLinux via apt package manager.

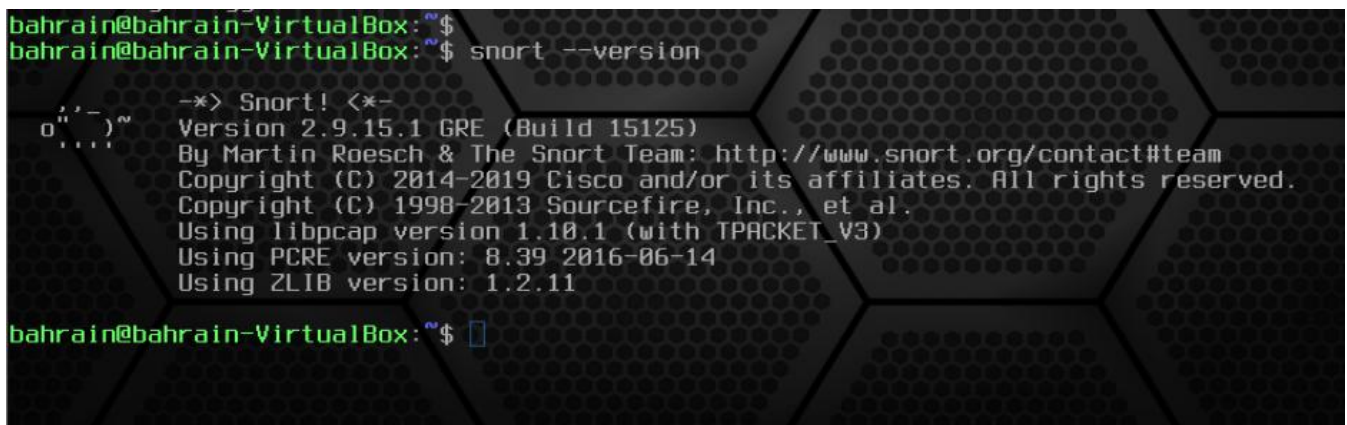


Figure 10: confirms that snort has been installed on BodhiLinux.

```

BODHILINUX 1 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

bahrain@bahrain-VirtualBox: ~
GNU nano 6.2 /etc/snort/snort.conf
# Recommended for most installs
# output unified2: filename merged.log, limit 128, nostamp, mpls_event_types, vlan
output unified2: filename snort.log, limit 128, nostamp, mpls_event_types, vlan

# Additional configuration for specific types of installs
# output alert_unified2: filename snort.alert, limit 128, nostamp
output alert_unified2: filename snort.alert, limit 128, nostamp
# output log_unified2: filename snort.log, limit 128, nostamp

# syslog
output alert_syslog: LOG_AUTH LOG_ALERT[]

# pcap
# output log_tcpdump: tcpdump.log

# Fast alert logging for the daily cron script in Debian
output alert_fast: snort.alert.fast

# metadata reference data. do not modify these lines
include classification.config

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line

```

Figure 11: Snort output plugin has been configured.

```

BODHILINUX 1 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

bahrain@bahrain-VirtualBox: ~
bahrain@bahrain-VirtualBox:~$ sudo snort -s -l empi33 -c /etc/snort/snort.conf
sudo: password for bahrain:
Running in IDS mode

Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plugins!
Parsing Rules file "/etc/snort/snort.conf"
PortVar 'HTTP_PORTS' defined: { 80:81 311 303 591 593 981 1229 1414 1741 1838 2381 2381 2089 3837 3128 3782 4343 4848 5258 6988 7088 7081 7144 7145 7518 7777 7779 8080 8080 8014 8028 8080 8
805 8080 8098 8110 8123 8108:8181 8243 8208 8308 8808 8808 8899 9088 9868 9888 9898 9891 9443 9999 11371 34443 34444 41808 50882 55555 }
PortVar 'SHELLCODE_PORTS' defined: { 0:79 81:65535 }
PortVar 'ORACLE_PORTS' defined: { 1524:65535 }
PortVar 'SSH_PORTS' defined: { 22 }
PortVar 'FTP_PORTS' defined: { 21 2108 3535 }
PortVar 'SIP_PORTS' defined: { 5060:5061 5060 }
PortVar 'FILE_DOWNLOAD_PORTS' defined: { 80:81 110 143 311 303 591 593 981 1229 1414 1741 1838 2381 2381 2089 3837 3128 3782 4343 4848 5258 6988 7088 7081 7144 7145 7518 7777 7779 8080 8080 801
4 8028 8080 8085 8088 8098 8110 8123 8108:8181 8243 8208 8308 8808 8808 8899 9088 9868 9888 9898 9891 9443 9999 11371 34443 34444 41808 50882 55555 }
PortVar 'FTP_PORTS' defined: { 2123 2152 3386 }

Detection:
SearchMethod = HC-Full-G
Split By/By group = enabled
SearchMethod-Optimizations = enabled
Maximum pattern length = 20
Loaded Packet Limit: 256
Loading dynamic engine /usr/lib/snort/snort_dynamicengine/libsf_engine.so... done
Loading all dynamic detection libs from /usr/lib/snort/snort_dynamicrules...
WARNING: No dynamic libraries found in directory /usr/lib/snort/snort_dynamicrules
Finished Loading all dynamic detection libs from /usr/lib/snort/snort_dynamicrules
Loading all dynamic preprocessor libs from /usr/lib/snort/snort_dynamicpreprocessor/...
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_reputation_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_dns_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_mach_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_dnp3_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_smtp_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_ssl_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_sdl_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_ssh_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_telnet_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_gtp_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_pop_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_imap_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_appid_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_sip_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor/libsf_dce2_preproc.so... done
Finished Loading all dynamic preprocessor libs from /usr/lib/snort/snort_dynamicpreprocessor/
Log directory = /var/log/snort

```

Figure 12: Shows Snort initializing IDS mode on Bodhi Linux.

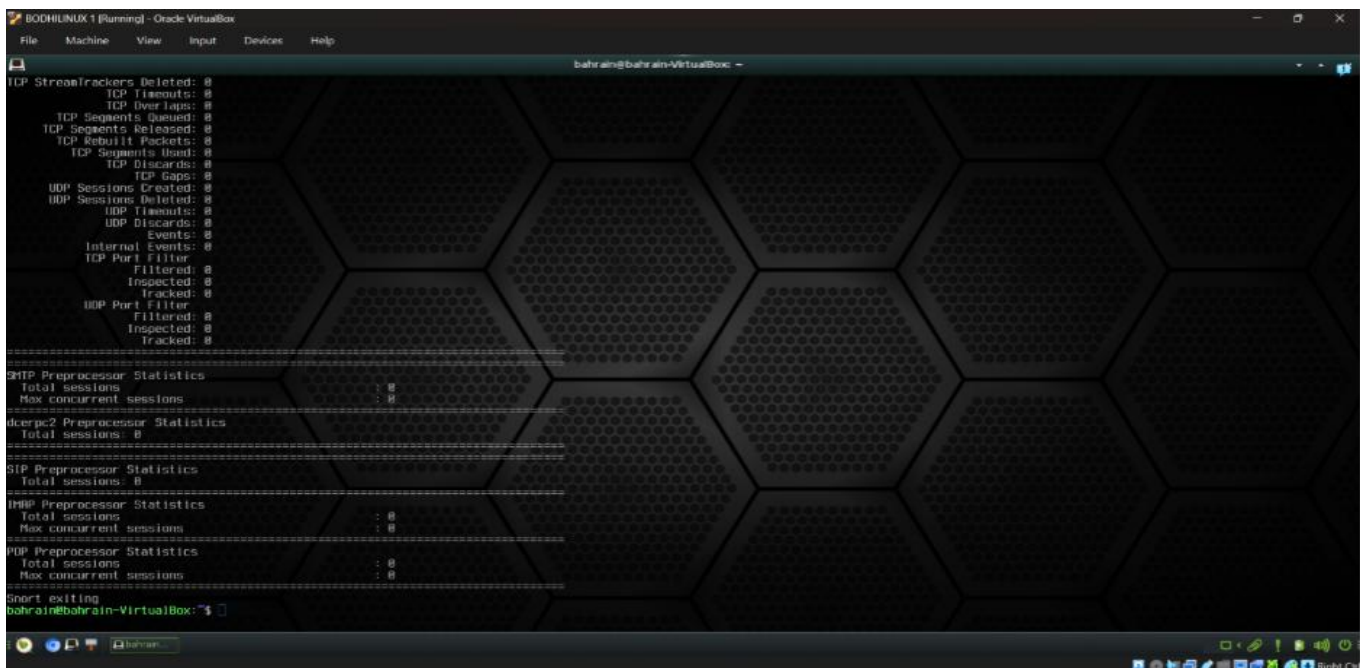


Figure 13: Confirms a successful IDS session.

3. Log Collection and Event Verification

This section ensures that the agents and syslog integration are running correctly as all logs from all resources were successfully received and stored by Wazuh.

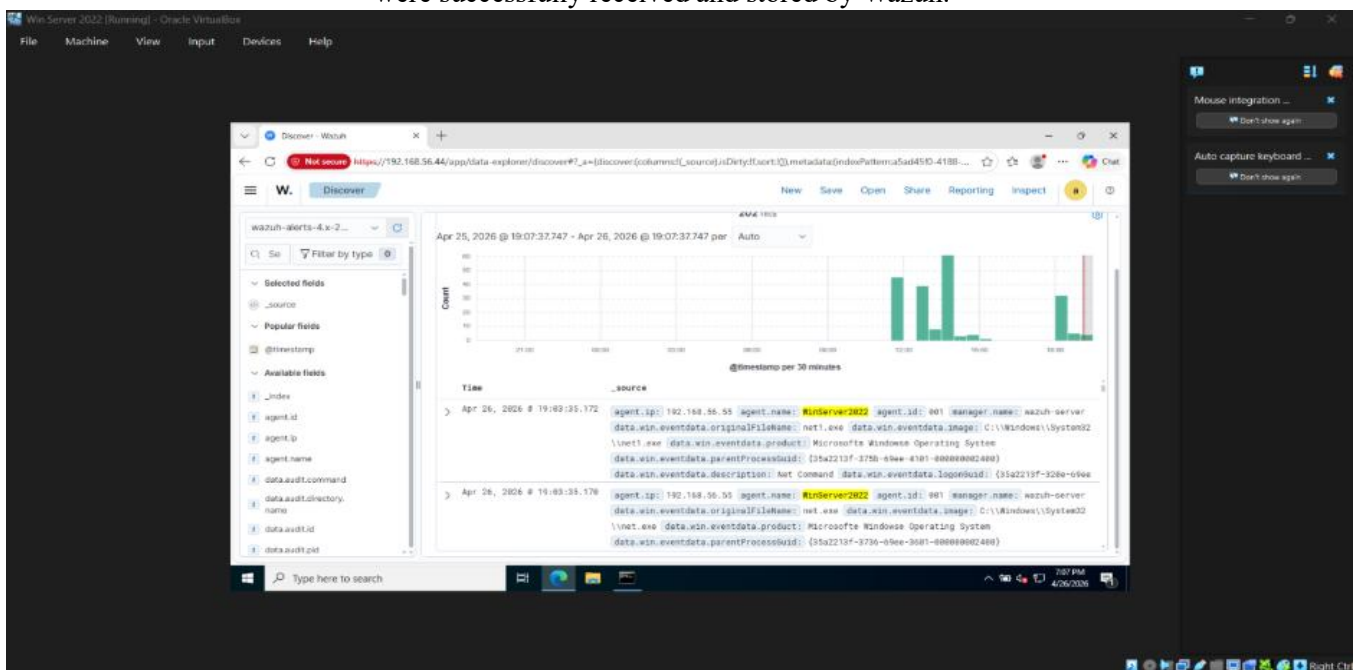


Figure 14: shows the Windows Server Event Logs in Wazuh Discover.

@timestamp per 3 hours	
agent.id	001
agent.ip	192.168.56.55
agent.name	WinServer2022
data.win.eventdata.authenticationPackageName	NTLM
data.win.eventdata.failureReason	%2313
data.win.eventdata.ipAddress	192.168.56.1
data.win.eventdata.ipPort	54982

Figure 15: Shows a detailed log from event ID 4625

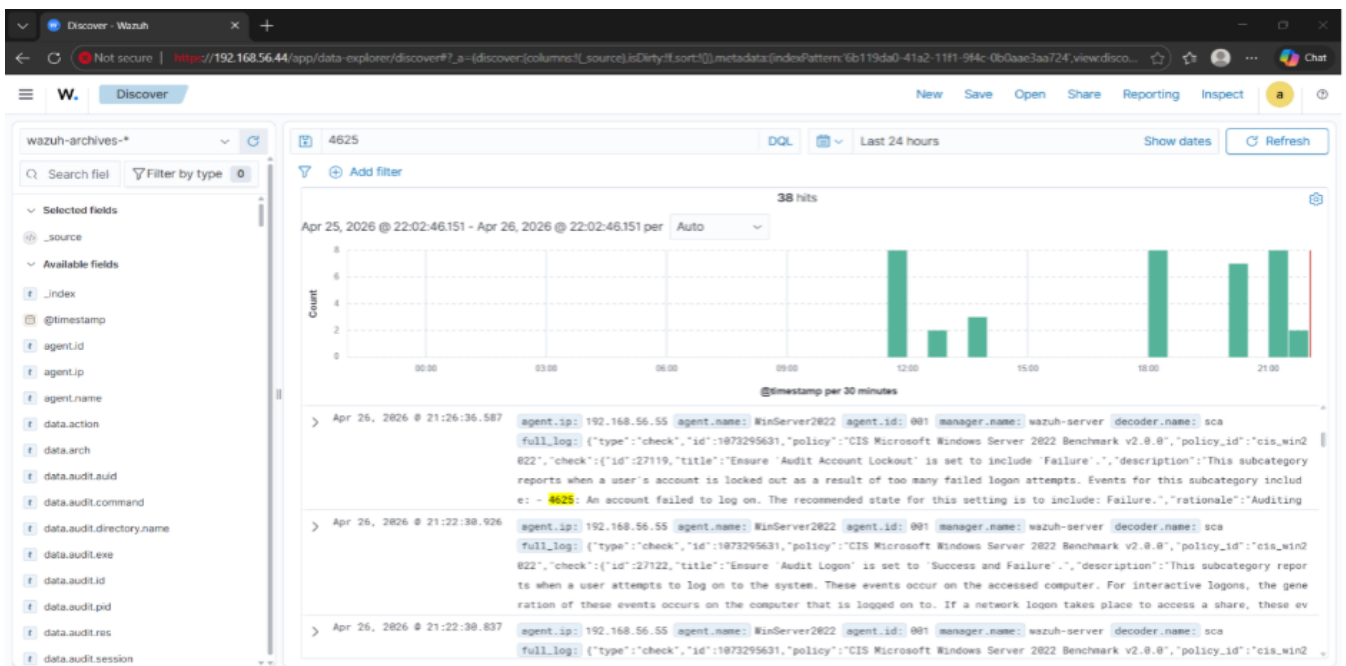


Figure 16: Shows 38 hits for Windows Event 4625.

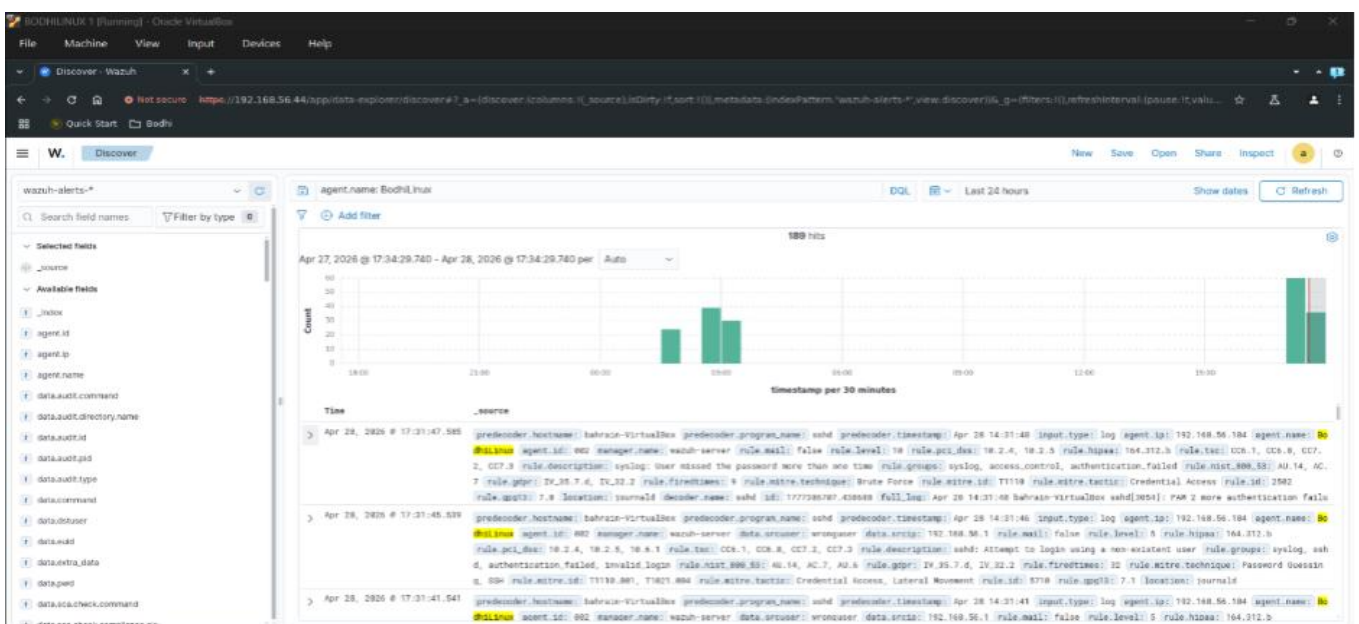


Figure 17: Shows BodhiLinux agent events in Wazuh discover with 93 hits.

timestamp per 30 minutes	
_index	wazuh-alerts-4.x-2026.04.28
agent.id	002
agent.ip	192.168.56.104
agent.name	BodhiLinux
decoder.name	sshd
full_log	Apr 28 14:31:48 bahrain-VirtualBox sshd[3054]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.1
id	1777386787.438688
input.type	log
location	journalid
manager.name	wazuh-server

Figure 18: Shows expanded Bodhi Linux SSH authentication failure event detail.

```

bahrain@bahrain-VirtualBox:~$ ls /var/log/snort
alert      snort.alert.fast  snort.log.1779115477
snort.alert snort.log         snort.log.1779116112
bahrain@bahrain-VirtualBox:~$ ls /var/log/snort
alert      snort.alert.fast  snort.log.1779115477  snort.log.1779116272
snort.alert snort.log         snort.log.1779116112
bahrain@bahrain-VirtualBox:~$ cat /var/log/snort/alert
bahrain@bahrain-VirtualBox:~$ ls -la /var/log/snort/
total 1156
drwxr-xr-x 2 snort adm  4096  18 17:59 .
drwxr-xr-x 8 root  sys  4096  18 17:08 ..
-rw-r--r-- 1 root  adm    0  18 12:55 alert
-rw-r--r-- 1 snort adm 16888  18 18:01 snort.alert
-rw-r--r-- 1 root  adm 178251  18 18:01 snort.alert.fast
-rw-r--r-- 1 snort adm  93562  18 18:01 snort.log
-rw-r--r-- 1 root  adm   760  18 12:55 snort.log.1779116112
-rw-r--r-- 1 root  adm 861752  18 18:02 snort.log.1779116272
bahrain@bahrain-VirtualBox:~$

```

Figure 19: Shows the snort log directory on Bodhi Linux confirming alert and packet capture files.

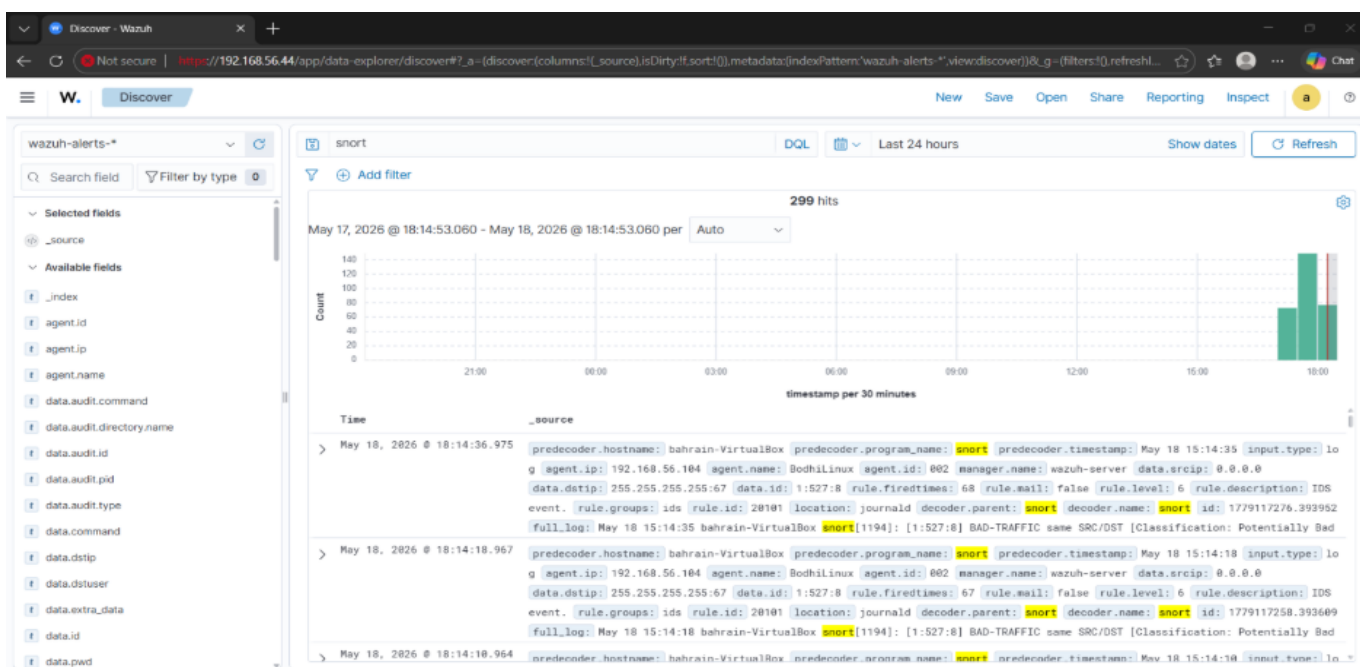


Figure 20: Shows Snort IDS alerts on wazuh discover with 299 hits.

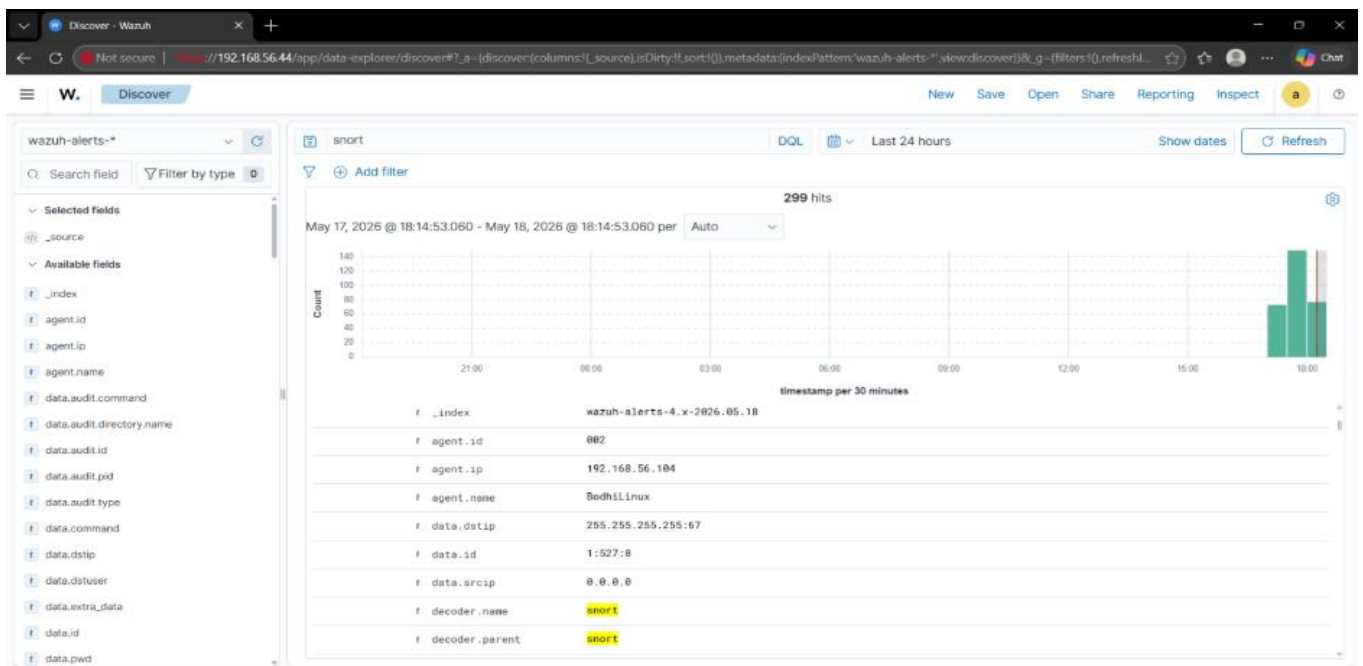


Figure 21: Shows a detailed snort IDS alert event confirming successful login into Wazuh.

4. Event Generation

Using Kali Linux, security events were generated against all three integrated systems to validate that Wazuh could detect and log a real attack activity such as ICMP ping sweeps, SSH brute force attempts, SMB credential attacks, and Nmap port scans.

```

kali@kali:~$ ping 192.168.56.44
PING 192.168.56.44 (192.168.56.44) 56(84) bytes of data:
64 bytes from 192.168.56.44: icmp_seq=1 ttl=127 time=1.02 ms
64 bytes from 192.168.56.44: icmp_seq=2 ttl=127 time=0.599 ms
64 bytes from 192.168.56.44: icmp_seq=3 ttl=127 time=0.451 ms
64 bytes from 192.168.56.44: icmp_seq=4 ttl=127 time=0.543 ms
^C
--- 192.168.56.44 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3152ms
rtt min/avg/max/mdev = 0.543/0.703/1.021/0.107 ms

kali@kali:~$ ping 192.168.56.55
PING 192.168.56.55 (192.168.56.55) 56(84) bytes of data:
64 bytes from 192.168.56.55: icmp_seq=1 ttl=120 time=2.07 ms
64 bytes from 192.168.56.55: icmp_seq=2 ttl=120 time=0.563 ms
64 bytes from 192.168.56.55: icmp_seq=3 ttl=120 time=0.528 ms
64 bytes from 192.168.56.55: icmp_seq=4 ttl=120 time=0.540 ms
^C
--- 192.168.56.55 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3063ms
rtt min/avg/max/mdev = 0.528/0.924/2.067/0.659 ms

kali@kali:~$ ping 192.168.56.104
PING 192.168.56.104 (192.168.56.104) 56(84) bytes of data:
64 bytes from 192.168.56.104: icmp_seq=1 ttl=64 time=0.507 ms
64 bytes from 192.168.56.104: icmp_seq=2 ttl=64 time=0.408 ms
64 bytes from 192.168.56.104: icmp_seq=3 ttl=64 time=1.28 ms
64 bytes from 192.168.56.104: icmp_seq=4 ttl=64 time=1.03 ms
^C
--- 192.168.56.104 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3720ms
rtt min/avg/max/mdev = 0.408/0.873/1.370/0.335 ms

kali@kali:~$

```

Figure 22: Kali Linux pinging all target systems confirming full network reachability.


```

kali@kali:~$ sudo apt autoremove --purge
libheif-providers libheif-thread-0.1 libheif10 libheif-dev libheif-dev10 libheif-dev10 libheif-dev10
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2154

kali@kali:~$ crackmapexec smb 192.168.56.55 --u Administrator -p wrongpassword
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] Windows Server 2022 Build 20348 x64 (name:WIN-SCUAEMQES6)
(domain:WIN-SCUAEMQES6) (signing:False) (smbr:False)
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] WIN-SCUAEMQES6\Administrator:wrongpassword STATUS_LOGON_FA
ILURE

kali@kali:~$ crackmapexec smb 192.168.56.55 --u Administrator -p wrongpassword
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] Windows Server 2022 Build 20348 x64 (name:WIN-SCUAEMQES6)
(domain:WIN-SCUAEMQES6) (signing:False) (smbr:False)
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] WIN-SCUAEMQES6\Administrator:wrongpassword STATUS_LOGON_FA
ILURE

kali@kali:~$ crackmapexec smb 192.168.56.55 --u Administrator -p wrongpassword
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] Windows Server 2022 Build 20348 x64 (name:WIN-SCUAEMQES6)
(domain:WIN-SCUAEMQES6) (signing:False) (smbr:False)
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] WIN-SCUAEMQES6\Administrator:wrongpassword STATUS_LOGON_FA
ILURE

kali@kali:~$ crackmapexec smb 192.168.56.55 --u Administrator -p wrongpassword
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] Windows Server 2022 Build 20348 x64 (name:WIN-SCUAEMQES6)
(domain:WIN-SCUAEMQES6) (signing:False) (smbr:False)
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] WIN-SCUAEMQES6\Administrator:wrongpassword STATUS_LOGON_FA
ILURE

kali@kali:~$ crackmapexec smb 192.168.56.55 --u Administrator -p wrongpassword
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] Windows Server 2022 Build 20348 x64 (name:WIN-SCUAEMQES6)
(domain:WIN-SCUAEMQES6) (signing:False) (smbr:False)
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] WIN-SCUAEMQES6\Administrator:wrongpassword STATUS_LOGON_FA
ILURE

kali@kali:~$ crackmapexec smb 192.168.56.55 --u Administrator -p wrongpassword
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] Windows Server 2022 Build 20348 x64 (name:WIN-SCUAEMQES6)
(domain:WIN-SCUAEMQES6) (signing:False) (smbr:False)
[*] 192.168.56.55 445 WIN-SCUAEMQES6 [!] WIN-SCUAEMQES6\Administrator:wrongpassword STATUS_LOGON_FA
ILURE

```

Figure 25: Shows failed SMB attempt against Windows server.

Task 2 - XDR Detection and Automated Response

Introduction

The current assignment involves the implementation of Extended Detection and Response (XDR) via Wazuh. There were three different cyber attacks that were performed and identified: SMB brute force attack, malware detection with VirusTotal and EICAR, and suspicious PowerShell command execution via Sysmon. Detection for each one was set up, events were identified, and the response was implemented successfully and proved by evidence.

Implementation Summary

Attack	Detection Rule	Logs / Event IDs	Response Action	Result
Attack 1: SMB Brute Force.	Custom rule Wazuh rule 100100 derived from failed login rule 60122.	Event ID 4625 for Windows Security ; Source IP is 192.168.56.105, Target User Administrator.	Block Windows Firewall inbound for Kali IP for ICMP/SMB	Resultant: Attacker traffic blocked; Ping responded to 100% packet loss on SMB port 445.
Attack 2: Malware / EICAR with VirusTotal	Rule FIM 554, Rule VirusTotal 87105, Result rule Active Response 100092, Deletion rule 553	FIM File Added rule 554; VirusTotal Malicious Detection rule 87105; Active Response rule 100092; File Deleted rule 553	remove-threat.exe Active Response removed the detected file	Successful: Test-Path returned False for eicar.txt
Attack 3: Suspicious PowerShell	Wazuh custom rule 100900 for Sysmon event id 1 with PowerShell -nop parameter	Sysmon event id 1 ; Image and command line fields of powershell	Block from Windows firewall outbound connection to powershell.exe	Successful : Test-NetConnection reported TcpTestSucceeded

Document Structure

The following attacks consist of all three completed attack packages. The attack package contains information on threat simulation, rule set up, Wazuh dashboard response evidence, logs/event ID numbers, response set up, and evidence of validation to follow the Task 2 rubric.

Attack 1 Evidence Package: SMB Brute Force Detection and IP Blocking

Objectives: In this part, an example of an attack through XDR will be presented in Wazuh. A brute-force attack will be created using Kali Linux towards Windows Server 2022 through SMB authentications. The detection of repeated failed logins is made through a rule on Wazuh, while the blocking is done through the Windows Firewall.

Attack 1 Rubric Mapping

Rubric Requirement	Evidence in This Section	Key Screenshot(s)	Status
Threat Simulation & Detection.	SMB login failures were created by Kali and Wazuh created a custom event with ID 100100.	Figures 32-33	Successful
Detection Rule Configuration	The Wazuh rule with ID 100100 was set up on local_rules.xml.	Figure 30	Successful
Logs / Event IDs	Windows Security Log event ID 4625, source 192.168.56.105, and targeted username Administrator.	Figures 34-36	Successful
Response Configuration	Windows firewall inbound rules filtered traffic from Kali to Windows SMB port TCP/445 and ICMP.	Figure 37	Successful
Result Validation	Kali ping resulted in 100% packet loss; Nmap reported TCP/445 filtered.	Figures 38-39	Successful

Environment and Agent Identification

Purpose: The name of the Windows Wazuh agent has been changed to incorporate the student ID such that the Wazuh dashboard proof will belong to this implementation. The old disconnected agent should be disregarded since the new one is registered.

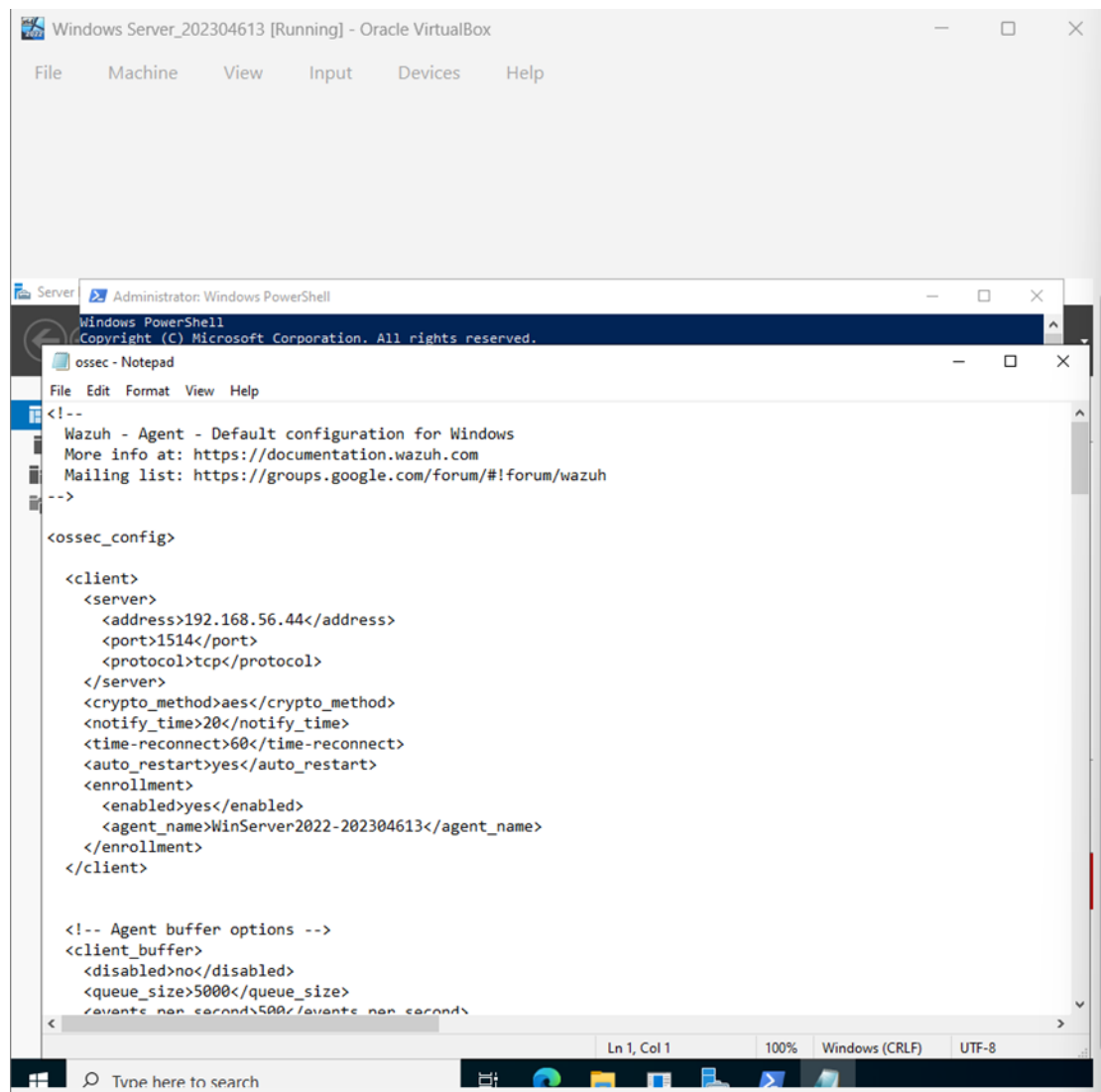


Figure 26: Windows Wazuh agent configuration showing agent_name WinServer2022-202304613.

```

[wazuh-user@wazuh-server ~]$ sudo -i
[root@wazuh-server ~]# sudo systemctl restart wazuh-manager
[root@wazuh-server ~]# sudo systemctl status wazuh-manager
■ wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-05-16 13:30:11 UTC; 9s ago
     Process: 5558 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 232 (limit: 9464)
   Memory: 257.7M
      CPU: 20.294s
   CGroup: /system.slice/wazuh-manager.service
           └─5622 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             5623 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             5624 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             5627 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             5630 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             5654 /var/ossec/bin/wazuh-integratord
             5676 /var/ossec/bin/wazuh-authd
             5691 /var/ossec/bin/wazuh-db
             5731 /var/ossec/bin/wazuh-execd
             5743 /var/ossec/bin/wazuh-analysisd
             5760 /var/ossec/bin/wazuh-syscheckd
             5774 /var/ossec/bin/wazuh-remoted
             5775 /var/ossec/bin/wazuh-remoted
             5815 /var/ossec/bin/wazuh-logcollector
             5833 /var/ossec/bin/wazuh-monitord
             5877 /var/ossec/bin/wazuh-modulesd

May 16 13:30:06 wazuh-server env[5558]: Started wazuh-analysisd...
May 16 13:30:06 wazuh-server env[5558]: Started wazuh-syscheckd...
May 16 13:30:07 wazuh-server env[5558]: Started wazuh-remoted...
May 16 13:30:07 wazuh-server env[5558]: Started wazuh-logcollector...
May 16 13:30:08 wazuh-server env[5558]: Started wazuh-monitord...
May 16 13:30:08 wazuh-server env[5875]: 2026/05/16 13:30:08 wazuh-modulesd:router: INFO: Loaded router module.
May 16 13:30:08 wazuh-server env[5875]: 2026/05/16 13:30:08 wazuh-modulesd:content_manager: INFO: Loaded content_manag
May 16 13:30:08 wazuh-server env[5875]: 2026/05/16 13:30:08 wazuh-modulesd:inventory-harvester: INFO: Loaded inventory
May 16 13:30:09 wazuh-server env[5558]: Started wazuh-modulesd...
May 16 13:30:11 wazuh-server env[5558]: Completed.
[root@wazuh-server ~]# sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: wazuh-server (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: WinServer2022, IP: any, Disconnected
  ID: 002, Name: BodhiLinux, IP: any, Disconnected
  ID: 003, Name: WinServer2022-202304613, IP: any, Active

List of agentless devices:
[root@wazuh-server ~]#

```

Figure 27: Wazuh server agent list showing WinServer2022-202304613 as an active agent.

IP Address Verification

Windows Server acted as the target machine, while Kali Linux was the machine used to carry out the attack. The connection between the two systems was established prior to performing the brute force attack.

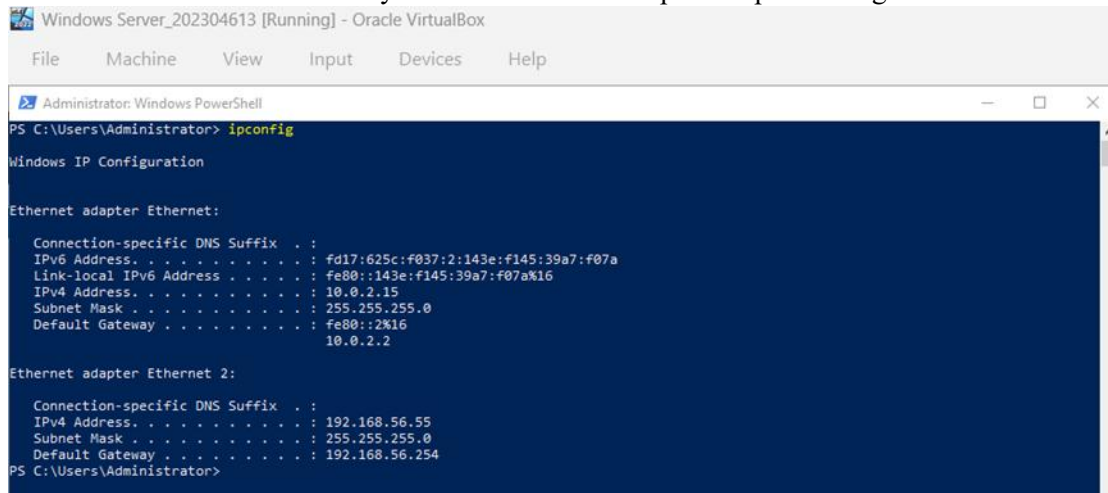


Figure 28: Windows Server target IP address 192.168.56.55.

```
KALI_202304613 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
File Actions Edit View Help

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:f5:46:5a brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic eth0
       valid_lft 86234sec preferred_lft 86234sec
   inet6 fd17:625c:f037:2:a00:27ff:fef5:465a/64 scope global dynamic mngtmpaddr proto kernel_ra
       valid_lft 86235sec preferred_lft 14235sec
   inet6 fe80::a00:27ff:fef5:465a/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:17:f6:86 brd ff:ff:ff:ff:ff:ff
   inet 192.168.56.105/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
       valid_lft 434sec preferred_lft 434sec
   inet6 fe80::fef0:580e:a361:40c0/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

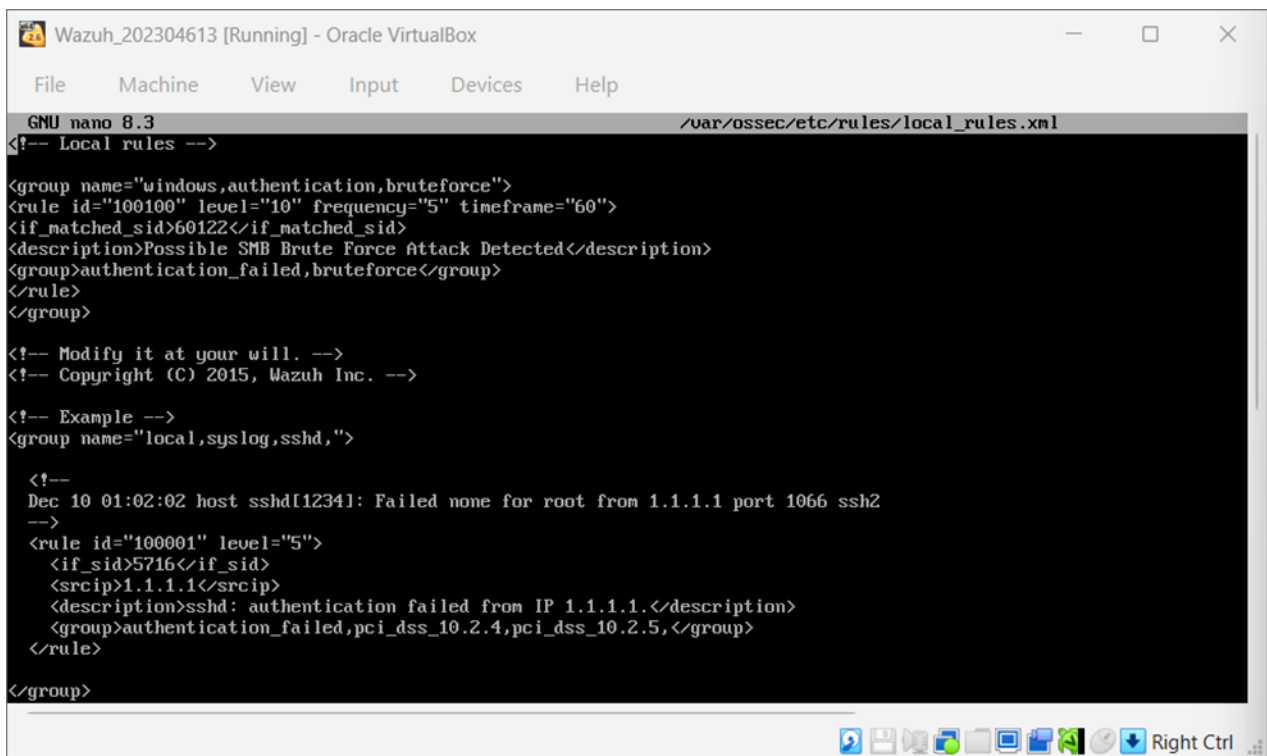
(kali@kali)-[~]
$ ping -c 4 192.168.56.55
PING 192.168.56.55 (192.168.56.55) 56(84) bytes of data.
64 bytes from 192.168.56.55: icmp_seq=1 ttl=128 time=1.93 ms
64 bytes from 192.168.56.55: icmp_seq=2 ttl=128 time=0.398 ms
64 bytes from 192.168.56.55: icmp_seq=3 ttl=128 time=0.674 ms
64 bytes from 192.168.56.55: icmp_seq=4 ttl=128 time=1.33 ms

— 192.168.56.55 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3097ms
rtt min/avg/max/mdev = 0.398/1.081/1.927/0.593 ms
```

Figure 29: Kali Linux IP address and successful connectivity test to the Windows Server before the attack.

Detection Rule Configuration Evidence

Rule used: The custom Wazuh rule with the ID 100100 was customized to detect multiple occurrences of Windows failed logon attempts. The alert message is ("Possible SMB Brute Force Attack Detected"), while the alert level is 10.



```
GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml
<!-- Local rules -->

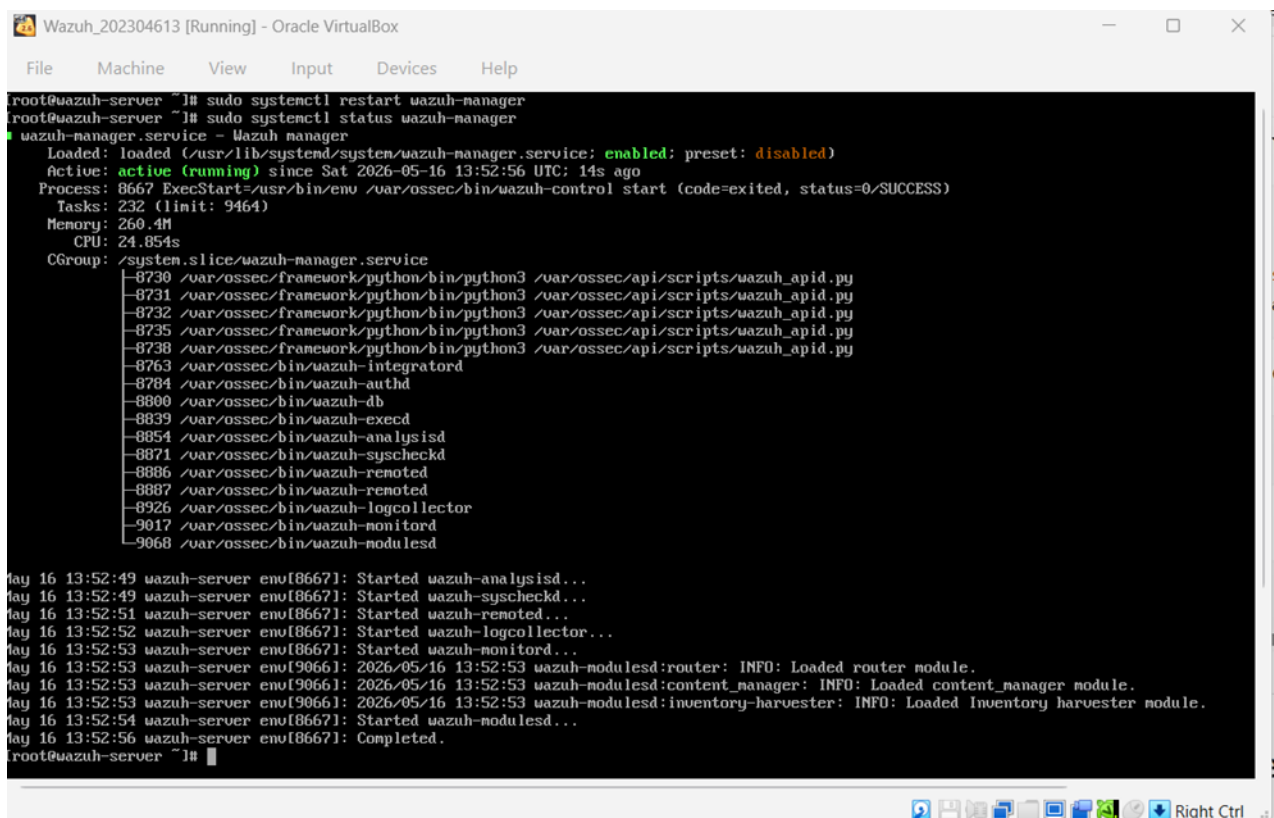
<group name="windows,authentication,bruteforce">
<rule id="100100" level="10" frequency="5" timeframe="60">
<if_matched_sid>60122</if_matched_sid>
<description>Possible SMB Brute Force Attack Detected</description>
<group>authentication_failed,bruteforce</group>
</rule>
</group>

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>
```

Figure 30: Custom Wazuh rule 100100 configured in local_rules.xml for SMB brute force detection.



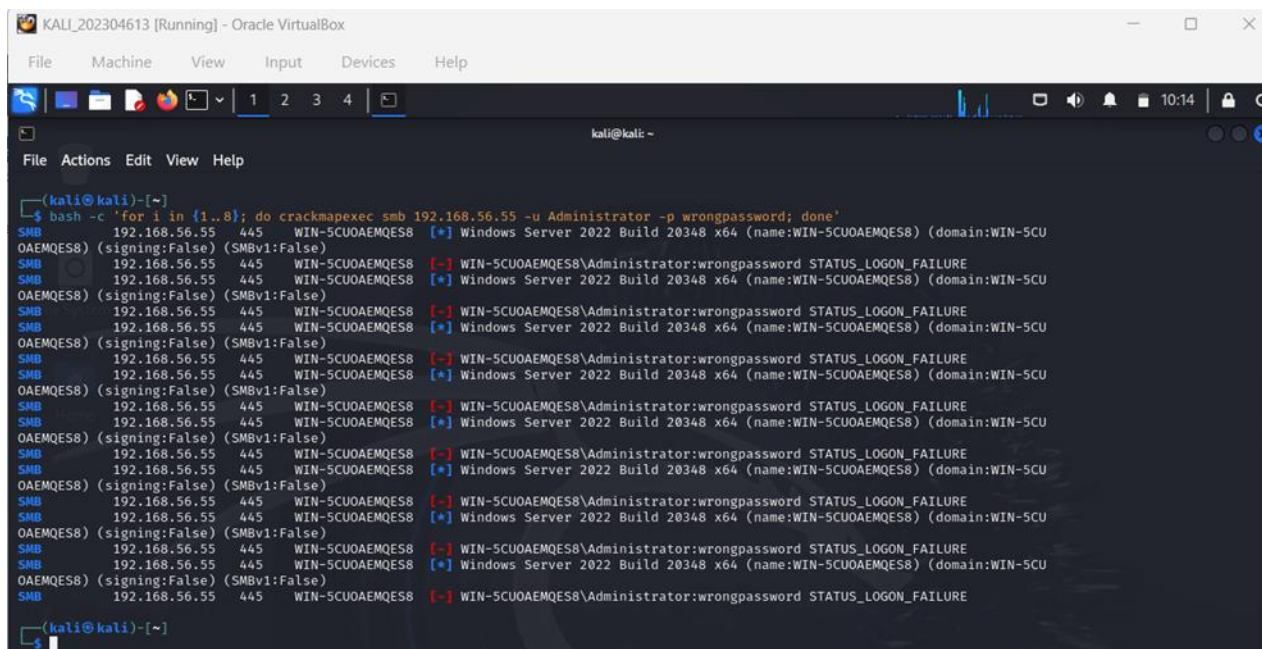
```
root@wazuh-server ~# sudo systemctl restart wazuh-manager
root@wazuh-server ~# sudo systemctl status wazuh-manager
wazuh-manager.service - Wazuh manager
Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
Active: active (running) since Sat 2026-05-16 13:52:56 UTC; 14s ago
Process: 8667 ExecStart=/usr/bin/envu /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
Tasks: 232 (limit: 9464)
Memory: 260.4M
CPU: 24.854s
CGroup: /system.slice/wazuh-manager.service
├─8730 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─8731 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─8732 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─8735 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─8738 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─8763 /var/ossec/bin/wazuh-integrator
├─8784 /var/ossec/bin/wazuh-authd
├─8800 /var/ossec/bin/wazuh-db
├─8839 /var/ossec/bin/wazuh-execd
├─8854 /var/ossec/bin/wazuh-analysisd
├─8871 /var/ossec/bin/wazuh-syscheckd
├─8886 /var/ossec/bin/wazuh-remoted
├─8887 /var/ossec/bin/wazuh-remoted
├─8926 /var/ossec/bin/wazuh-logcollector
├─9017 /var/ossec/bin/wazuh-monitord
└─9068 /var/ossec/bin/wazuh-modulesd

May 16 13:52:49 wazuh-server envu[8667]: Started wazuh-analysisd...
May 16 13:52:49 wazuh-server envu[8667]: Started wazuh-syscheckd...
May 16 13:52:51 wazuh-server envu[8667]: Started wazuh-remoted...
May 16 13:52:52 wazuh-server envu[8667]: Started wazuh-logcollector...
May 16 13:52:53 wazuh-server envu[8667]: Started wazuh-monitord...
May 16 13:52:53 wazuh-server envu[9066]: 2026/05/16 13:52:53 wazuh-modulesd:router: INFO: Loaded router module.
May 16 13:52:53 wazuh-server envu[9066]: 2026/05/16 13:52:53 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
May 16 13:52:53 wazuh-server envu[9066]: 2026/05/16 13:52:53 wazuh-modulesd:inventory-harvester: INFO: Loaded inventory harvester module.
May 16 13:52:54 wazuh-server envu[8667]: Started wazuh-modulesd...
May 16 13:52:56 wazuh-server envu[8667]: Completed.
root@wazuh-server ~#
```

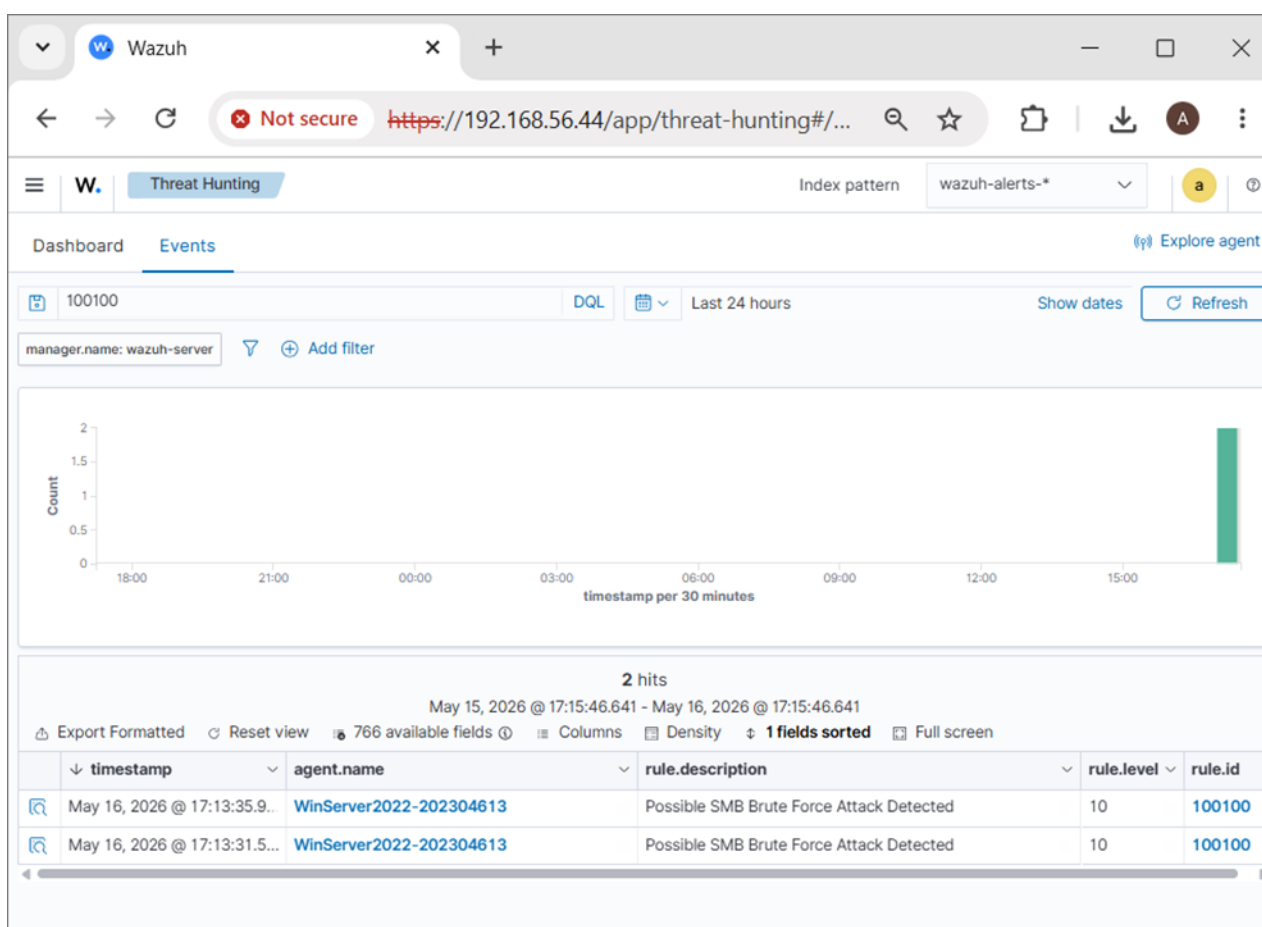
Figure 31: Wazuh manager running successfully after rule configuration.

Threat Simulation Evidence

Tool for attack: CrackMapExec from Kali Linux was used to cause multiple instances of failed SMB authentications against the Windows Server. The image below confirms that STATUS_LOGON_FAILURE appeared in the output due to the attack.



Wazuh Dashboard Response Evidence



Logs and Event ID Identification

The Wazuh document contains the details that contain the relevant Windows failed logon details. The fields confirm the source of the attack, target accounts, and Windows Security event id.

Field	Value / Evidence
Custom rule ID	100100
Alert description	Possible SMB Brute Force Attack Detected
Agent name	WinServer2022-202304613
Target IP address	192.168.56.55
Source / attacker IP address	192.168.56.105
Target user	Administrator
Relevant Windows event ID	4625 - failed logon
Response action	Block Kali IP using Windows Firewall

The screenshot displays the Wazuh Threat Hunting interface. On the left, a dashboard shows a chart and a table of events. The table lists two events from May 16, 2026, at 17:13:35.9, both identified as 'Possible SMB Brute Force Attack Detected' for agent 'WinServer2022-202304613'. The right pane, titled 'Document Details', shows the JSON structure of the selected event. Key fields include 'agent.ip' (192.168.56.55), 'agent.name' (WinServer2022-202304613), 'data.win.eventdata.ipAddress' (192.168.56.105), 'data.win.eventdata.ipPort' (49318), 'data.win.eventdata.subjectName' (Administrator), and 'data.win.eventdata.targetName' (Administrator).

Field	Value
index	wazuh-alerts-4.x-2026.05.16
agent.id	003
agent.ip	192.168.56.55
agent.name	WinServer2022-202304613
data.win.eventdata.authenticationPackageName	NTLM
data.win.eventdata.failureReason	%2313
data.win.eventdata.ipAddress	192.168.56.105
data.win.eventdata.ipPort	49318
data.win.eventdata.keyLength	0
data.win.eventdata.logonProcessName	NTLSSp
data.win.eventdata.logonType	3
data.win.eventdata.processId	0x0
data.win.eventdata.status	0xc000006d
data.win.eventdata.subStatus	0xc000006a
data.win.eventdata.subjectId	0x0
data.win.eventdata.subjectName	S-1-0-0
data.win.eventdata.targetDomainName	WIN-SQUAENQES8
data.win.eventdata.targetName	Administrator
data.win.eventdata.targetUser	S-1-0-0

Figure 34: Wazuh document details showing the attacker IP address 192.168.56.105 and target user Administrator.

Wazuh Threat Hunting

Dashboard Events

100100 DQL

manager.name: wazuh-server agent.ip: 192.168.56.55

Count

timestamp

May 15, 2026 @ 17:30:46.981

Export Formatted Reset view 766 available fields

Full screen

timestamp	agent.name
May 16, 2026 @ 17:13:35.9...	WinServer2022-202304613
May 16, 2026 @ 17:13:31.5...	WinServer2022-202304613

Document Details

View surrounding documents View single document

data.win.system.channel	Security
data.win.system.computer	WIN-5CUOAEMQES8
data.win.system.eventID	4625
data.win.system.eventRecordID	18325
data.win.system.keywords	0x8010000000000000
data.win.system.level	0
data.win.system.message	"An account failed to log on."
Subject:	Security
ID:	-1-A-A
data.win.system.opcode	0
data.win.system.processID	604
data.win.system.providerGuid	{54849625-5478-4994-a5ba-3e3b0328c30d}
data.win.system.providerName	Microsoft-Windows-Security-Auditing
data.win.system.severityValue	AUDIT_FAILURE
data.win.system.systemTime	2026-05-16T14:13:39.1445456Z
data.win.system.task	12544
data.win.system.threadID	4592
data.win.system.version	0
decoder.name	windows_eventchannel
full_log	{ "win": { "system": { "providerName": "Microsoft-Windows-Security-Auditing" } } }

Figure 35: Wazuh document details showing Windows Security Event ID 4625 for failed logon activity.

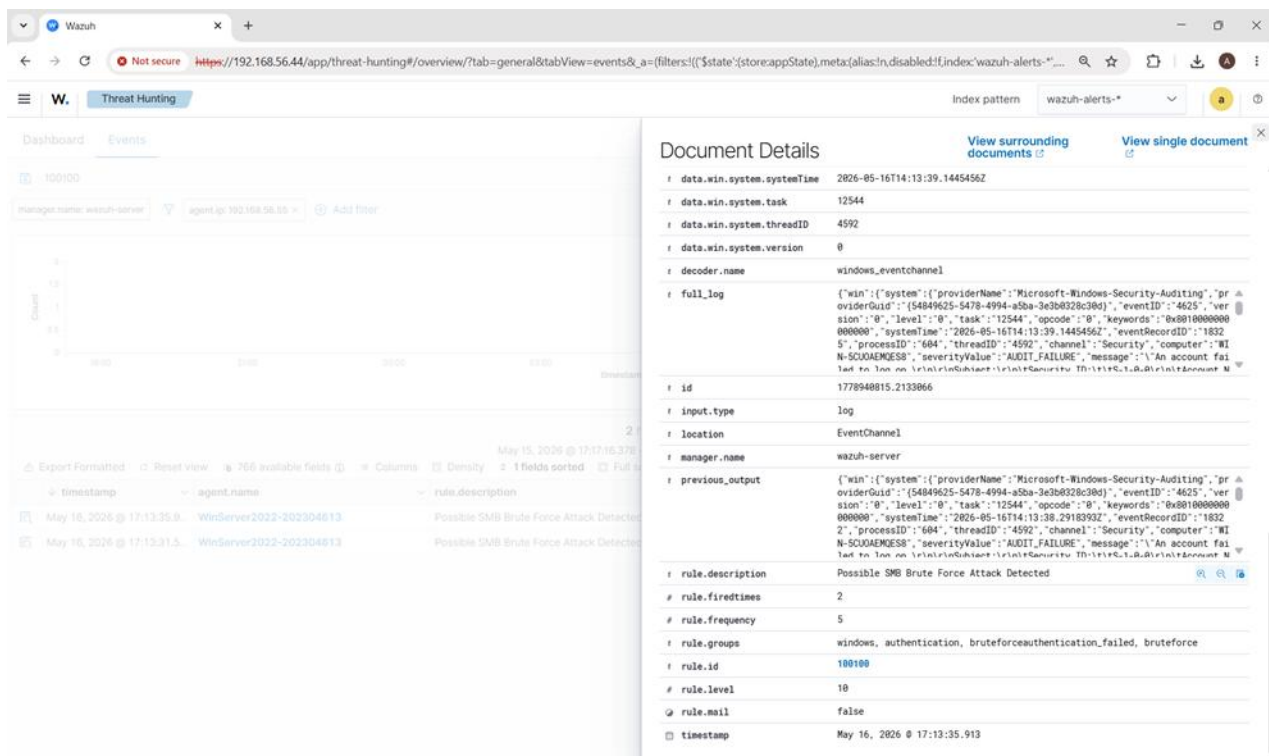
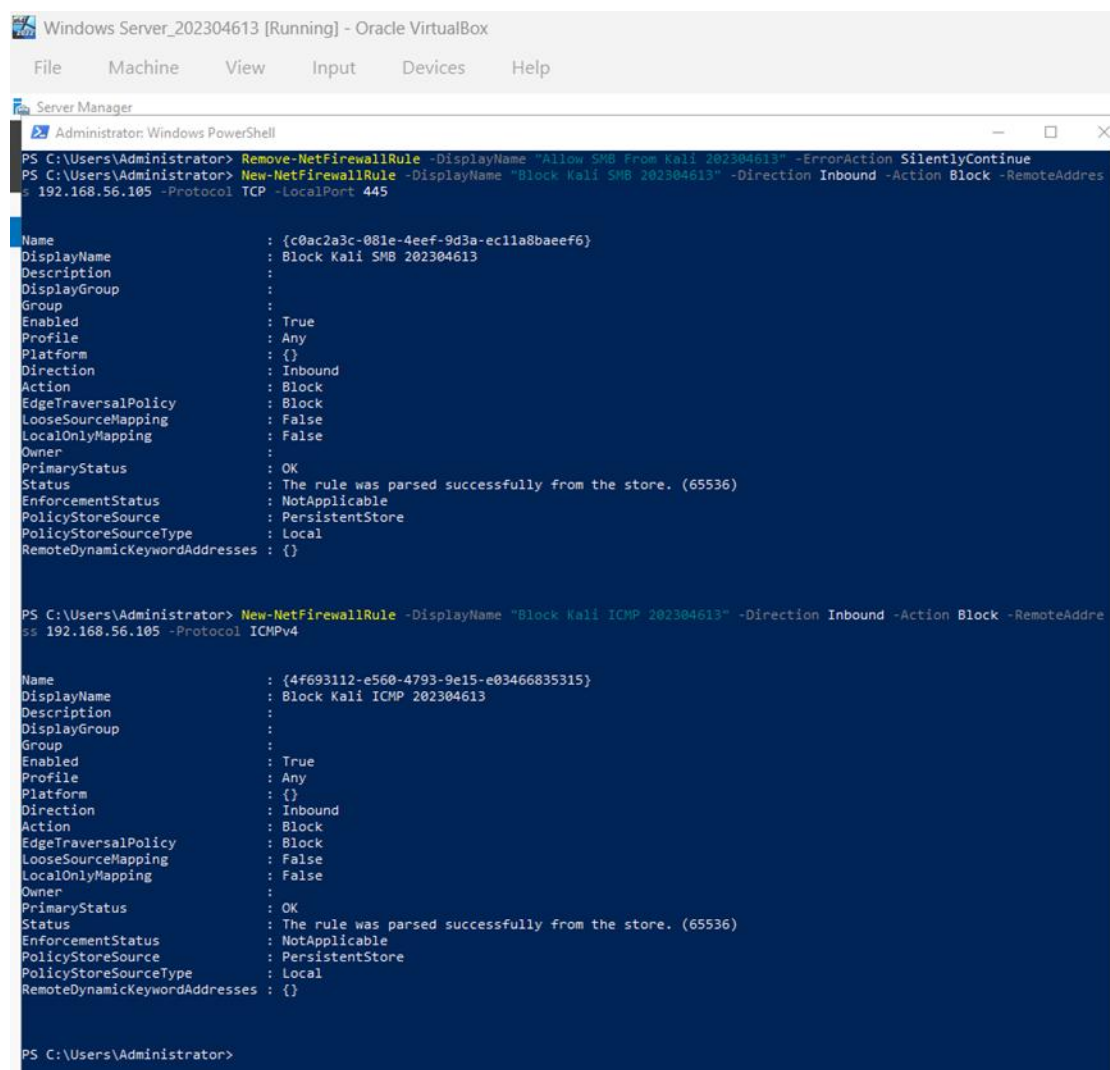


Figure 36: Wazuh document details showing custom rule ID 100100, alert level 10, and the brute force alert description.

Response Configuration Evidence

Response action: Once the alert was confirmed in Wazuh, firewall rules were set up for blocking the attacker IP address 192.168.56.105 on both SMB and ICMP ports. The above action is an example of containing the attack machine.

A screenshot of a Windows Server 2023 virtual machine running in Oracle VM VirtualBox. The window title is "Windows Server_202304613 [Running] - Oracle VirtualBox". The menu bar includes File, Machine, View, Input, Devices, and Help. Below the menu is the "Server Manager" tab. A PowerShell window is open, showing the configuration of two Windows Firewall rules. The first rule is "Block Kali SMB 202304613" with ID {c0ac2a3c-081e-4eef-9d3a-ec11a8baeef6}, blocking inbound TCP traffic on port 445 from 192.168.56.105. The second rule is "Block Kali ICMP 202304613" with ID {4f693112-e560-4793-9e15-e03466835315}, blocking inbound ICMPv4 traffic from 192.168.56.105. Both rules are enabled and have a status of "The rule was parsed successfully from the store. (65536)".

```
PS C:\Users\Administrator> Remove-NetFirewallRule -DisplayName "Allow SMB From Kali 202304613" -ErrorAction SilentlyContinue
PS C:\Users\Administrator> New-NetFirewallRule -DisplayName "Block Kali SMB 202304613" -Direction Inbound -Action Block -RemoteAddress 192.168.56.105 -Protocol TCP -LocalPort 445

Name                                     : {c0ac2a3c-081e-4eef-9d3a-ec11a8baeef6}
DisplayName                             : Block Kali SMB 202304613
Description                             :
DisplayGroup                             :
Group                                    :
Enabled                                 : True
Profile                                  : Any
Platform                                : {}
Direction                               : Inbound
Action                                   : Block
EdgeTraversalPolicy                     : Block
LooseSourceMapping                       : False
LocalOnlyMapping                         : False
Owner                                    :
PrimaryStatus                           : OK
Status                                  : The rule was parsed successfully from the store. (65536)
EnforcementStatus                       : NotApplicable
PolicyStoreSource                       : PersistentStore
PolicyStoreSourceType                   : Local
RemoteDynamicKeywordAddresses           : {}

PS C:\Users\Administrator> New-NetFirewallRule -DisplayName "Block Kali ICMP 202304613" -Direction Inbound -Action Block -RemoteAddress 192.168.56.105 -Protocol ICMPv4

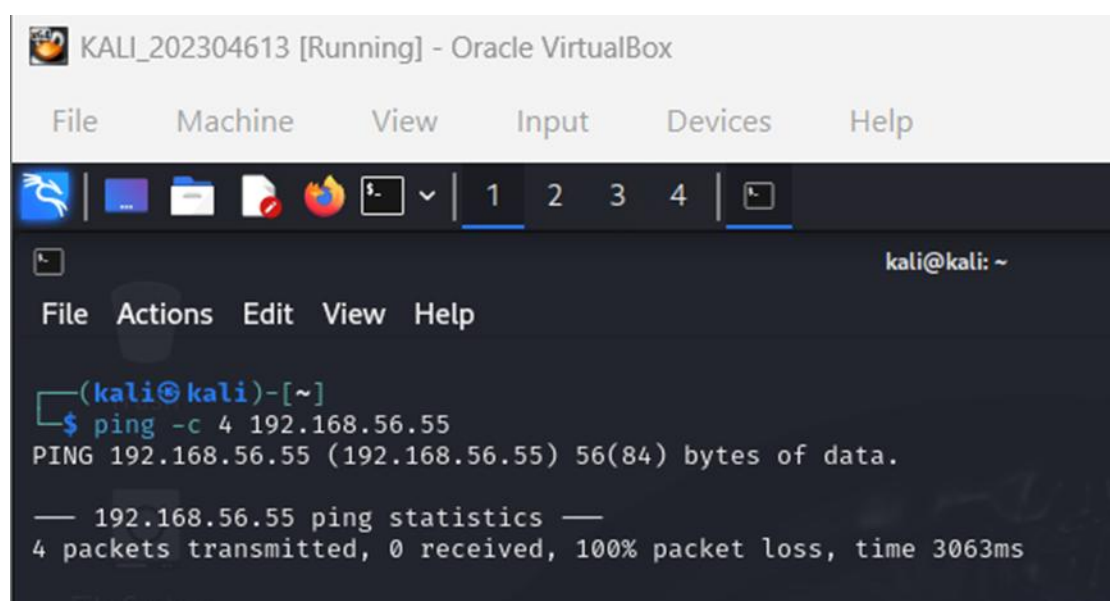
Name                                     : {4f693112-e560-4793-9e15-e03466835315}
DisplayName                             : Block Kali ICMP 202304613
Description                             :
DisplayGroup                             :
Group                                    :
Enabled                                 : True
Profile                                  : Any
Platform                                : {}
Direction                               : Inbound
Action                                   : Block
EdgeTraversalPolicy                     : Block
LooseSourceMapping                       : False
LocalOnlyMapping                         : False
Owner                                    :
PrimaryStatus                           : OK
Status                                  : The rule was parsed successfully from the store. (65536)
EnforcementStatus                       : NotApplicable
PolicyStoreSource                       : PersistentStore
PolicyStoreSourceType                   : Local
RemoteDynamicKeywordAddresses           : {}

PS C:\Users\Administrator>
```

Figure 37: Windows Firewall response rules configured to block Kali IP 192.168.56.105 for SMB and ICMP traffic.

Result Validation Evidence

The response was authenticated using Kali Linux. The ICMP traffic resulted in 100% packet loss, while Nmap showed the SMB port 445 to be filtered post-firewall block.

A screenshot of a Kali Linux virtual machine running in Oracle VM VirtualBox. The window title is "KALI_202304613 [Running] - Oracle VirtualBox". The menu bar includes File, Machine, View, Input, Devices, and Help. Below the menu is a taskbar with icons for a terminal, a file manager, a web browser, and a terminal. The terminal window is open, showing the command "ping -c 4 192.168.56.55" and the output "PING 192.168.56.55 (192.168.56.55) 56(84) bytes of data. — 192.168.56.55 ping statistics — 4 packets transmitted, 0 received, 100% packet loss, time 3063ms".

```
KALI_202304613 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
File Actions Edit View Help
(kali@kali)-[~]
$ ping -c 4 192.168.56.55
PING 192.168.56.55 (192.168.56.55) 56(84) bytes of data.
— 192.168.56.55 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3063ms
```

Figure 38: Kali validation showing 100% packet loss after the firewall block response.

```

(kali@kali)-[~]
$ ping -c 4 192.168.56.55
PING 192.168.56.55 (192.168.56.55) 56(84) bytes of data.
— 192.168.56.55 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3063ms

(kali@kali)-[~]
$ sudo nmap -p 445 192.168.56.55
[sudo] password for kali:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-16 10:26 EDT
Nmap scan report for 192.168.56.55
Host is up (0.0045s latency).

PORT      STATE      SERVICE
445/tcp    filtered  microsoft-ds
MAC Address: 08:00:27:50:77:07 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.54 seconds

```

Figure 39: Nmap validation showing SMB port 445 filtered after the firewall response was applied.

Attack 1 Result

Result: Successful. Kali Linux made repeated failed attempts at authenticating via SMB with Windows Server 2022. The attempt was detected by Wazuh based on customized rule ID 100100, where the alert was shown on the dashboard for the renamed agent named WinServer2022-202304613. The document provided the confirmed Windows Security Event ID 4625, the attacking IP 192.168.56.105, and the attacked user Administrator. The response was implemented via Windows Firewall rule blocking access to SMB and ICMP ports from Kali. The response was confirmed successful due to 100% packet loss via Kali ping and TCP/445 filtered by Nmap.

Attack 1 Deliverable Checklist

- Screenshot demonstrating attack detection on the Wazuh dashboard: done.
- Screenshot/config demonstrating rule configuration: done, utilizing local_rules.xml file and rule ID 100100.
- Relevant logs/events IDs: done, utilizing Windows Security Event ID 4625 and document detail information.
- Screenshot/config demonstrating rule response configuration: done, utilizing Windows Firewall blocking rules.
- Successful result validation: completed using ping 100% packet loss and Nmap TCP/445 filtered evidence.

Attack 2: Malware Activity Detection and Automated Response (EICAR + VirusTotal)

Attack Scenario	Malware Activity using the EICAR test file
Endpoint / Agent	Windows Server 2022 - WinServer2022-202304613

Detection Tools	Wazuh File Integrity Monitoring (FIM), VirusTotal integration
Response Action	Wazuh Active Response executed remove-threat.exe and removed eicar.txt

1. Objective

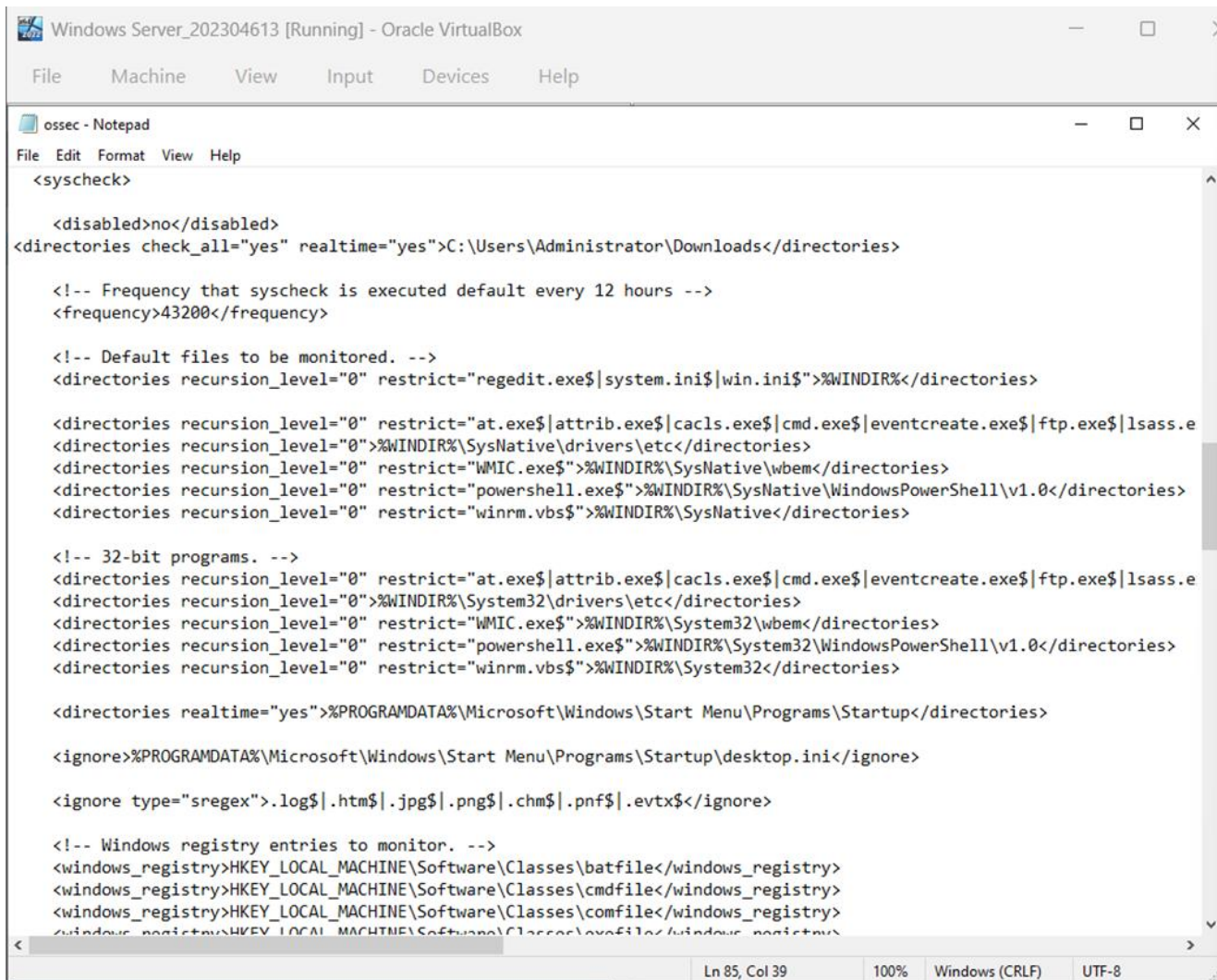
This attack demonstrates Extended Detection and Response (XDR) using Wazuh. A safe EICAR test file was downloaded to the monitored Windows Downloads directory to simulate malware activity. Wazuh detected the new file through File Integrity Monitoring, enriched the alert with VirusTotal, and executed an active response to remove the file.

2. Rubric Mapping for Attack 2

Requirement	Evidence Provided	Rule / Event IDs	Status
Threat Simulation & Detection	EICAR file downloaded into the monitored Downloads directory; Wazuh FIM detected file creation.	554, 87105	Successful
Detection Rule Configuration	Windows FIM directory monitoring, VirusTotal integration, and custom Active Response result rule configured.	554, 87105, 100092	Successful
Logs / Event IDs	Wazuh dashboard and Document Details show file creation, VirusTotal malicious verdict, Active Response execution, and deletion.	554, 87105, 100092, 553	Successful
Response Configuration	remove-threat.exe configured as a Wazuh Active Response command triggered by VirusTotal rule 87105.	87105 -> 100092	Successful
Result Validation	Dashboard shows response execution and file deletion; PowerShell Test-Path returned False.	100092, 553	Successful

3. Configuration Evidence

The following screenshots provide rule and response configuration evidence. They are included as images to meet the feedback requirement that the configuration/command pieces must appear as screenshots, not only as plain text.



```
<syscheck>

  <disabled>no</disabled>
<directories check_all="yes" realtime="yes">C:\Users\Administrator\Downloads</directories>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <!-- Default files to be monitored. -->
  <directories recursion_level="0" restrict="regedit.exe$|system.ini$|win.ini$" >%WINDIR%</directories>

  <directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|ftp.exe$|lsass.e
  <directories recursion_level="0" >%WINDIR%\SysNative\drivers\etc</directories>
  <directories recursion_level="0" restrict="WMIC.exe$" >%WINDIR%\SysNative\wbem</directories>
  <directories recursion_level="0" restrict="powershell.exe$" >%WINDIR%\SysNative\WindowsPowerShell\v1.0</directories>
  <directories recursion_level="0" restrict="winrm.vbs$" >%WINDIR%\SysNative</directories>

  <!-- 32-bit programs. -->
  <directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|ftp.exe$|lsass.e
  <directories recursion_level="0" >%WINDIR%\System32\drivers\etc</directories>
  <directories recursion_level="0" restrict="WMIC.exe$" >%WINDIR%\System32\wbem</directories>
  <directories recursion_level="0" restrict="powershell.exe$" >%WINDIR%\System32\WindowsPowerShell\v1.0</directories>
  <directories recursion_level="0" restrict="winrm.vbs$" >%WINDIR%\System32</directories>

  <directories realtime="yes" >%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup</directories>

  <ignore>%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup\desktop.ini</ignore>

  <ignore type="sregex">*.log$|.htm$|.jpg$|.png$|.chm$|.pnf$|.evtx$</ignore>

  <!-- Windows registry entries to monitor. -->
  <windows_registry>HKEY_LOCAL_MACHINE\Software\Classes\batfile</windows_registry>
  <windows_registry>HKEY_LOCAL_MACHINE\Software\Classes\cmdfile</windows_registry>
  <windows_registry>HKEY_LOCAL_MACHINE\Software\Classes\comfile</windows_registry>
  <windows_registry>HKEY_LOCAL_MACHINE\Software\Classes\exefile</windows_registry>
```

Figure 40: Windows Wazuh agent FIM configuration monitoring C:\Users\Administrator\Downloads in real time.

```
root@wazuh-server:~  
GNU nano 8.3 /var/ossec/etc/ossec.conf  
</localfile>  
  
<localfile>  
  <log_format>audit</log_format>  
  <location>/var/log/audit/audit.log</location>  
</localfile>  
  
<localfile>  
  <log_format>syslog</log_format>  
  <location>/var/ossec/logs/active-responses.log</location>  
</localfile>  
  
<command>  
  <name>remove-threat</name>  
  <executable>remove-threat.exe</executable>  
  <timeout_allowed>no</timeout_allowed>  
</command>  
  
<active-response>  
  <disabled>no</disabled>  
  <command>remove-threat</command>  
  <location>local</location>  
  <rules_id>87105</rules_id>  
</active-response>  
  
<integration>  
  <name>virustotal</name>  
  <api_key>dec  <api_key>REDACTED</api_key>  
  <group>sysch  
  <alert_format>json</alert_format>  
</integration>
```

Figure 41: Wazuh manager configuration showing VirusTotal integration, remove-threat command, and Active Response triggered by rule ID 87105. The API key is hidden.

```
root@wazuh-server:~  
GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml  
<!-- Local rules -->  
  
<group name="windows,authentication,bruteforce">  
  <rule id="100100" level="10" frequency="5" timeframe="60">  
    <if_matched_sid>60122</if_matched_sid>  
    <description>Possible SMB Brute Force Attack Detected</description>  
    <group>authentication_failed,bruteforce</group>  
  </rule>  
</group>  
<!-- Modify it at your will. -->  
<!-- Copyright (C) 2015, Wazuh Inc. -->  
  
<!-- Example -->  
<group name="local,syslog,sshd,">  
  
  <!--  
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ss  
  -->  
  <rule id="100001" level="5">  
    <if_sid>5716</if_sid>  
    <srcip>1.1.1.1</srcip>  
    <description>sshd: authentication failed from IP 1.1.1.1.</description>  
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>  
  </rule>  
  
</group>  
  
<group name="virustotal,">  
  <rule id="100092" level="12">  
    <if_sid>657</if_sid>  
    <match>Successfully removed threat</match>  
    <description>$(parameters.program) removed threat located at $(parameters.a  
  </rule>
```

Figure 42: local_rules.xml showing custom rule ID 100092 for a successful Active Response threat removal event.

```

nameserver 8.8.8.8
[root@wazuh-server ~]# ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=24.5 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=19.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=18.6 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 18.561/20.985/24.457/2.518 ms
[root@wazuh-server ~]# ping -c 3 google.com
PING google.com (142.250.202.78) 56(84) bytes of data.
64 bytes from pnfjra-an-in-f14.1e100.net (142.250.202.78): icmp_seq=1 ttl=255 time
=20.0 ms
64 bytes from pnfjra-an-in-f14.1e100.net (142.250.202.78): icmp_seq=2 ttl=255 time
=27.1 ms
64 bytes from pnfjra-an-in-f14.1e100.net (142.250.202.78): icmp_seq=3 ttl=255 time
=19.0 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 19.038/22.044/27.091/3.590 ms
[root@wazuh-server ~]# curl -I https://www.virustotal.com
HTTP/2 405
allow: GET
content-type: text/html; charset=UTF-8
x-cloud-trace-context: 873d7975091e74c09b95af2af5ff5e60
content-length: 188
date: Sat, 16 May 2026 19:30:31 GMT
server: Google Frontend
via: 1.1 google
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000

[root@wazuh-server ~]# sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; pres
   Active: active (running) since Sat 2026-05-16 19:30:51 UTC; 54ms ago
   Process: 21187 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (cod
   Tasks: 237 (limit: 9464)
   Memory: 247.2M
   CPU: 21.035s
   CGroup: /system.slice/wazuh-manager.service

```

Figure 43: Wazuh server internet connectivity and Wazuh manager active/running status after the VirusTotal connectivity issue was fixed.

The screenshot shows a Windows PowerShell window titled "Administrator: Windows PowerShell" running on a Windows Server. The user is in the directory "C:\Program Files (x86)\ossec-agent\active-response\bin". A directory listing shows a file named "remove-threat.exe" with a size of 7268184 bytes, last written on 5/5/2026 at 10:18 PM. The user then navigates to the "Downloads" directory and runs two commands: "Remove-Item .\eicar.txt -Force -ErrorAction SilentlyContinue" and "Invoke-WebRequest -Uri https://secure.eicar.org/eicar.com.txt -OutFile eicar.txt".

```

PS C:\Users\Administrator> dir "C:\Program Files (x86)\ossec-agent\active-response\bin\remove-threat.exe"

Directory: C:\Program Files (x86)\ossec-agent\active-response\bin

Mode                LastWriteTime         Length Name
----                -
-a----             5/5/2026  10:18 PM       7268184 remove-threat.exe

PS C:\Users\Administrator> cd C:\Users\Administrator\Downloads
PS C:\Users\Administrator\Downloads> Remove-Item .\eicar.txt -Force -ErrorAction SilentlyContinue
PS C:\Users\Administrator\Downloads> Invoke-WebRequest -Uri https://secure.eicar.org/eicar.com.txt -OutFile eicar.txt
PS C:\Users\Administrator\Downloads>

```

Figure 44: Windows PowerShell evidence showing remove-threat.exe exists and the EICAR test file was downloaded into the monitored Downloads directory.

4. Threat Simulation and Detection Evidence

The EICAR file was used as a safe malware test file. After it was placed in the monitored directory, Wazuh generated a file creation alert and VirusTotal identified the file as malicious.

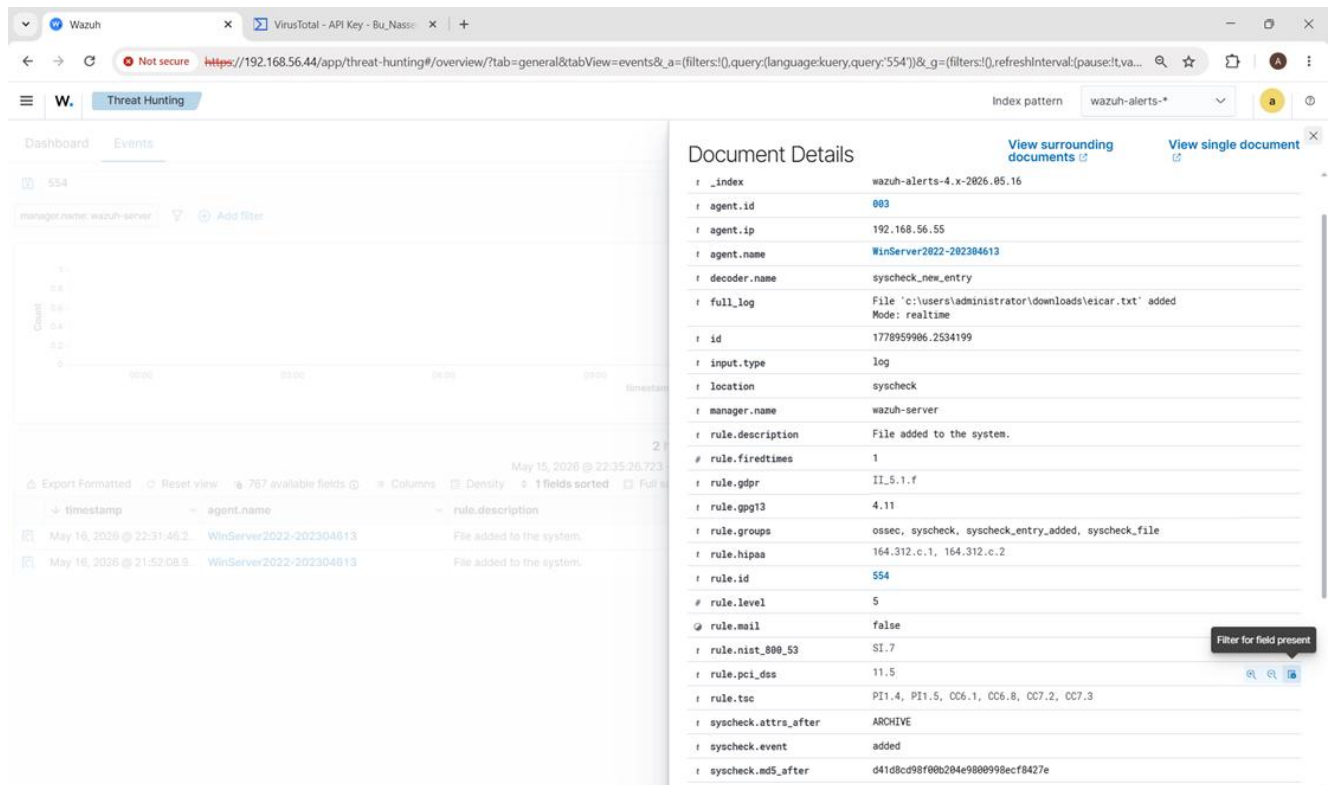


Figure 45: Document Details for rule ID 554 showing that eicar.txt was added to the monitored Downloads directory.

Document Details		View surrounding documents	View single document
data.virustotal.scan_date	2026-05-16 19:19:52		
data.virustotal.sha1	3395856ce81f2b7382dee72602f798b642f14140		
data.virustotal.source.alert_id	1778959907.2538828		
data.virustotal.source.file	c:\users\administrator\downloads\eicar.txt		
data.virustotal.source.md5	44d88612fea8a8f36de82e1278abb02f		
data.virustotal.source.sha1	3395856ce81f2b7382dee72602f798b642f14140		
data.virustotal.total	68		
decoder.name	json		
id	1778959910.2546857		
input.type	log		
location	virustotal		
manager.name	wazuh-server		
rule.description	VirusTotal: Alert - c:\users\administrator\downloads\eicar.txt - 66 engines detected this file		
rule.firedtimes	3		
rule.gdpr	IV.35.7.d		
rule.groups	virustotal		
rule.id	87105		
rule.level	12		
rule.mail	true		
rule.mitre.id	T1203		
rule.mitre.tactic	Execution		
rule.mitre.technique	Exploitation for Client Execution		
rule.pci_dss	10.6.1, 11.4		
timestamp	May 16, 2026 @ 22:31:50.330		

Figure 46: VirusTotal Document Details showing the EICAR file was found and marked malicious, with 66 positive detections.

Document Details		View surrounding documents	View single document	✕
data.virustotal.scan_date	2026-05-16 19:19:52			
data.virustotal.sha1	3395856ce81f2b7382dee72602f798b642f14140			
data.virustotal.source.alert_id	1778959907.2538828			
data.virustotal.source.file	c:\users\administrator\downloads\eicar.txt			
data.virustotal.source.md5	44d88612fea8a8f36de82e1278abb02f			
data.virustotal.source.sha1	3395856ce81f2b7382dee72602f798b642f14140			
data.virustotal.total	68			
decoder.name	json			
id	1778959910.2546857			
input.type	log			
location	virustotal			
manager.name	wazuh-server			
rule.description	VirusTotal: Alert - c:\users\administrator\downloads\eicar.txt - 66 engines detected this file			
# rule.firedtimes	3			
rule.gdpr	IV_35.7.d			
rule.groups	virustotal			
rule.id	87105			
# rule.level	12			
rule.mail	true			
rule.mitre.id	T1203			
rule.mitre.tactic	Execution			
rule.mitre.technique	Exploitation for Client Execution			
rule.pci_dss	10.6.1, 11.4			
timestamp	May 16, 2026 @ 22:31:50.330			

Figure 47: VirusTotal alert details showing rule ID 87105, severity level 12, the eicar.txt file path, and MITRE mapping.

5. Wazuh Dashboard Response Evidence

The doctor feedback required dashboard response evidence for each attack. Figure 48 provides the main Attack 2 dashboard response, showing the full chain from file creation to malware detection, automated removal, and file deletion under the renamed agent WinServer2022-202304613.

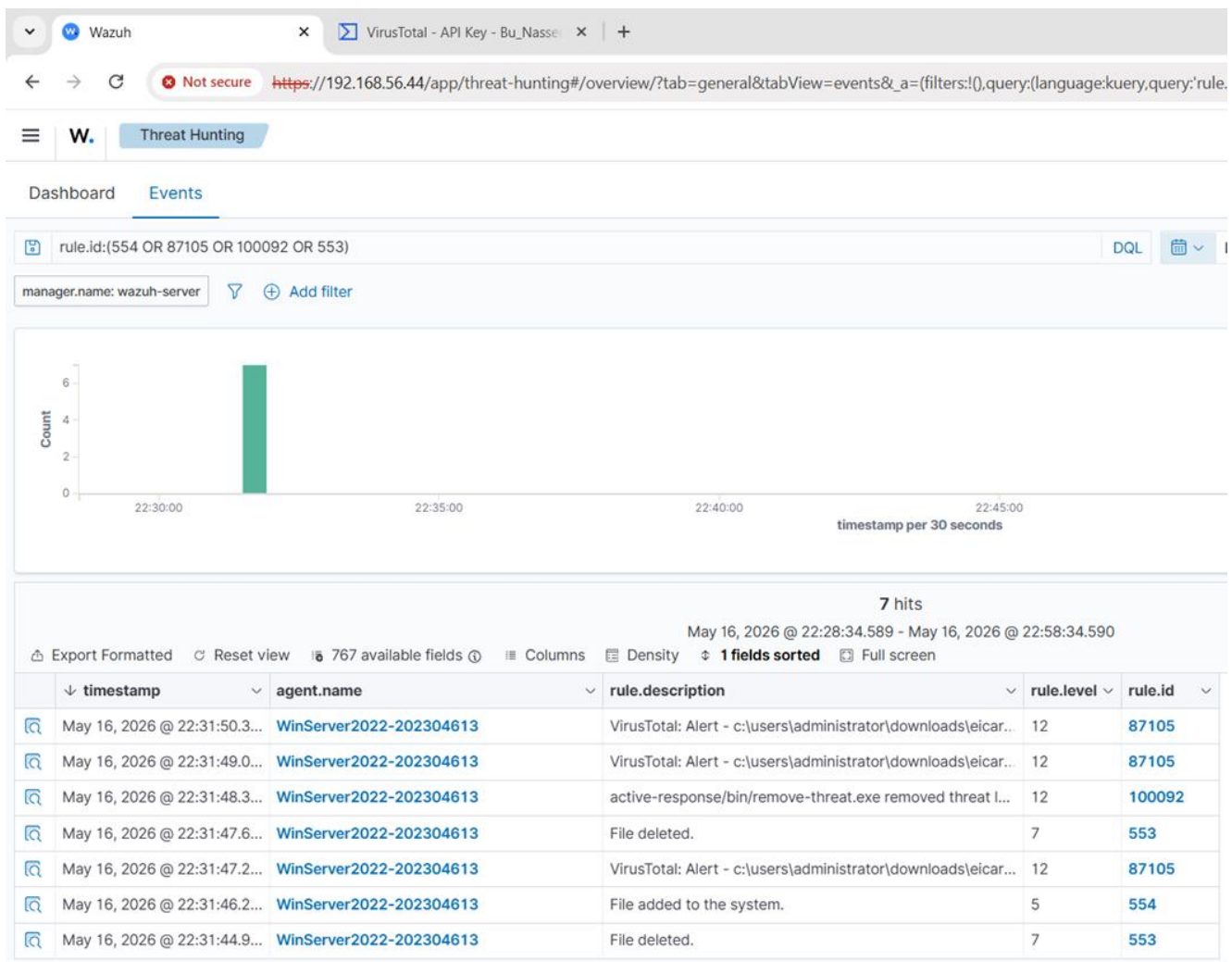


Figure 48: Wazuh dashboard response showing the full Attack 2 chain: 554 file added, 87105 VirusTotal alert, 100092 Active Response executed, and 553 file deleted.

6. Response Execution and Validation

The response action was configured through Wazuh Active Response. When VirusTotal rule 87105 was triggered, Wazuh executed remove-threat.exe on the Windows endpoint. The Wazuh dashboard generated rule ID 100092 to confirm that the file was removed, and rule ID 553 confirmed the file deletion event.

Document Details

[View surrounding documents](#)
[View single document](#)

data.parameters.alert.rule.mitre.technique	Exploitation for Client Execution
data.parameters.alert.rule.pci_dss	10.6.1, 11.4
data.parameters.alert.timestamp	2026-05-16T19:31:47.241+0000
data.parameters.extra_args	
data.parameters.program	active-response/bin/remove-threat.exe
data.version	1
decoder.name	ar_log_json
decoder.parent	ar_log_json
full_log	2026/05/16 22:22:26 active-response/bin/remove-threat.exe: {"version": 1, "origin": {"name": "node01", "module": "wazuh-execd"}, "command": "add", "parameters": {"extra_args": [], "alert": {"timestamp": "2026-05-16T19:31:47.241+0000", "rule": {"level": 12, "description": "VirusTotal: Alert - c:\\users\\administrator\\downloads\\eicar.txt - 66 engines detected this file", "id": "87105", "mitre": {"id": ["T1203"], "tactic": ["Execution"], "technique": ["Exploitation for Client Execution"]}} "firedtimes": 1 "mai
id	1778959908.2541546
input.type	log
location	active-response\\active-responses.log
manager.name	wazuh-server
rule.description	active-response/bin/remove-threat.exe removed threat located at c:\\users\\administrator\\downloads\\eicar.txt
rule.firedtimes	1
rule.groups	virustotal
rule.id	100092
rule.level	12
rule.mail	true
timestamp	May 16, 2026 @ 22:31:48.336

Figure 49: Document Details for rule ID 100092 showing Active Response remove-threat.exe successfully removed eicar.txt.

Document Details

[View surrounding documents](#)[View single document](#)

t _index	wazuh-alerts-4.x-2026.05.16
t agent.id	003
t agent.ip	192.168.56.55
t agent.name	WinServer2022-202304613
t decoder.name	syscheck_deleted
t full_log	File 'c:\users\administrator\downloads\eicar.txt' deleted Mode: realtime
t id	1778959907.2538828
t input.type	log
t location	syscheck
t manager.name	wazuh-server
t rule.description	File deleted.
# rule.firedtimes	2
t rule.gdpr	II_5.1.f
t rule.gpg13	4.11
t rule.groups	ossec, syscheck, syscheck_entry_deleted, syscheck_file
t rule.hipaa	164.312.c.1, 164.312.c.2
t rule.id	553
# rule.level	7
rule.mail	false
t rule.mitre.id	T1070.004 T1485
t rule.mitre.tactic	Defense Evasion, Impact
t rule.mitre.technique	File Deletion, Data Destruction
t rule.nist_800_53	SI.7
t rule.pci_dss	11.5
t rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3

Figure 50: Document Details for rule ID 553 showing that eicar.txt was deleted from the monitored directory.

Document Details		View surrounding documents	View single document
# rule.level	7		
rule.mail	false		
t rule.mitre.id	T1070.004 T1485		
t rule.mitre.tactic	Defense Evasion, Impact		
t rule.mitre.technique	File Deletion, Data Destruction		
t rule.nist_800_53	SI.7		
t rule.pci_dss	11.5		
t rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3		
t syscheck.attrs_after	ARCHIVE		
t syscheck.event	deleted		
t syscheck.md5_after	44d88612fea8a8f36de82e1278abb02f		
t syscheck.mode	realtime		
syscheck.mtime_after	May 16, 2026 @ 22:22:24.000		
t syscheck.path	c:\users\administrator\downloads\eicar.txt		
t syscheck.sha1_after	3395856ce81f2b7382dee72602f798b642f14140		
t syscheck.sha256_after	275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabb651fd0f		
# syscheck.size_after	68		
t syscheck.uid_after	S-1-5-32-544		
t syscheck.uname_after	Administrators		
t syscheck.win_perm_after.allowed	DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES		
t syscheck.win_perm_after.name	SYSTEM, Administrators, Administrator		
timestamp	May 16, 2026 @ 22:31:47.609		

Figure 51: Additional rule 553 details showing the deleted file path, hashes, syscheck event status, and deletion metadata.

```

Windows Server_202304613 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator> Test-Path C:\Users\Administrator\Downloads\eicar.txt
False
PS C:\Users\Administrator>

```

Figure 52: PowerShell validation showing Test-Path returned False, confirming eicar.txt no longer exists on the endpoint.

7. Logs and Event IDs Identified

Rule ID	Source / Module	Meaning	Evidence
554	syscheck	File added to the system: eicar.txt was created in the monitored Downloads folder.	Figures 45 and 48
87105	virustotal	VirusTotal detected eicar.txt as malicious with 66 positive detections.	Figures 46, 47 and 48
100092	active-response log	remove-threat.exe successfully removed the detected threat.	Figures 48 and 49
553	syscheck	File deleted: eicar.txt was removed from the endpoint.	Figures 48, 50 and 51

8. Attack 2 Result

Attack 2 was successful. The EICAR test file was downloaded into the monitored Downloads directory on Windows Server 2022. Wazuh File Integrity Monitoring detected the file creation using rule ID 554. The alert was enriched by VirusTotal, which identified the file as malicious and generated rule ID 87105. Wazuh Active Response executed remove-threat.exe and generated rule ID 100092, confirming that the threat was removed. Wazuh also generated rule ID 553 confirming that the file was deleted. Finally, the response was validated using PowerShell, where Test-Path returned False, confirming that eicar.txt was removed from the endpoint.

9. Deliverables Checklist - Attack 2

Deliverable	Evidence in this document	Status
Threat Detection Evidence	Wazuh dashboard and Document Details for rules 554 and 87105	Complete
Rule Configuration Evidence	FIM configuration, VirusTotal integration, Active Response block, and custom rule 100092	Complete
Logs / Event IDs	554, 87105, 100092 and 553 identified and explained	Complete
Response Configuration Evidence	remove-threat command and Active Response triggering on rule 87105	Complete
Result Validation	Dashboard rule 100092, rule 553, and PowerShell Test-Path False	Complete

Attack 3: Suspicious PowerShell Activity Detection and Response

1. Attack Overview

Scenario: Suspicious PowerShell execution was simulated on Windows Server 2022 using the -nop parameter. This parameter is commonly associated with suspicious or evasive PowerShell activity because it can prevent profile scripts from loading and is often seen in process creation telemetry during malicious script execution attempts.

Goal: Detect the suspicious PowerShell command using Wazuh and Sysmon, then apply a response action that blocks outbound web traffic from powershell.exe. The response is validated by confirming that PowerShell can no longer establish a TCP connection to example.com on port 80.

2. Environment and Tools Used

Component	Used For
Windows Server 2022 agent	Endpoint where the suspicious PowerShell command was executed.
Agent name	WinServer2022-202304613, showing the required student ID in Wazuh evidence.
Sysmon64	Collected Windows process creation telemetry.
Wazuh agent	Forwarded Sysmon Operational logs to the Wazuh manager.
Wazuh manager and dashboard	Detected rule 100900 and displayed dashboard/document details evidence.
Windows Firewall	Response action used to block outbound network traffic from powershell.exe.

3. Rubric and Requirement Mapping

Task 2 Requirement	Attack 3 Evidence Provided	Status
Generate the attack	PowerShell command executed with -nop and web request to example.com.	Met
Configure detection rule	Custom Wazuh rule ID 100900 detects powershell.exe command lines containing -nop.	Met
Verify detection using logs/events	Wazuh dashboard and Document Details show rule 100900, Sysmon Event ID 1, PowerShell image, and command line.	Met
Configure response action	Windows Firewall rule blocks outbound traffic from powershell.exe.	Met
Confirm response successfully executed	Test-NetConnection shows TcpTestSucceeded: False after the block rule.	Met
Doctor feedback: dashboard response	Dashboard Events and Document Details are included for the attack.	Met

4. Log Source and Collection Configuration

Sysmon was confirmed as running on the Windows Server endpoint, and the Wazuh agent was configured to collect the Microsoft-Windows-Sysmon/Operational event channel. This provides the process creation telemetry needed for suspicious PowerShell detection.

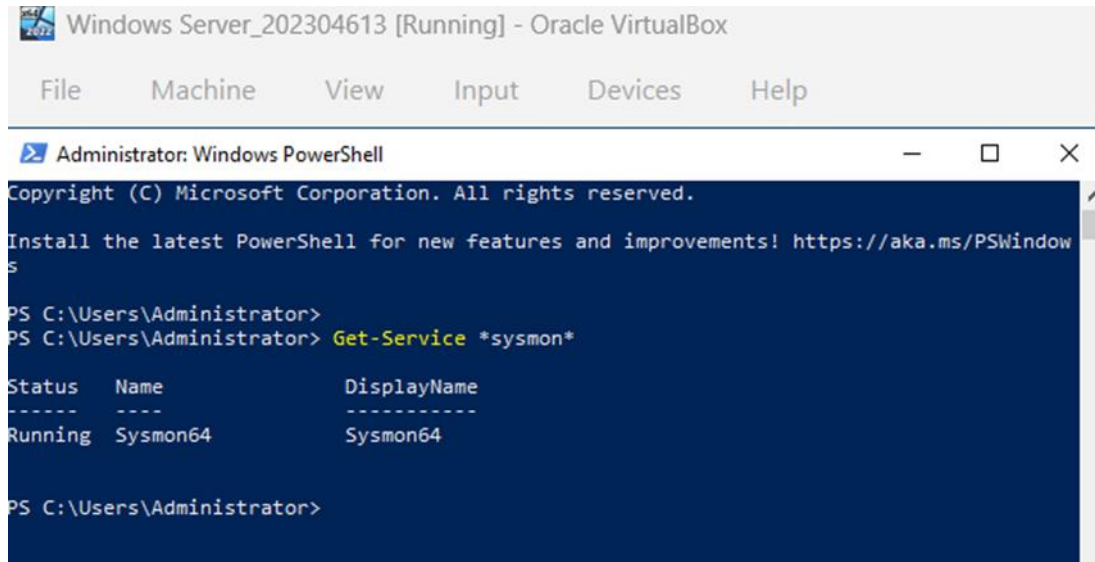


Figure 53: Sysmon64 service running on Windows Server 2022.

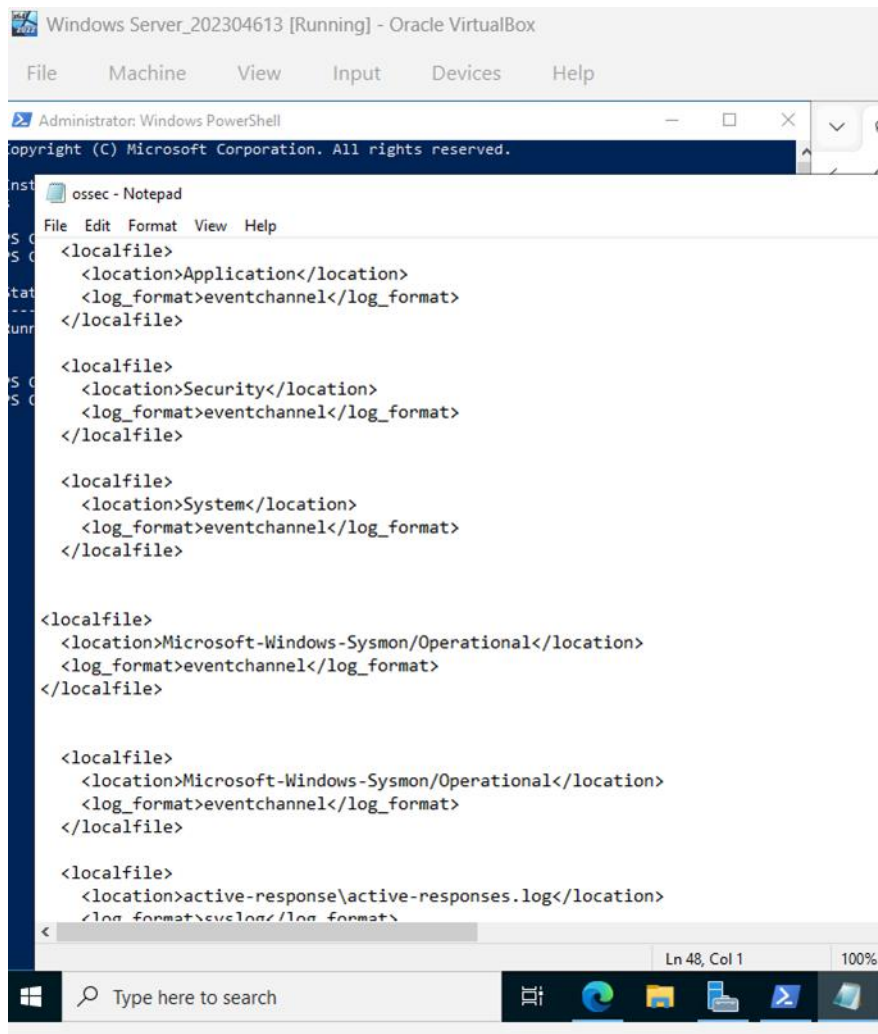
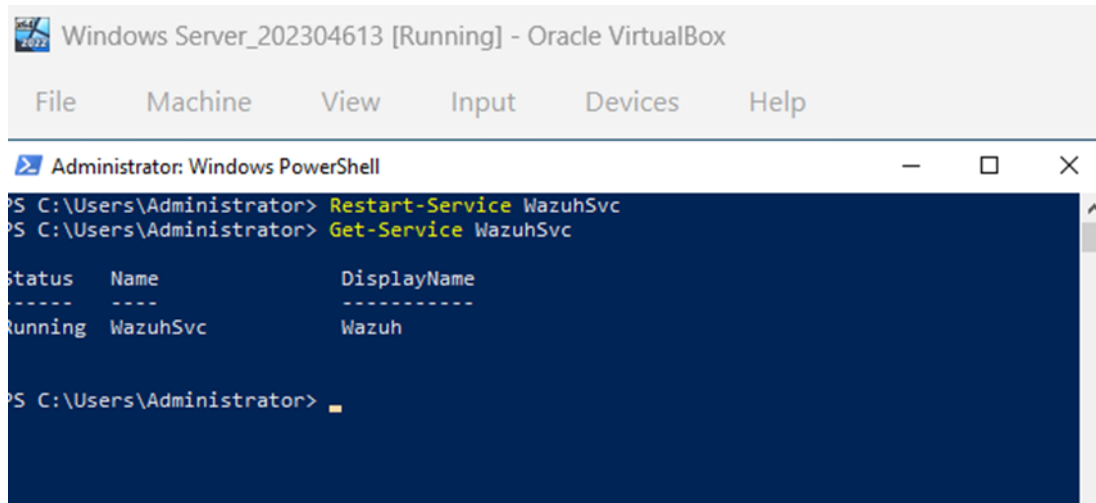


Figure 54: Wazuh agent configuration collecting Microsoft-Windows-Sysmon/Operational logs.



```
Windows Server_202304613 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Administrator: Windows PowerShell
PS C:\Users\Administrator> Restart-Service WazuhSvc
PS C:\Users\Administrator> Get-Service WazuhSvc

Status      Name      DisplayName
-----
Running     WazuhSvc  Wazuh

PS C:\Users\Administrator>
```

Figure 55: Wazuh agent service restarted and running after the Sysmon log collection configuration.

5. Detection Rule Configuration Evidence

A custom Wazuh rule was configured to detect PowerShell process creation where the command line contains the -nop parameter. The rule is associated with Sysmon Event ID 1 process creation telemetry and generates rule ID 100900 at level 12.

```
root@wazuh-server:~  
GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml  
<description>sshd: authentication failed from IP 1.1.1.1.</description>  
<group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>  
</rule>  
  
</group>  
  
<group name="virustotal,">  
  <rule id="100092" level="12">  
    <if_sid>657</if_sid>  
    <match>Successfully removed threat</match>  
    <description>$(parameters.program) removed threat located at $(parameters.a  
  </rule>  
  
  <rule id="100093" level="12">  
    <if_sid>657</if_sid>  
    <match>Error removing threat</match>  
    <description>Error removing threat located at $(parameters.alert.data.virusto  
  </rule>  
</group>  
  
<group name="ssh,authentication_failed,bruteforce,">  
  
  <rule id="100300" level="10" frequency="5" timeframe="60">  
    <if_matched_sid>5503</if_matched_sid>  
    <description>SSH Brute Force Attack Detected on Bodhi Linux</description>  
    <group>authentication_failed,sshd,bruteforce</group>  
  </rule>  
  
</group>  
  
<group name="sysmon,powershell,lab,">  
  <rule id="100900" level="12">  
    <if_group>sysmon_event1</if_group>  
    <field name="win.eventdata.image" type="pcre2">(?i)\\powershell\\.exe$</field>  
    <field name="win.eventdata.commandLine" type="pcre2">(?i)-nop</field>  
    <description>LAB ALERT: PowerShell executed with -nop parameter</description>  
    <group>powershell,suspicious_process,sysmon_event1</group>  
  </rule>  
</group>  
|  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Figure 56: Custom Wazuh rule 100900 configured in local_rules.xml to detect PowerShell -nop execution.

```
root@wazuh-server:~  
[root@wazuh-server ~]# sudo systemctl restart wazuh-manager  
[root@wazuh-server ~]# sudo systemctl status wazuh-manager  
● wazuh-manager.service - Wazuh manager  
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; pres>  
   Active: active (running) since Sat 2026-05-16 20:36:53 UTC; 5s ago  
     Process: 24479 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (cod>  
    Tasks: 230 (limit: 9464)  
   Memory: 620.9M  
      CPU: 32.532s  
   CGroup: /system.slice/wazuh-manager.service  
           └─24541 /var/ossec/framework/python/bin/python3 /var/ossec/api/scrip>  
             └─24542 /var/ossec/framework/python/bin/python3 /var/ossec/api/scrip>  
               └─24543 /var/ossec/framework/python/bin/python3 /var/ossec/api/scrip>  
                 └─24546 /var/ossec/framework/python/bin/python3 /var/ossec/api/scrip>  
                   └─24549 /var/ossec/framework/python/bin/python3 /var/ossec/api/scrip>  
                     └─24573 /var/ossec/bin/wazuh-integratord  
                       └─24594 /var/ossec/bin/wazuh-authd  
                         └─24610 /var/ossec/bin/wazuh-db  
                           └─24646 /var/ossec/bin/wazuh-execd  
                             └─24661 /var/ossec/bin/wazuh-analysisd  
                               └─24679 /var/ossec/bin/wazuh-syscheckd  
                                 └─24700 /var/ossec/bin/wazuh-remoted  
                                   └─24702 /var/ossec/bin/wazuh-remoted  
                                     └─24737 /var/ossec/bin/wazuh-logcollector  
                                       └─24748 /var/ossec/bin/wazuh-monitor  
                                         └─24768 /var/ossec/bin/wazuh-modulesd  
                                           └─25964 /usr/bin/python3 /usr/bin/dnf list "installed|" grep sudo  
  
May 16 20:36:46 wazuh-server env[24479]: Started wazuh-analysisd...  
May 16 20:36:48 wazuh-server env[24479]: Started wazuh-syscheckd...  
May 16 20:36:49 wazuh-server env[24479]: Started wazuh-remoted...  
May 16 20:36:49 wazuh-server env[24479]: Started wazuh-logcollector...  
May 16 20:36:50 wazuh-server env[24479]: Started wazuh-monitor...  
May 16 20:36:50 wazuh-server env[24765]: 2026/05/16 20:36:50 wazuh-modulesd:route>  
May 16 20:36:50 wazuh-server env[24765]: 2026/05/16 20:36:50 wazuh-modulesd:conte>  
May 16 20:36:50 wazuh-server env[24765]: 2026/05/16 20:36:50 wazuh-modulesd:inven>  
May 16 20:36:51 wazuh-server env[24479]: Started wazuh-modulesd...  
May 16 20:36:53 wazuh-server env[24479]: Completed.  
lines 1-36/36 (END)
```

Figure 57: Wazuh manager restarted successfully and running after the custom rule configuration.

6. Attack Simulation Evidence

The suspicious activity was generated by running a PowerShell command with the `-nop` parameter and a web request. This creates a Sysmon process creation event that Wazuh can evaluate against the custom detection rule.

`powershell -nop -W hidden -c "iwr http://example.com"`

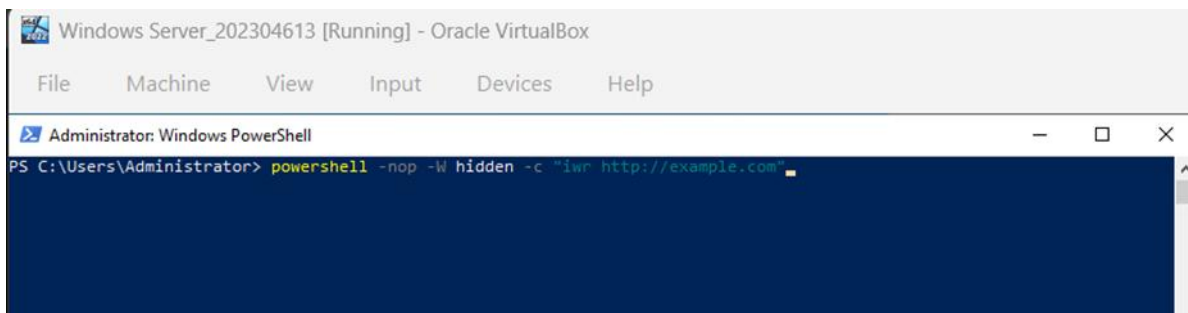


Figure 58: Suspicious PowerShell command executed with the `-nop` parameter.

7. Wazuh Dashboard Response Evidence

The Wazuh dashboard detected the suspicious PowerShell activity using custom rule ID 100900. The alert appears under the renamed Windows agent WinServer2022-202304613, satisfying the requirement to show the student ID in the agent name.

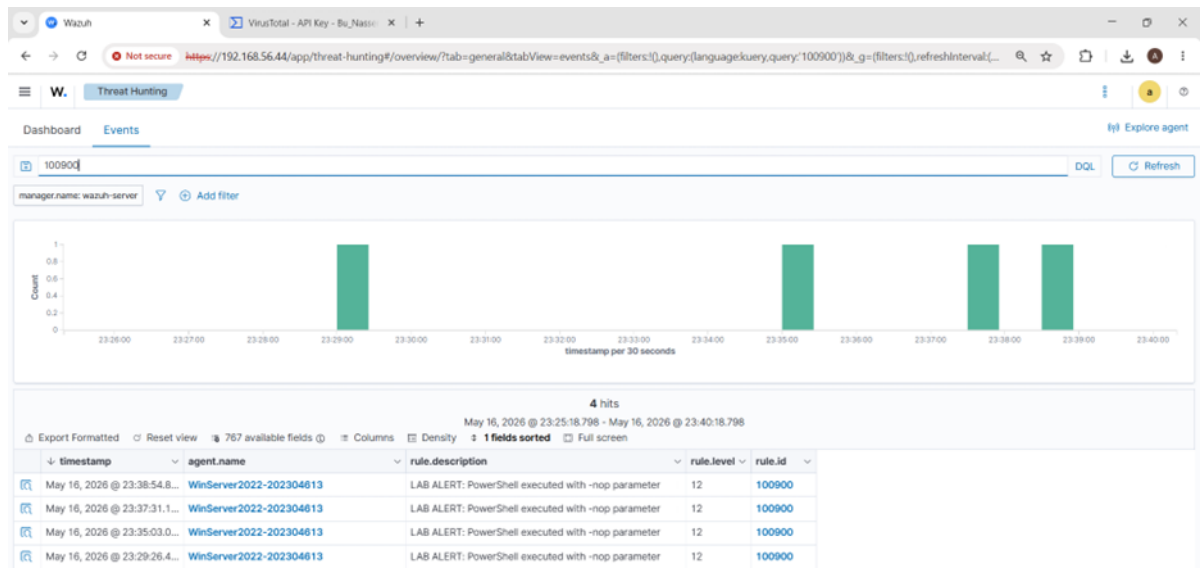


Figure 59: Wazuh dashboard response showing rule ID 100900 for suspicious PowerShell execution.

8. Logs and Event ID Evidence

Document Details confirmed that the alert came from the Sysmon Operational channel. The important log identifiers are Sysmon Event ID 1, the PowerShell executable path, the user context, and the command line containing the -nop parameter.

Document Details

[View surrounding documents](#)[View single document](#)

Table JSON

_index	wazuh-alerts-4.x-2026.05.16
agent.id	003
agent.ip	192.168.56.55
agent.name	WinServer2022-202304613
data.win.eventdata.commandLine	"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe" -dd -c "iwr http://example.com\"
data.win.eventdata.company	Microsoft Corporation
data.win.eventdata.currentDirectory	C:\\Users\\Administrator\\
data.win.eventdata.description	Windows PowerShell
data.win.eventdata.fileVersion	10.0.20348.1 (WinBuild.160101.0800)
data.win.eventdata.hashes	SHA256=1C84C8632C5269F24876ED9F49FA810B49F77E1E92E8918FC164C34B020F9A94
data.win.eventdata.image	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe
data.win.eventdata.integrityLevel	High
data.win.eventdata.logonGuid	{35a2213f-d3ab-6a08-fd04-090000000000}
data.win.eventdata.logonId	0x904fd
data.win.eventdata.originalFileName	PowerShell.EXE
data.win.eventdata.parentCommandLine	"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe"
data.win.eventdata.parentImage	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe
data.win.eventdata.parentProcessGuid	{35a2213f-d5a3-6a08-7501-000000002b00}

Figure 59: Document Details showing the PowerShell command line and powershell.exe image path.

Document Details

[View surrounding documents](#)[View single document](#)

data.win.eventdata.parentProcessGuid	{35a2213f-d5a3-6a08-7501-000000002b00}
data.win.eventdata.parentProcessId	548
data.win.eventdata.parentUser	WIN-5CUOAEMQES8\Administrator
data.win.eventdata.processGuid	{35a2213f-d5d7-6a08-7701-000000002b00}
data.win.eventdata.processId	3576
data.win.eventdata.product	Microsoft® Windows® Operating System
data.win.eventdata.sessionId	1
data.win.eventdata.user	WIN-5CUOAEMQES8\Administrator
data.win.eventdata.utcTime	2026-05-16 20:38:47.731
data.win.system.channel	Microsoft-Windows-Sysmon/Operational
data.win.system.computer	WIN-5CUOAEMQES8
data.win.system.eventID	1
data.win.system.eventRecordID	15815
data.win.system.keywords	0x0000000000000000
data.win.system.level	4
data.win.system.message	"Process Create: RuleName: - UtcTime: 2026-05-16 20:38:47.731 ProcessGuid: {35a2213f-d5d7-6a08-7701-000000002b00} ProcessId: 3576 Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe FileVersion: 10.0.20348.1 (WinBuild 160101.0800)
data.win.system.opcode	0
data.win.system.processID	1908
data.win.system.providerGuid	{5770385f-c22a-43e0-bf4c-06f5698ffbd9}

Figure 60: Document Details showing Microsoft-Windows-Sysmon/Operational and Sysmon Event ID 1.

Document Details

[View surrounding documents](#)
[View single document](#)

data.win.system.opcode	
data.win.system.processID	1908
data.win.system.providerGuid	{5770385f-c22a-43e0-bf4c-06f5698ffbd9}
data.win.system.providerName	Microsoft-Windows-Sysmon
data.win.system.severityValue	INFORMATION
data.win.system.systemTime	2026-05-16T20:38:47.7327803Z
data.win.system.task	1
data.win.system.threadID	2952
data.win.system.version	5
decoder.name	windows_eventchannel
full_log	<pre>{ "win": { "system": { "providerName": "Microsoft-Windows-Sysmon", "providerGuid": "{5770385f-c22a-43e0-bf4c-06f5698ffbd9}", "eventID": "1", "version": "5", "level": "4", "task": "1", "opcode": "0", "keywords": "0x0000000000000000", "systemTime": "2026-05-16T20:38:47.7327803Z", "eventRecordID": "15815", "processID": "1908", "threadID": "2952", "channel": "Microsoft-Windows-Sysmon/Operational", "computer": "WIN-5CUOAEQES8", "severityValue": "INFORMATION", "message": "\\Process Create\\r\\nRuleName: -\\r\\nUtcTime: 2026-05-16 20:38:47.731\\r\\nProcessG" } } }</pre>
id	1778963934.3202378
input.type	log
location	EventChannel
manager.name	wazuh-server
rule.description	LAB ALERT: PowerShell executed with -nop parameter
rule.firedtimes	2
rule.groups	sysmon, powershell, lab
rule.id	100900
rule.level	12
rule.mail	true
timestamp	May 16, 2026 @ 23:38:54.850

Filter for field present

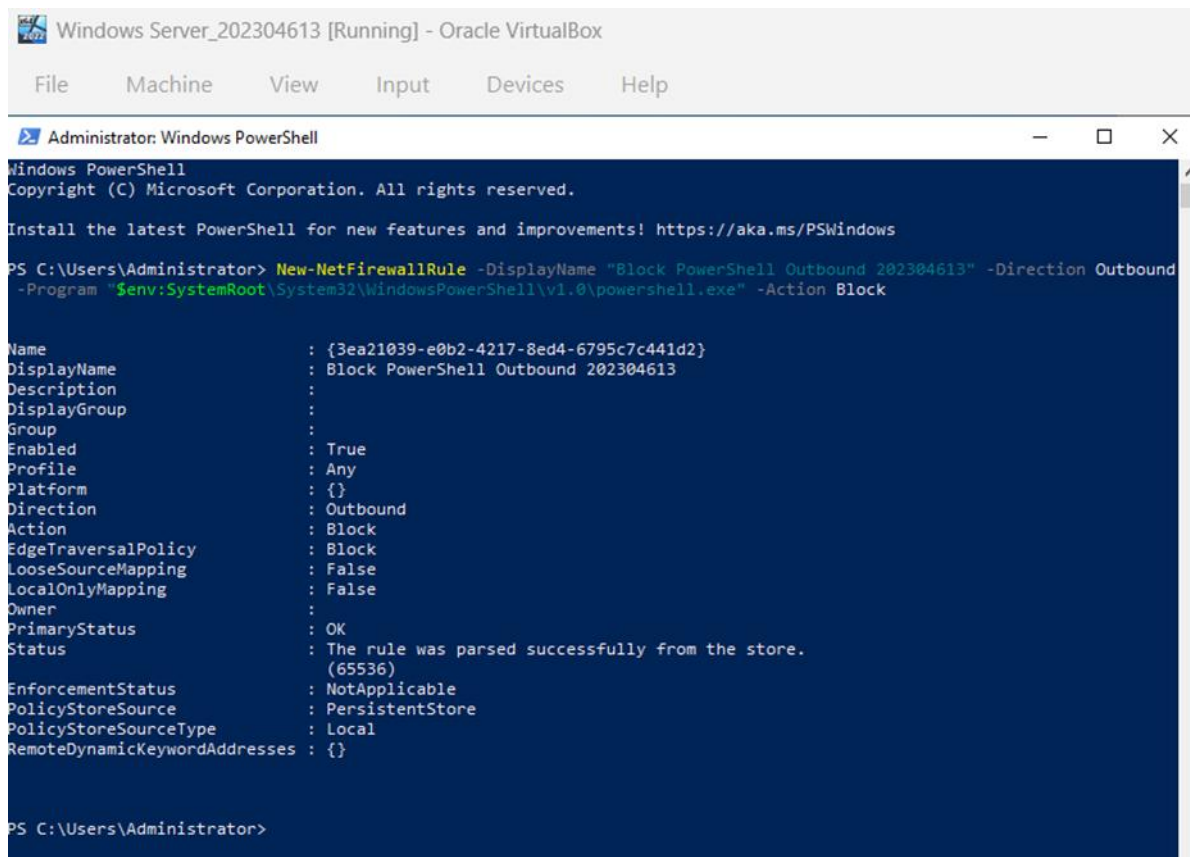
Figure 61: Document Details showing rule ID 100900 and the alert description.

Field	Value / Meaning
Agent name	WinServer2022-202304613
Log channel	Microsoft-Windows-Sysmon/Operational
Event ID	1 - Process Create
Custom rule ID	100900
Rule level	12
Rule description	LAB ALERT: PowerShell executed with -nop parameter
Detected process	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Suspicious indicator	Command line contains -nop

9. Response Configuration Evidence

The response action configured for this attack was a Windows Firewall outbound block rule for powershell.exe. This response prevents PowerShell from making outbound web connections after suspicious PowerShell activity is detected.

New-NetFirewallRule -DisplayName "Block PowerShell Outbound 202304613" -Direction Outbound -Program "\$env:SystemRoot\System32\WindowsPowerShell\v1.0\powershell.exe" -Action Block

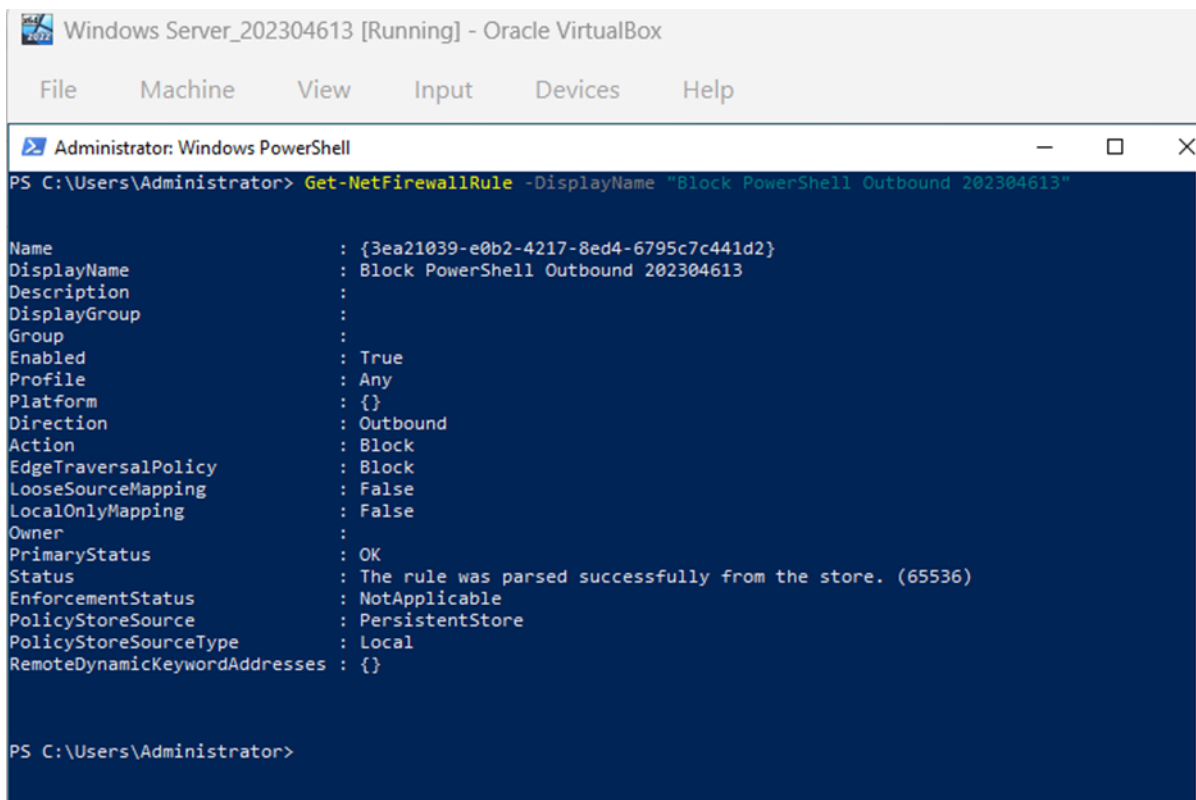


The screenshot shows a Windows PowerShell terminal window titled "Administrator: Windows PowerShell". The command prompt is at "PS C:\Users\Administrator>". The user has entered the command: `New-NetFirewallRule -DisplayName "Block PowerShell Outbound 202304613" -Direction Outbound -Program "$env:SystemRoot\System32\WindowsPowerShell\v1.0\powershell.exe" -Action Block`. The output displays the properties of the newly created rule:

```
Name : {3ea21039-e0b2-4217-8ed4-6795c7c441d2}
DisplayName : Block PowerShell Outbound 202304613
Description :
DisplayGroup :
Group :
Enabled : True
Profile : Any
Platform : {}
Direction : Outbound
Action : Block
EdgeTraversalPolicy : Block
LooseSourceMapping : False
LocalOnlyMapping : False
Owner :
PrimaryStatus : OK
Status : The rule was parsed successfully from the store.
        (65536)
EnforcementStatus : NotApplicable
PolicyStoreSource : PersistentStore
PolicyStoreSourceType : Local
RemoteDynamicKeywordAddresses : {}
```

The prompt returns to "PS C:\Users\Administrator>" at the bottom.

Figure 62: Windows Firewall response rule configured to block outbound traffic from powershell.exe.



```
Windows Server_202304613 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Administrator: Windows PowerShell
PS C:\Users\Administrator> Get-NetFirewallRule -DisplayName "Block PowerShell Outbound 202304613"

Name                : {3ea21039-e0b2-4217-8ed4-6795c7c441d2}
DisplayName          : Block PowerShell Outbound 202304613
Description          :
DisplayGroup         :
Group                :
Enabled              : True
Profile              : Any
Platform             : {}
Direction            : Outbound
Action               : Block
EdgeTraversalPolicy  : Block
LooseSourceMapping   : False
LocalOnlyMapping     : False
Owner                :
PrimaryStatus        : OK
Status               : The rule was parsed successfully from the store. (65536)
EnforcementStatus    : NotApplicable
PolicyStoreSource    : PersistentStore
PolicyStoreSourceType : Local
RemoteDynamicKeywordAddresses : {}

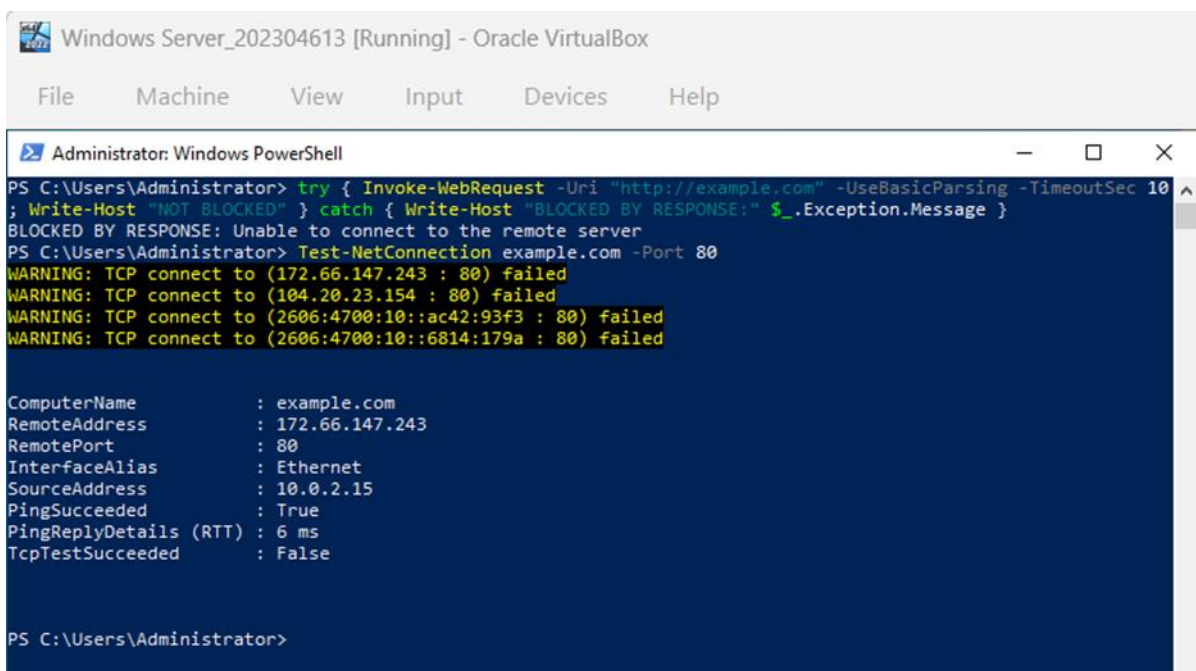
PS C:\Users\Administrator>
```

Figure 63: Firewall rule verification showing the PowerShell outbound block rule is enabled and active.

10. Result Validation Evidence

The response was validated by testing outbound TCP connectivity from PowerShell to example.com on port 80. The result shows that ICMP/ping can still succeed, but TCP port 80 fails. This is expected because the response rule specifically blocks PowerShell outbound web traffic, not all network traffic.

Test-NetConnection example.com -Port 80



```
Windows Server_202304613 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Administrator: Windows PowerShell
PS C:\Users\Administrator> try { Invoke-WebRequest -Uri "http://example.com" -UseBasicParsing -TimeoutSec 10 }
; Write-Host "NOT BLOCKED" } catch { Write-Host "BLOCKED BY RESPONSE:" $_.Exception.Message }
BLOCKED BY RESPONSE: Unable to connect to the remote server
PS C:\Users\Administrator> Test-NetConnection example.com -Port 80
WARNING: TCP connect to (172.66.147.243 : 80) failed
WARNING: TCP connect to (104.20.23.154 : 80) failed
WARNING: TCP connect to (2606:4700:10::ac42:93f3 : 80) failed
WARNING: TCP connect to (2606:4700:10::6814:179a : 80) failed

ComputerName        : example.com
RemoteAddress       : 172.66.147.243
RemotePort          : 80
InterfaceAlias      : Ethernet
SourceAddress       : 10.0.2.15
PingSucceeded       : True
PingReplyDetails (RTT) : 6 ms
TcpTestSucceeded    : False

PS C:\Users\Administrator>
```

Figure 64: Response validation showing TcpTestSucceeded: False after the PowerShell outbound block was applied.

11. Attack 3 Result

Attack 3 was successful. Suspicious PowerShell activity was generated using a command that included the -nop parameter. Sysmon recorded the process creation event, and Wazuh detected it using custom rule ID 100900. The Wazuh dashboard showed the alert under the renamed agent WinServer2022-202304613, and Document Details confirmed Sysmon Event ID 1, powershell.exe, and the command line. The response action was implemented by creating a Windows Firewall outbound block rule for powershell.exe. The response was validated using Test-NetConnection, which returned TcpTestSucceeded: False, proving that PowerShell outbound web traffic was blocked.

12. Attack 3 Deliverables Summary

Deliverable	Evidence Included
Threat Detection Evidence	Wazuh dashboard screenshot showing rule ID 100900 and the suspicious PowerShell alert.
Rule Configuration Evidence	local_rules.xml screenshot showing custom rule 100900.
Logs / Event IDs	Document Details screenshots showing Sysmon Operational channel and Event ID 1.
Response Configuration Evidence	Windows Firewall rule screenshots blocking outbound traffic from powershell.exe.
Result	Test-NetConnection screenshot showing TcpTestSucceeded: False.

Task 2 Requirement Mapping

Project Requirement	Evidence Provided in This Document
Simulate and detect any three cyber threats	Three attacks were implemented: SMB brute force, malware/EICAR with VirusTotal, and suspicious PowerShell execution.
Configure a detection rule for each attack	Rule 100100 was configured for SMB brute force, VirusTotal/FIM rules 554 and 87105 with response rule 100092 were used for malware detection and removal, and rule 100900 was configured for suspicious PowerShell detection.
Verify detection using logs/events	Event ID 4625 was identified for failed logon, rule IDs 554/87105/100092/553 were identified for the malware detection chain, and Sysmon Event ID 1 was identified for PowerShell process creation.
Configure an appropriate response action	A Windows Firewall block was used for the brute force attacker IP, remove-threat.exe was used for malware removal, and a Windows Firewall outbound block was used for powershell.exe.
Confirm response successfully executed	Ping/Nmap validation confirmed Attack 1 containment, Test-Path False confirmed Attack 2 removal, and TcpTestSucceeded False confirmed Attack 3 PowerShell outbound block.

Overall Task 2 Result

Successful. The three selected attacks meet the XDR Detection and Automated Response requirements. The final evidence includes attack generation, detection rule configuration, Wazuh dashboard response evidence, event/log identification, response configuration, and response validation for each attack.

Task 3 - SOAR Automation Using Shuffle

Introduction

The project demonstrates the SOAR process through integration of Wazuh with Shuffle. The chosen scenario involves SSH brute force attacks on the Bodhi Linux machine. Wazuh monitors multiple failed login attempts via a custom rule that raises an alert, which is then sent to Shuffle via webhooks, and Shuffle notifies the SOC analyst via an automated email message.

Implementation Summary

Item	Implementation Evidence
Selected attack scenario	SSH brute force attack generated from Kali Linux against Bodhi Linux using Medusa.
Wazuh detection rule	Custom Wazuh rule ID 100300 detects repeated SSH authentication failures.
Shuffle integration	Wazuh forwards JSON alerts to the Shuffle webhook using the ossec.conf integration block.
Playbook design	The workflow uses Wazuh Alert Trigger -> Wazuh logs -> Email notification.
Automated response / notification	Shuffle sends an automated email notification containing the SSH brute force alert details.
Verification result	Workflow execution finished successfully, the alert details were received by Shuffle, and the email arrived in the inbox.

Task 3 Requirement Mapping

Requirement	Evidence Provided	Key Figure(s)	Status
Wazuh Rule Configuration	Custom rule 100300 configured and alert generated in Wazuh.	Figures 65, 67, 68	Complete
Shuffle Integration	Webhook trigger and Wazuh ossec.conf integration configured to forward JSON alerts.	Figures 69, 70	Complete
Playbook Design	Workflow created with Wazuh alert trigger, log processing, and email action.	Figures 71, 72	Complete

Automated Response / Notification	Email notification sent automatically after the Wazuh alert triggered Shuffle.	Figures 75, 76	Complete
Output / Verification	Execution history and received alert details confirm successful playbook run.	Figures 73, 74 and 76	Complete

Evidence Package

```

root@wazuh-server:~
GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml
<description>sshd: authentication failed from IP 1.1.1.1.</description>
<group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
</group>

<group name="virustotal,">
  <rule id="100092" level="12">
    <if_sid>657</if_sid>
    <match>Successfully removed threat</match>
    <description>$(parameters.program) removed threat located at $(parame
  </rule>

  <rule id="100093" level="12">
    <if_sid>657</if_sid>
    <match>Error removing threat</match>
    <description>Error removing threat located at $(parameters.alert.data.v
  </rule>
</group>

<group name="ssh,authentication_failed,bruteforce,">

  <rule id="100300" level="10" frequency="5" timeframe="60">
    <if_matched_sid>5503</if_matched_sid>
    <description>SSH Brute Force Attack Detected on Bodhi Linux</descriptio
    <group>authentication_failed,sshd,bruteforce</group>
  </rule>
</group>

<group name="sysmon,powershell,lab,">
  <rule id="100900" level="12">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.image" type="pcre2">(?!)\powershell\.exe$</>
    <field name="win.eventdata.commandLine" type="pcre2">(?!)-nop</field>
    <description>LAB ALERT: PowerShell executed with -nop parameter</descri
    <group>powershell,suspicious_process,sysmon_event1</group>
  </rule>
</group>

```

Figure 65: Custom Wazuh rule 100300 configured in local_rules.xml to detect repeated SSH authentication failures on Bodhi Linux.

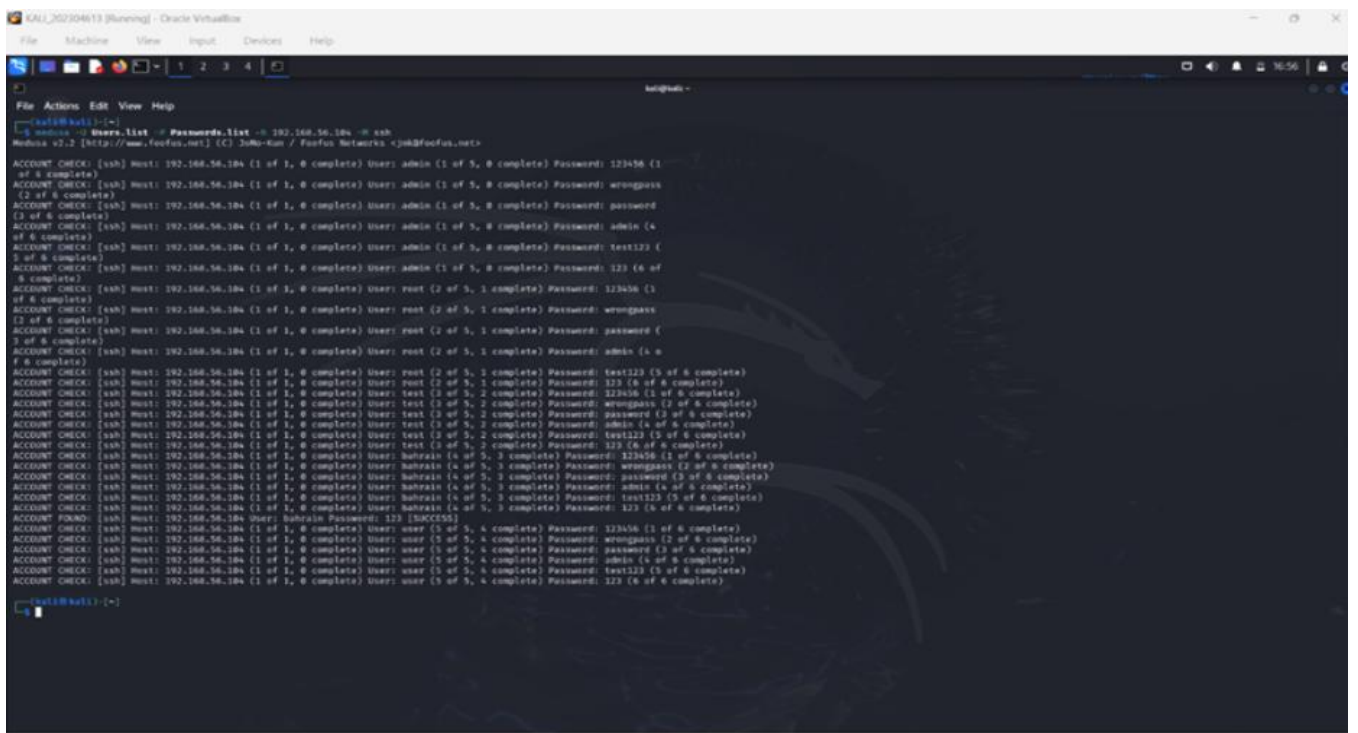


Figure 66: Kali Linux running Medusa to generate SSH brute force authentication attempts against the Bodhi Linux target.

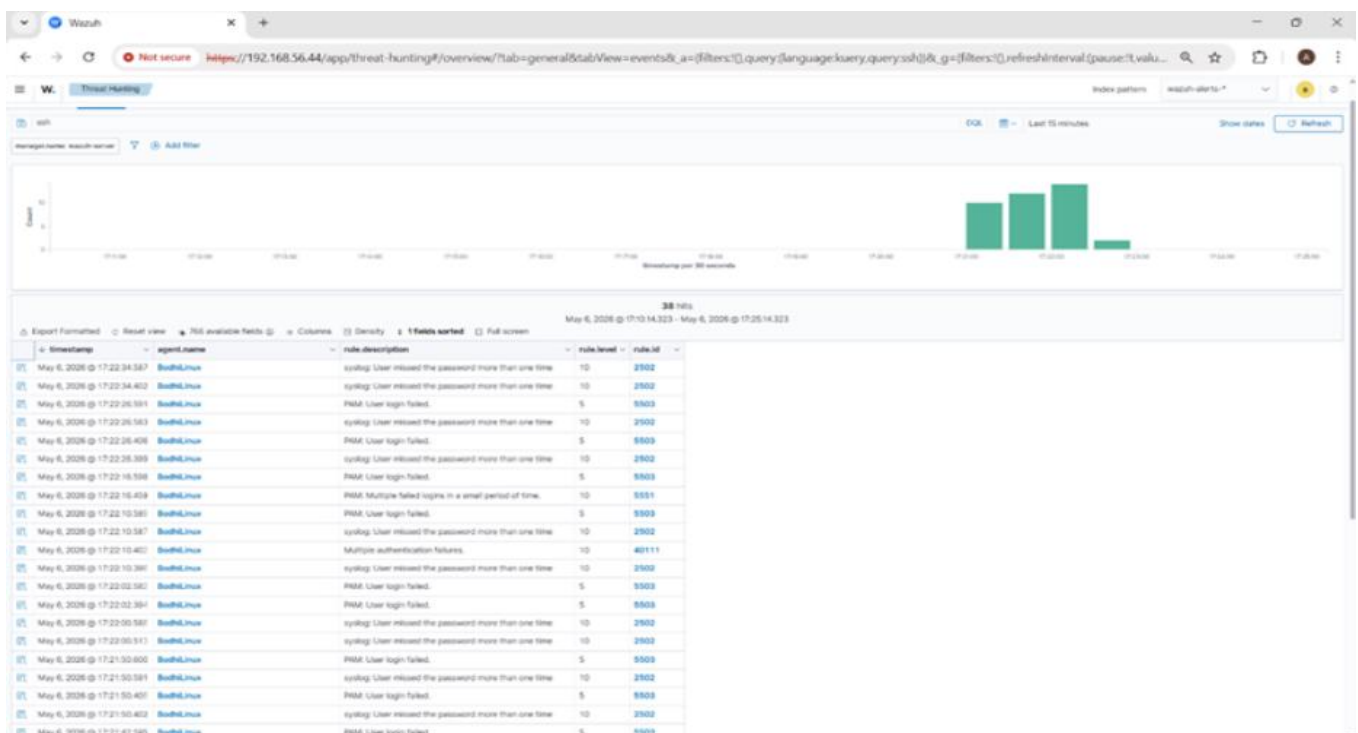


Figure 67: Wazuh dashboard showing the SSH brute force alert generated by custom rule ID 100300.

Document Details		View surrounding documents	View single document
agent.id	002		
agent.ip	192.168.56.104		
agent.name	BodhiLinux		
data.dstuser	bahrain		
data.euid	0		
data.srcip	192.168.56.105		
data.tty	ssh		
data.uid	0		
decoder.name	pam		
full_log	May 06 22:12:11 bahrain-VirtualBox sshd[4849]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.105 user=bahrain		
id	1778105531.1266791		
input.type	log		
location	journal		
manager.name	wazuh-server		
predecoder.hostname	bahrain-VirtualBox		
predecoder.program_name	sshd		
predecoder.timestamp	May 06 22:12:11		
previous_output	May 7 01:12:11 bahrain-VirtualBox sshd[4849]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.105 user=bahrain May 06 22:12:00 bahrain-VirtualBox sshd[4847]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.105 May 7 01:12:00 bahrain-VirtualBox sshd[4847]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.105 May 06 22:11:51 bahrain-VirtualBox sshd[4845]: pam_unix(sshd:auth): authentication failure		
rule.description	SSH Brute Force Attack Detected on Bodhi Linux		
rule.firedtimes	8		
rule.frequency	5		
rule.groups	ssh, authentication_failed, bruteforce, authentication_failed, sshd, bruteforce		
rule.id	100300		
rule.level	10		
rule.mail	false		
timestamp	May 7, 2026 @ 01:12:11.707		

Figure 68: Wazuh alert document details showing the SSH brute force alert fields, including BodhiLinux agent information, source IP 192.168.56.105, rule ID 100300, frequency, fired times, and severity level.

The screenshot displays the Shuffle webhooks configuration interface. On the left, a sidebar lists various triggers and apps. The main workspace shows a workflow titled 'Wazuh SSH Brute Force Notification'. A 'Wazuh logs' trigger is connected to a 'Wazuh Alert Trigger' app. The 'Wazuh Alert Trigger' app is configured with the following parameters:

- Webhook URI:** `r1/hooks/webhook_388f9700-a424-42`
- Authentication headers:** `AUTH_HEADER=AUTH_VALUE1`

The 'Webhook' status is 'running'. The interface also includes a 'Runtime Location (Cloud)' dropdown and a 'Save' button at the bottom.

Figure 69: Shuffle webhook trigger running and ready to receive alerts from Wazuh.

```
root@wazuh-server:~  
GNU nano 8.3 /var/ossec/etc/ossec.conf Modified  
</localfile>  
  
<localfile>  
  <log_format>syslog</log_format>  
  <location>/var/ossec/logs/active-responses.log</location>  
</localfile>  
<integration>  
  <name>virustotal</name>  
  <api_key> dec54d6559e9d0834658a53685c285244094e1b62448df93d1a4a1b735cfb12c</api_key>  
  <group>syscheck</group>  
  <alert_format>json</alert_format>  
</integration>  
  
<command>  
  <name>remove-threat</name>  
  <executable>remove-threat.exe</executable>  
  <timeout_allowed>no</timeout_allowed>  
</command>  
  
<active-response>  
  <disabled>no</disabled>  
  <command>remove-threat</command>  
  <location>local</location>  
  <rules_id>87105</rules_id>  
</active-response>  
  
<integration>  
  <name>virustotal</name>  
  <api_key>dec54d6559e9d0834658a53685c285244094e1b62448df93d1a4a1b735cfb12c</api_key>  
  <group>syscheck</group>  
  <alert_format>json</alert_format>  
</integration>  
  
<integration>  
  <name>shuffle</name>  
  <hook_url>https://shuffler.io/api/v1/hooks/webhook_388f9700-a424-4200-bff</hook_url>  
  <rule_id>100300</rule_id>  
  <alert_format>json</alert_format>  
</integration>  
  
</ossec_config>  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Figure 70: Wazuh ossec.conf integration configuration forwarding JSON alerts from Wazuh to the Shuffle webhook.

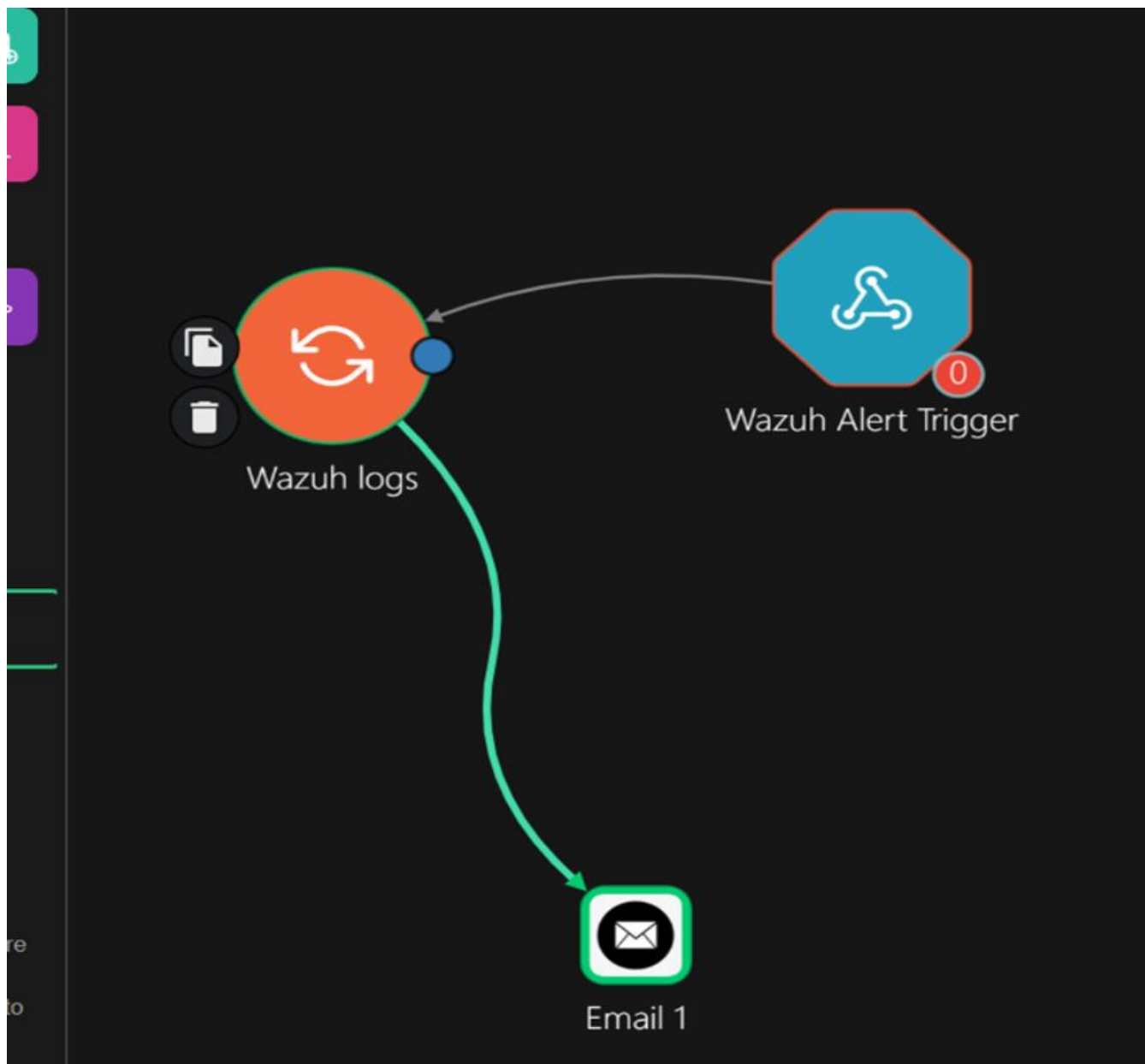


Figure 71: Complete Shuffle SOAR playbook configured as Wazuh Alert Trigger → Wazuh logs → Email 1.

Setup

Configuration



Send email shuffle

Recipients *

3bdulrahmanaltairey@gmail.com

Subject *

SSH Brute Force Alert - Wazuh Rule 1

Body *

Wazuh detected an SSH brute force attack on BodhiLinux.

Rule ID: 100300

Alert: SSH Brute Force Attack Detected on Bodhi Linux

Test Action

Figure 72: Shuffle Email action configuration showing the recipient, alert subject, and notification body for rule ID 100300.

Wazuh Workflow - Wazuh SSH Brute F

shuffler.io/workflows/af41cd69-e718-4b93-9c83-890f162c0ca8?execution_id=bcf49...

Search apps, triggers...

Popular Actions

Triggers

Your Apps

AI Agent

Singul

Shuffle Tools

Http

Email

Gmail

Apps need to be activated before they can be used. Search in the search bar from our 2500+ apps to activate them for.

Back to all runs

Details

Status FINISHED

Source webhook

Started 07/05/2026, 01:12:13

Finished 07/05/2026, 01:12:14

Location Cloud

```
{
  "$exec": {
    "severity": 3
    "pretext": "WAZUH Alert"
    "title": "SSH Brute Force Attack Detected on Bodhi Linux"
    "text": "May 06 22:12:11 bahrain-VirtualBox sshd[4849]: pam_unix(sshd:auth): authentication failure; logname=..."
    "rule_id": "100300"
    "timestamp": "2026-05-06T22:12:11.707+0000"
    "id": "1778105531.1266791"
    "all_fields": {...}
  }
}
```

Figure 73: Shuffle execution history showing a finished workflow run after the Wazuh alert triggered the playbook.

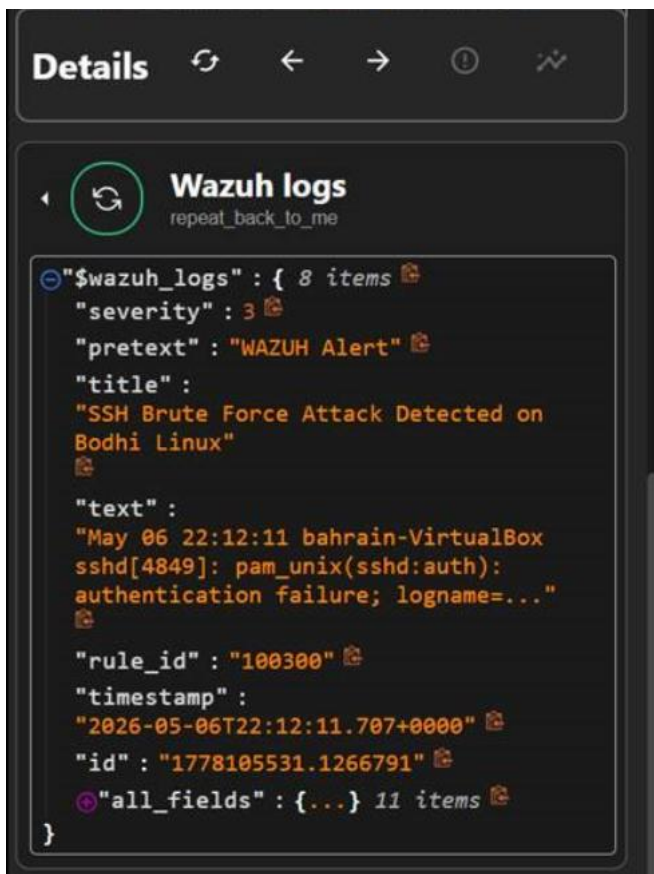


Figure 74: Shuffle Wazuh logs action showing the received SSH brute force alert details and rule ID 100300.

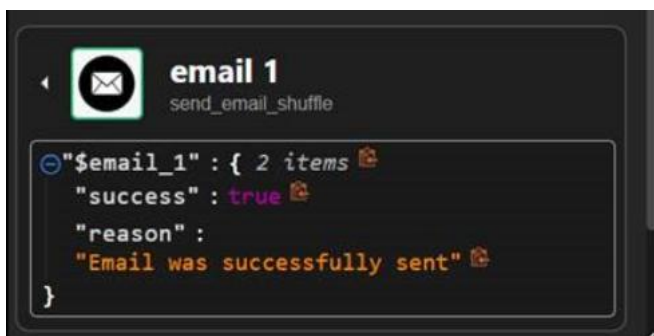


Figure 75: Shuffle Email action result showing that the automated SOC notification email was successfully sent.

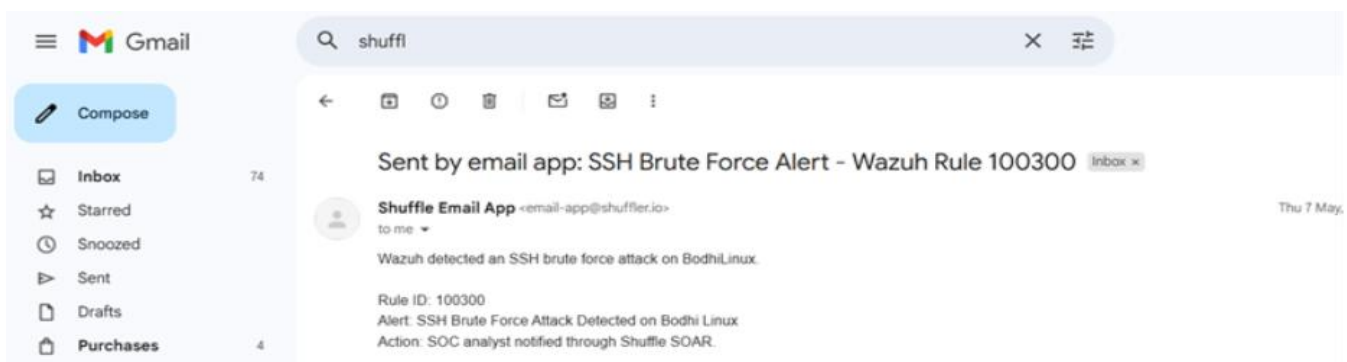


Figure 76: Email inbox showing the automated SSH brute force alert received from Shuffle.

Task 3 Result

Result: Success. The SSH brute-force attack detection was made possible via Wazuh with the help of a custom rule having ID 100300. It triggered the playbook on Shuffle through the Wazuh webhook integration, the execution of which was successful, as well as the sending of an e-mail notification as required in this case study.

Task 3 Deliverables Checklist

- Wazuh rules configuration proof: accomplished with custom rule ID 100300.
- Threat detection proof: accomplished with Wazuh dashboard and alert document information.
- Shuffle integration proof: accomplished with webhook trigger and ossec.conf integration screen capture.
- Playbook design proof: accomplished with the whole Shuffle process flow and Email action configuration.
- Automated notification proof: accomplished with Shuffle Email output and email inbox screen capture.

Task 4: Threat Intelligence Integration into Wazuh

Objective

The objective of this task was to integrate the external threat intelligence sources that have been chosen in Part A into Wazuh. This integration helps enrich security monitoring by enabling Wazuh to ingest and utilize threat intelligence data and improves detection and analysis of security events all over the monitored infrastructure.

The three external threat intelligence sources that have been chosen and used are: VirusTotal, AbuseIPDB, AlienVaultOTX.

Source 1: VirusTotal

Integration Method: Built-in API Integration

VirusTotal was integrated into Wazuh, using its built-in native integration module, this module was combined with File Integrity Monitoring (FIM) capability. This integration working by monitoring a specific directory on the Windows Server endpoint. When FIM detects a new file or a file that has been modified, it automatically sends the cryptographic hashes over to VirusTotal API right away. The API then scans the hash against 70 antivirus engines and comes back with a threat verdict. If the file is to be identified as malicious, Wazuh generated a high-severity alert and triggers an automated active response which removes the threat from the endpoint.

Configuration

The first step was to configure FIM on the Windows Server agent. The Wazuh agent configuration file (ossec.conf) was edited to enable real-time monitoring of the Downloads directory. This ensures any file that has been placed or modified in that directory to be immediately detected and its hashes sent to VirusTotal.

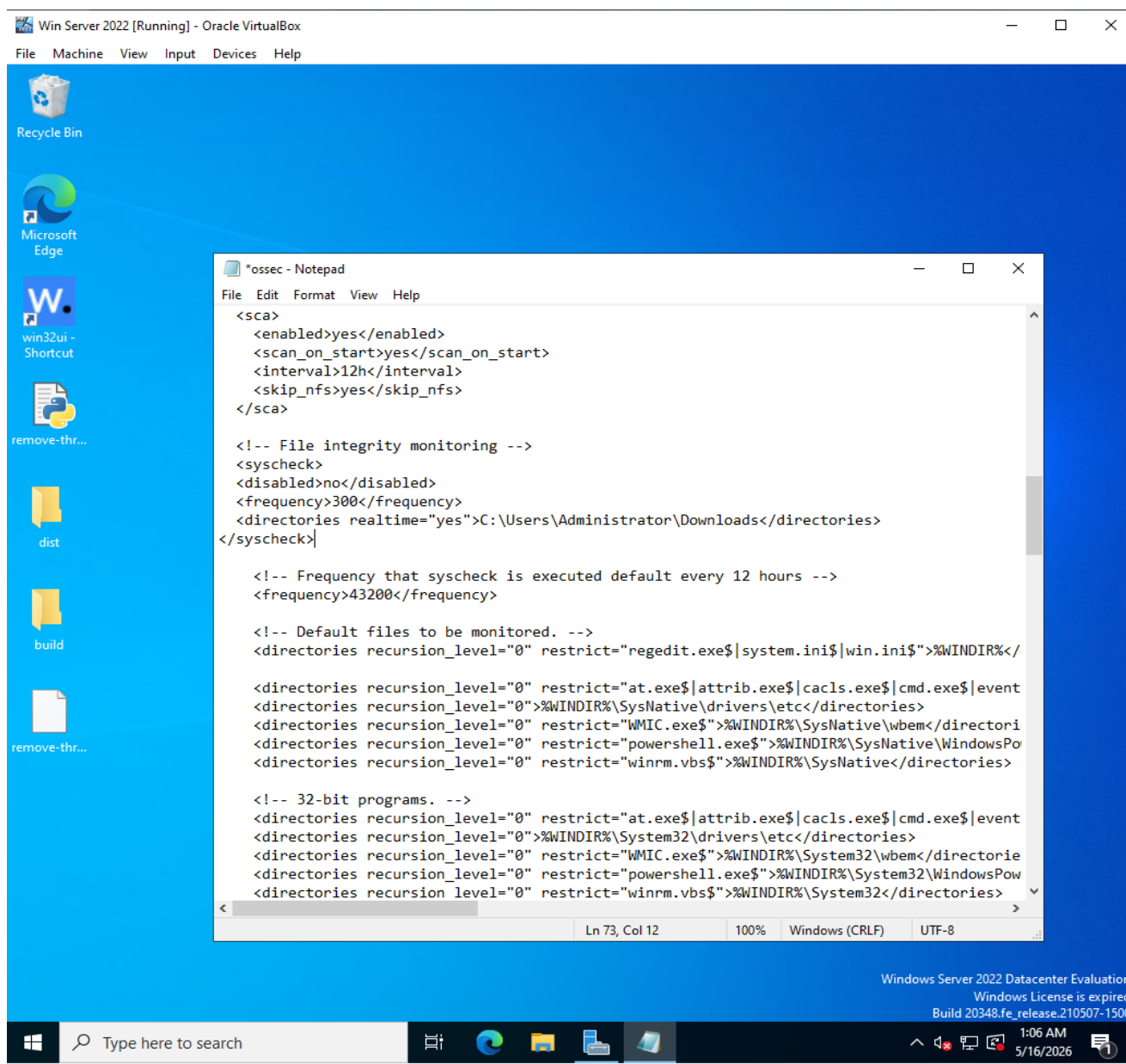


Figure 77: ossec.conf file in Windows showing the syscheck block enabled with real-time monitoring for the Downloads directory.

Python was then installed on Windows in order to support the action of automated threat removal capability which will be used later, and PyInstaller was then used to convert an active response Python script into a Windows executable. The script that was converted reads the VirusTotal alert data that come from Wazuh, extracts malicious file path, and makes sure to securely delete the file immediately from the endpoint.

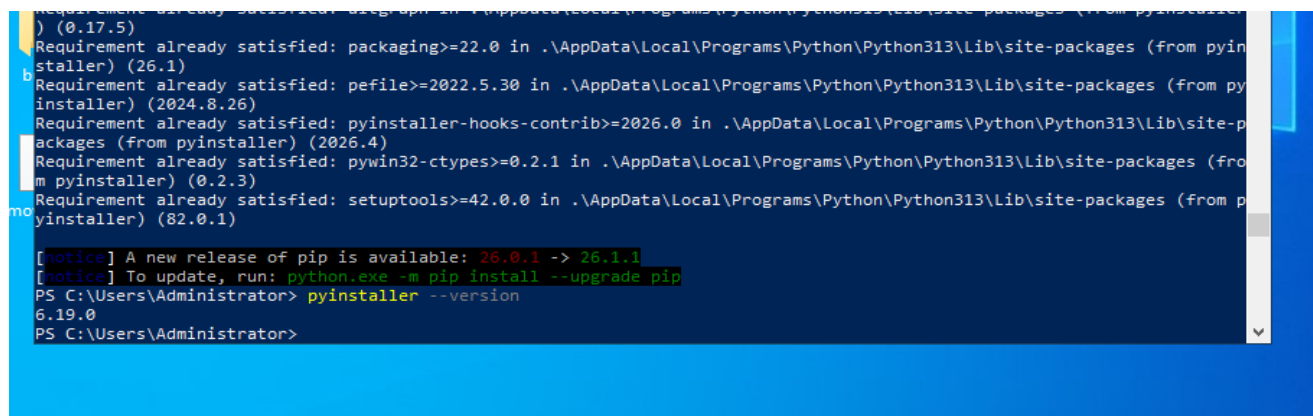


Figure 78: PowerShell showing Python version installed and confirming PyInstaller version.

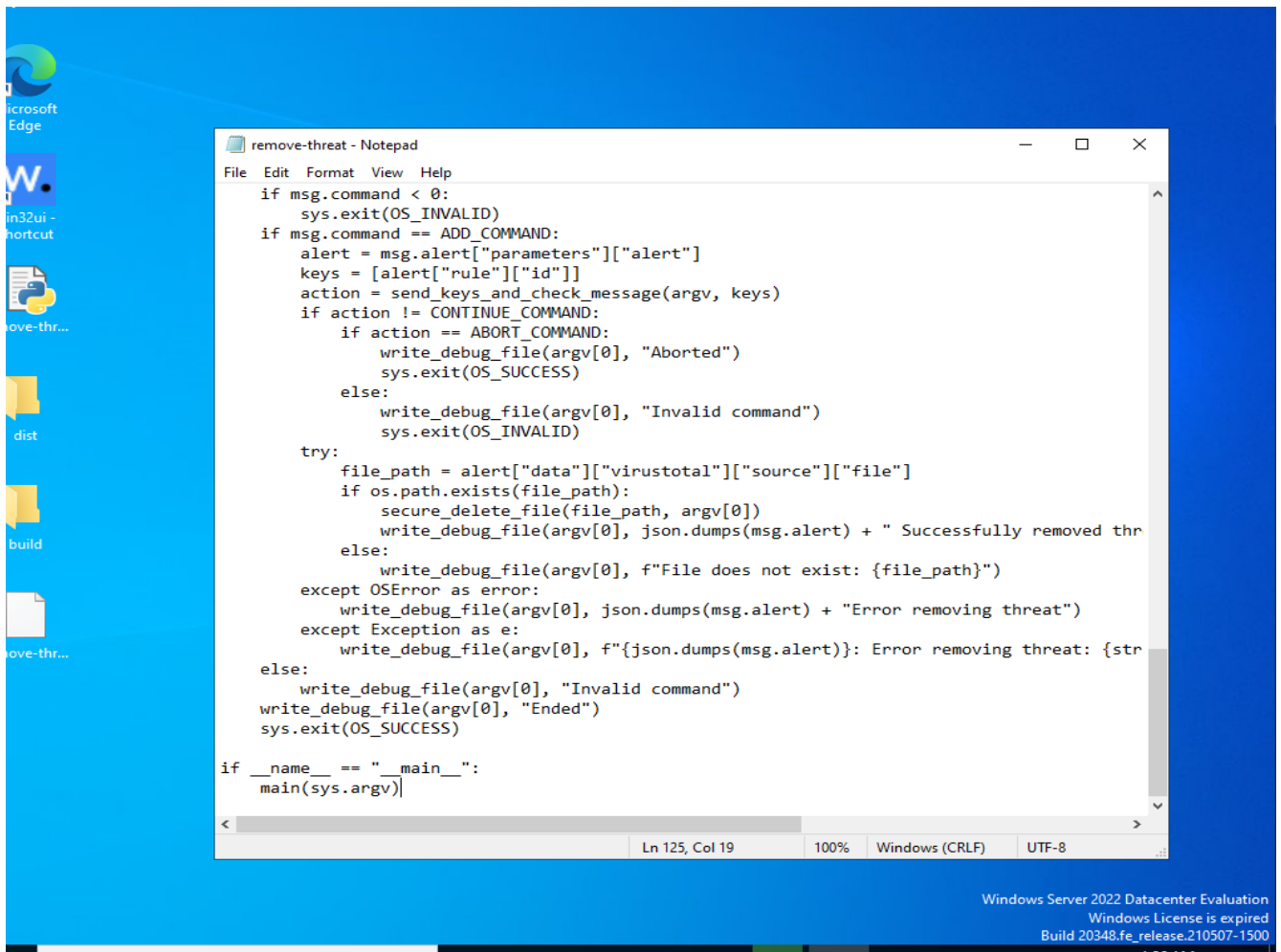


Figure 79: The remove-threat.py active response script open in Notepad on Windows.

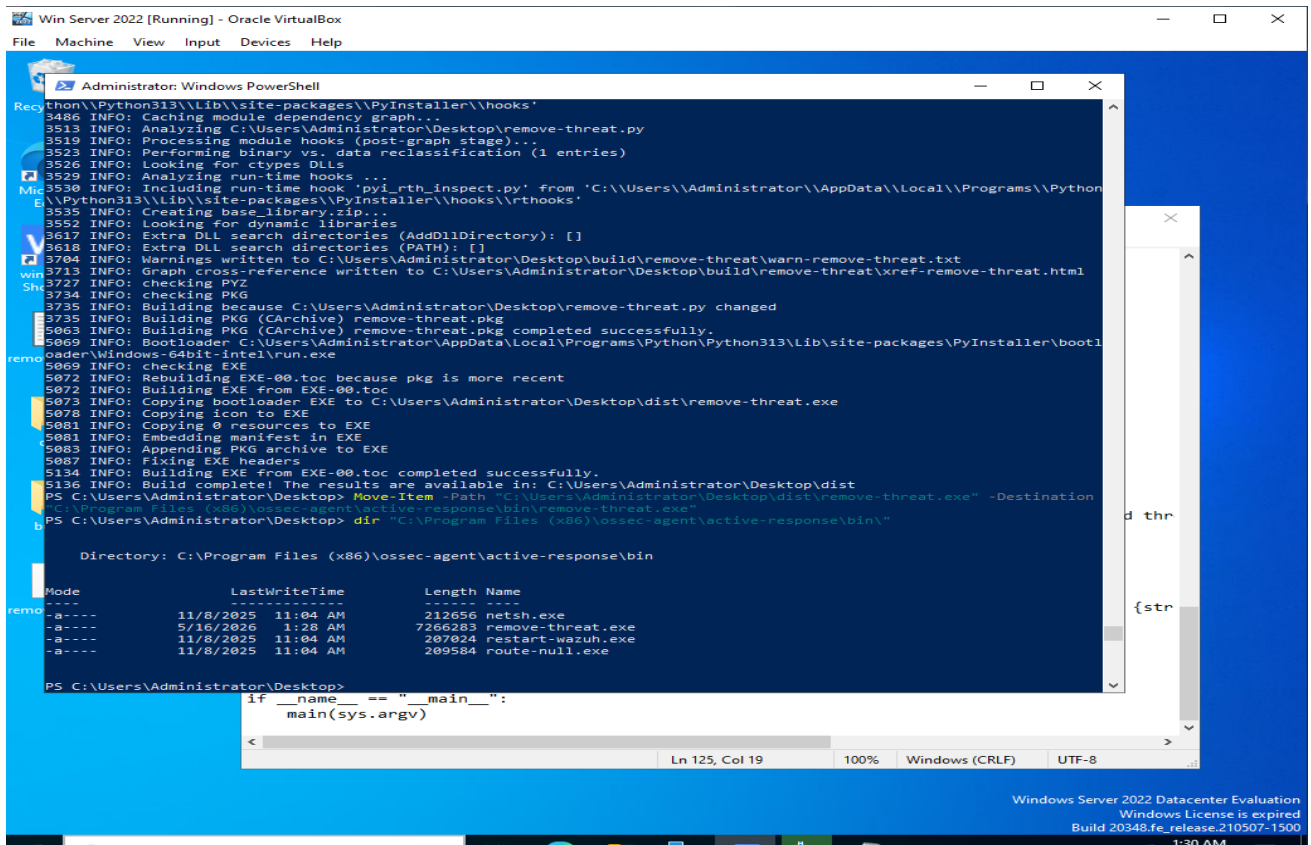
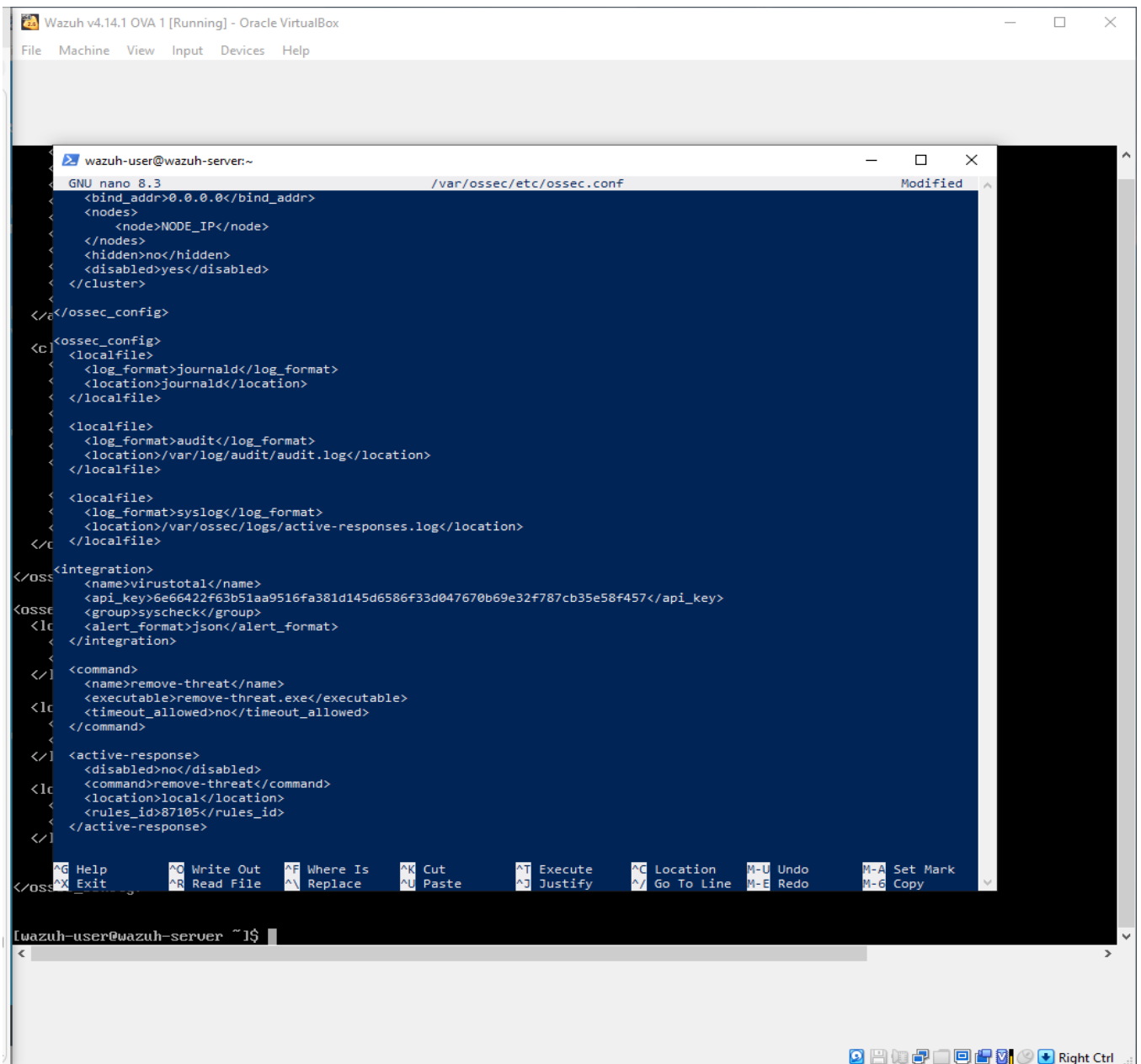


Figure 80: Windows PowerShell showing remove-threat.exe placed in the active-response\bin\ directory.

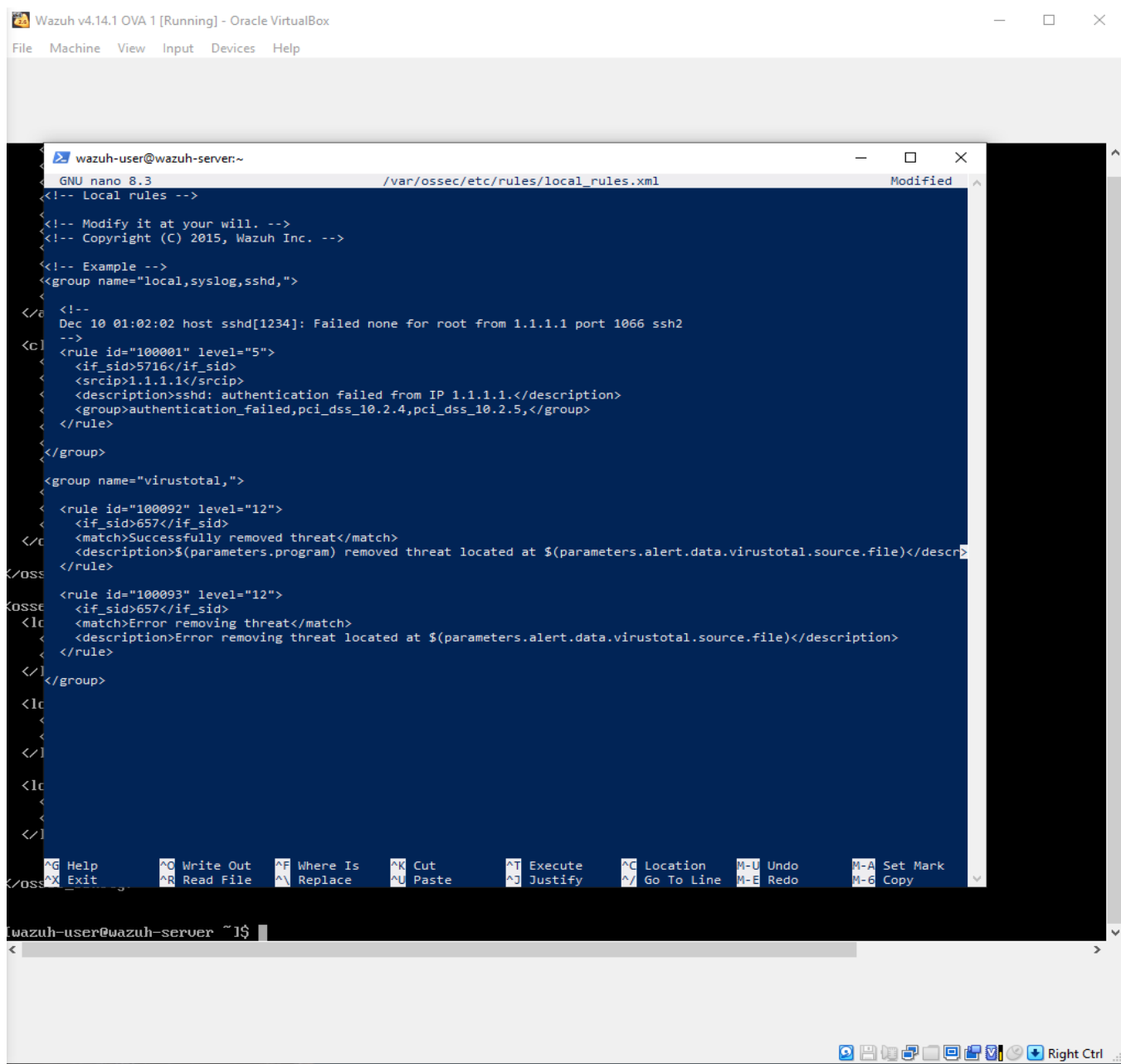
On the Wazuh Manager, the VirusTotal integration block was added to the main configuration file. The integration uses the syscheck group trigger, which means it fires whenever any FIM rule has been activated. The VirusTotal API key was taken from the VirusTotal website and was then pasted and configured in the integration block. The active response blocks were also included into the script, in order to instruct Wazuh on executing the remove-threat.exe script whenever rule 87105 fires.



```
GNU nano 8.3 /var/ossec/etc/ossec.conf Modified
<bind_addr>0.0.0.0</bind_addr>
<nodes>
  <node>NODE_IP</node>
</nodes>
<hidden>no</hidden>
<disabled>yes</disabled>
</cluster>
</ossec_config>
<ossec_config>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>
  <localfile>
    <log_format>audit</log_format>
    <location>/var/log/audit/audit.log</location>
  </localfile>
  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>
</ossec_config>
<integration>
  <name>virustotal</name>
  <api_key>6e66422f63b51aa9516fa381d145d6586f33d047670b69e32f787cb35e58f457</api_key>
  <group>syscheck</group>
  <alert_format>json</alert_format>
</integration>
<command>
  <name>remove-threat</name>
  <executable>remove-threat.exe</executable>
  <timeout_allowed>no</timeout_allowed>
</command>
<active-response>
  <disabled>no</disabled>
  <command>remove-threat</command>
  <location>local</location>
  <rules_id>87105</rules_id>
</active-response>
</ossec_config>
```

Figure 81: Wazuh Manager ossec.conf showing the VirusTotal integration block and active response configuration.

In a new file called `local_rules.xml`, custom local rules were created and added to generate alerts that confirm when the active response script successfully removed the threat.



The screenshot shows a terminal window titled "Wazuh v4.14.1 OVA 1 [Running] - Oracle VirtualBox". Inside the terminal, a nano editor is open, editing the file `/var/ossec/etc/rules/local_rules.xml`. The file contains XML rules for Wazuh. The rules are organized into groups: "local,syslog,sshd," and "virustotal,". The "local,syslog,sshd," group contains a rule with id "100001" that triggers an alert when an SSH authentication fails from IP 1.1.1.1. The "virustotal," group contains two rules: rule id "100092" which triggers an alert when a threat is successfully removed, and rule id "100093" which triggers an alert when there is an error removing a threat. Both rules in the "virustotal," group use the `$(parameters.alert.data.virustotal.source.file)` placeholder in their descriptions. The terminal window also shows a command prompt at the bottom with the prompt `wazuh-user@wazuh-server ~1$`.

```
wazuh-user@wazuh-server:~  
GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml Modified  
<!-- Local rules -->  
  
<!-- Modify it at your will. -->  
<!-- Copyright (C) 2015, Wazuh Inc. -->  
  
<!-- Example -->  
<group name="local,syslog,sshd,">  
  
<!--  
Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2  
-->  
<rule id="100001" level="5">  
  <if_sid>5716</if_sid>  
  <srcip>1.1.1.1</srcip>  
  <description>sshd: authentication failed from IP 1.1.1.1.</description>  
  <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>  
</rule>  
  
</group>  
  
<group name="virustotal,">  
  
  <rule id="100092" level="12">  
    <if_sid>657</if_sid>  
    <match>Successfully removed threat</match>  
    <description>$(parameters.program) removed threat located at $(parameters.alert.data.virustotal.source.file)</description>  
  </rule>  
  
  <rule id="100093" level="12">  
    <if_sid>657</if_sid>  
    <match>Error removing threat</match>  
    <description>Error removing threat located at $(parameters.alert.data.virustotal.source.file)</description>  
  </rule>  
  
</group>  
  
</>  
  
^G Help      ^O Write Out  ^F Where Is  ^K Cut       ^T Execute   ^C Location  ^U Undo      ^M Set Mark  
^X Exit      ^R Read File  ^A Replace   ^U Paste     ^D Justify   ^_ Go To Line ^E Redo      ^G Copy  
wazuh-user@wazuh-server ~1$
```

Figure 82: The `local_rules.xml` file showing custom rules 100092 and 100093 configured to alert on active response execution results.

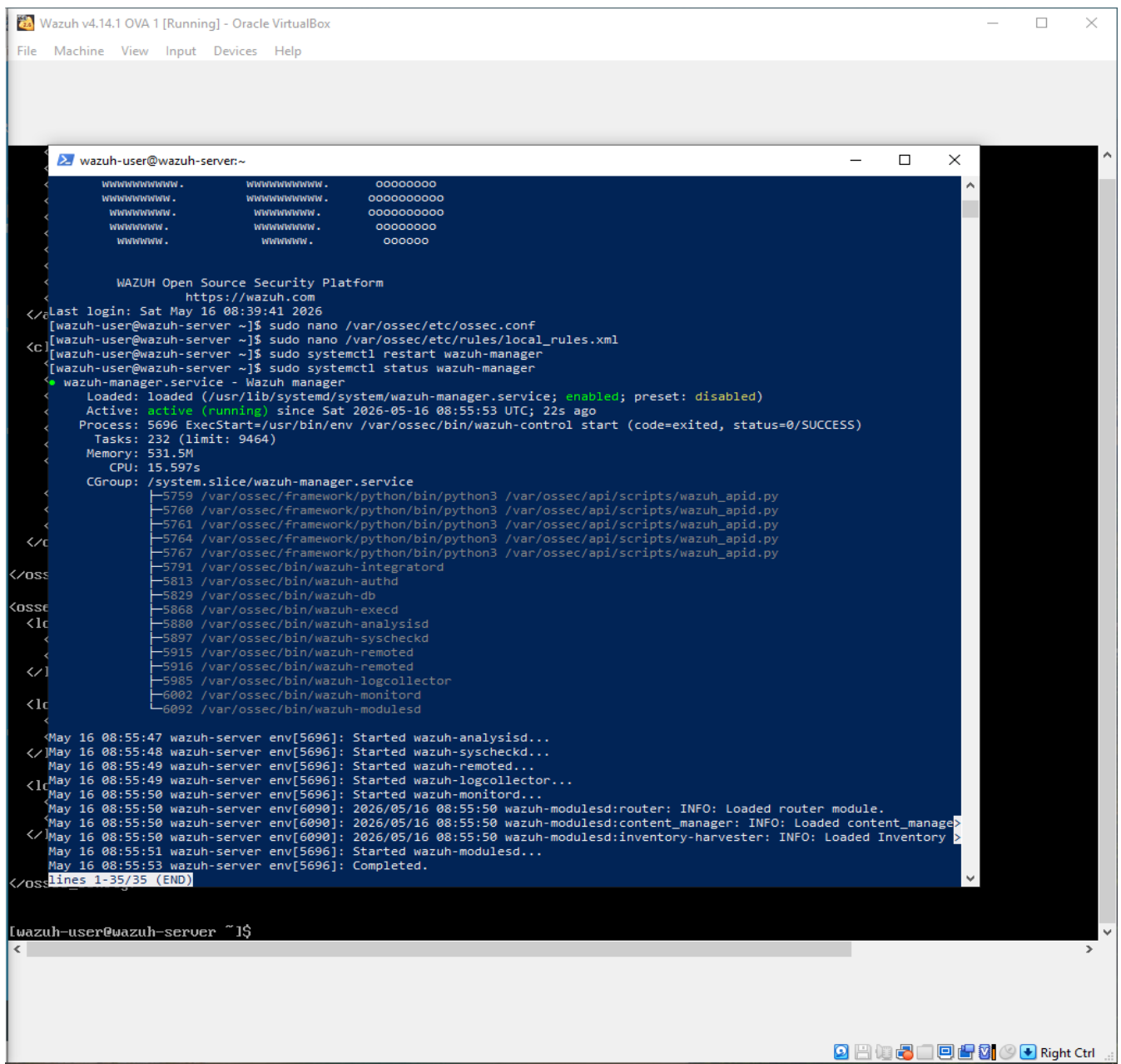
The image shows a terminal window titled 'wazuh-user@wazuh-server:~' within an Oracle VM VirtualBox environment. The terminal output displays the Wazuh Open Source Security Platform logo, the website 'https://wazuh.com', and the last login time 'Sat May 16 08:39:41 2026'. It then shows the execution of several commands: 'sudo nano /var/ossec/etc/ossec.conf', 'sudo nano /var/ossec/etc/rules/local_rules.xml', 'sudo systemctl restart wazuh-manager', and 'sudo systemctl status wazuh-manager'. The status output indicates that the 'wazuh-manager.service' is loaded, enabled, and active (running) since May 16, 2026, at 08:55:53 UTC. It also lists the process ID (5696), tasks (232), memory (531.5M), and CPU (15.597s) usage. A detailed list of processes is shown, including various Wazuh components like wazuh_apid.py, wazuh-integratord, wazuh-authd, wazuh-db, wazuh-execd, wazuh-analysisd, wazuh-syscheckd, wazuh-remoted, wazuh-logcollector, wazuh-monitord, and wazuh-modulesd. The terminal also shows a series of log messages indicating the successful start of these services. The bottom of the terminal shows the prompt '[wazuh-user@wazuh-server ~]\$'.

Figure 83: Terminal showing the Wazuh Manager service active and running after configuration changes were applied.

Verification of Integration

To verify whether the integration works, Windows Defender real-time protection was temporarily disabled and the EICAR antivirus test file was downloaded to the monitored Downloads folder on Windows. The EICAR antivirus test file is a globally recognized harmless test file used to validate threat detection systems. Wazuh's FIM module detected the file, and sent its hash to VirusTotal, then received a positive malicious verdict, which made the active response script triggered and immediately and automatically delete the file. It generated a complete alert chain in the Wazuh Dashboard.

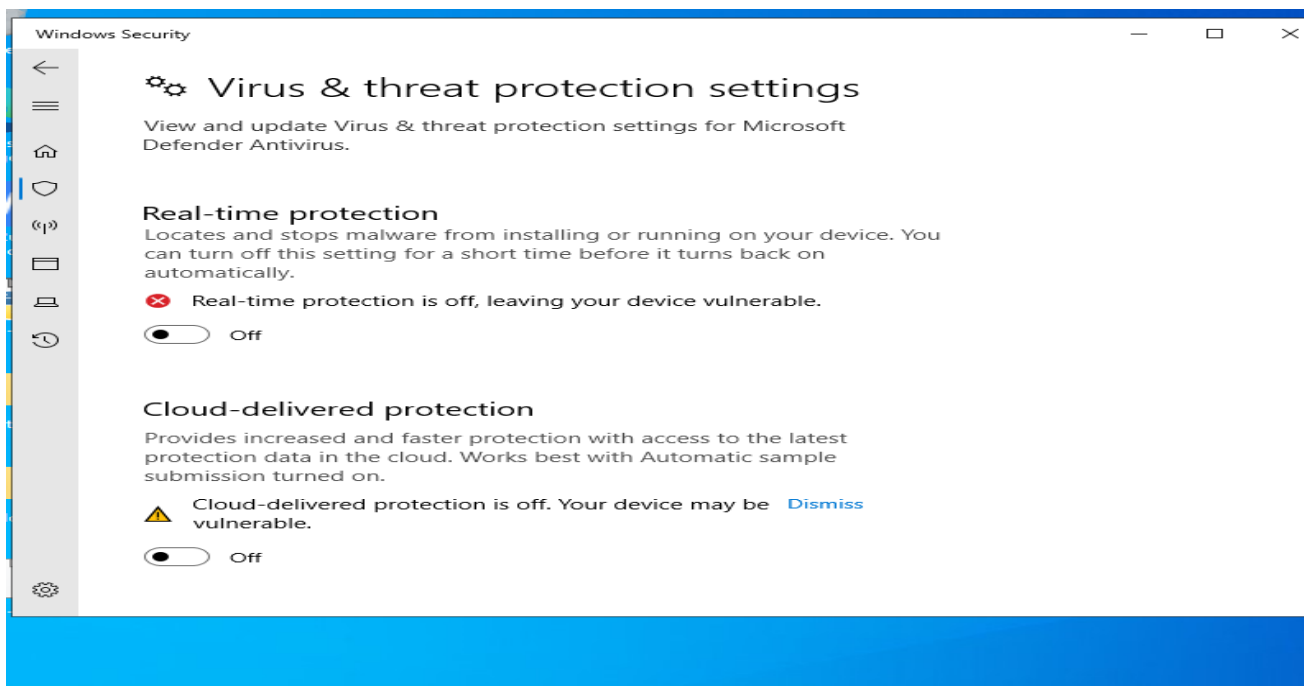


Figure 84: Windows Security showing Real-time protection turned Off.

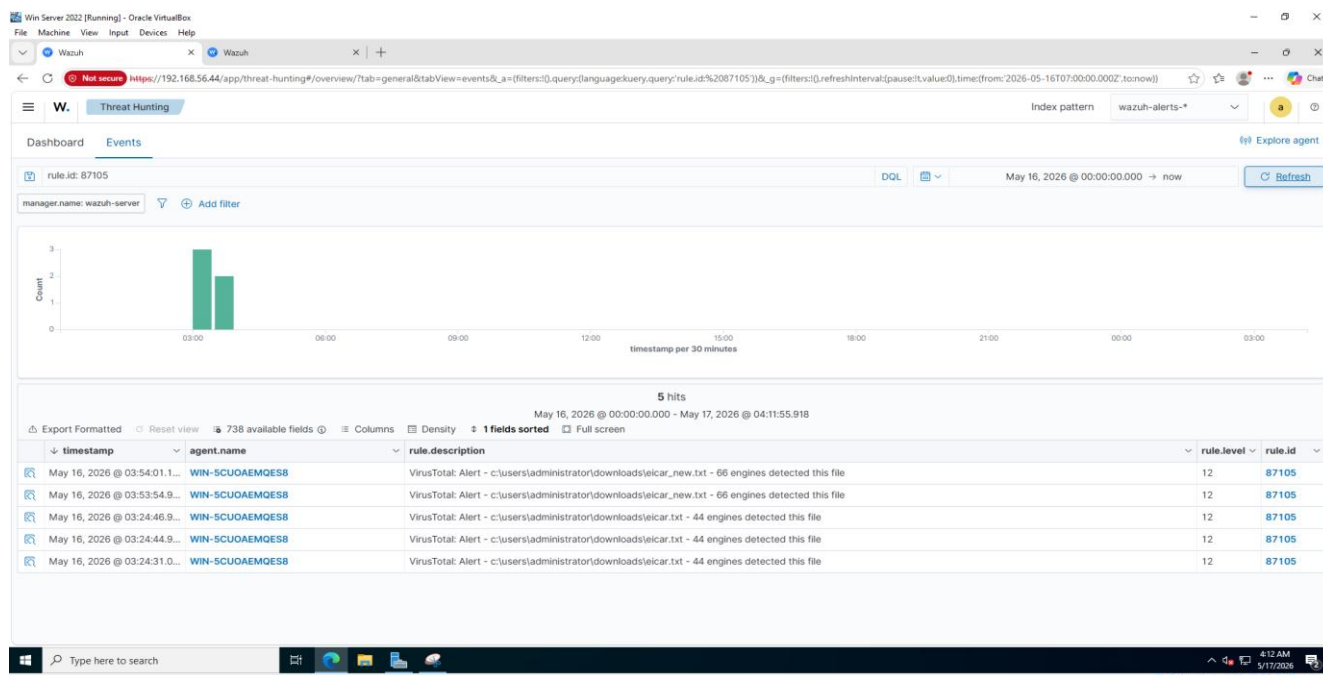
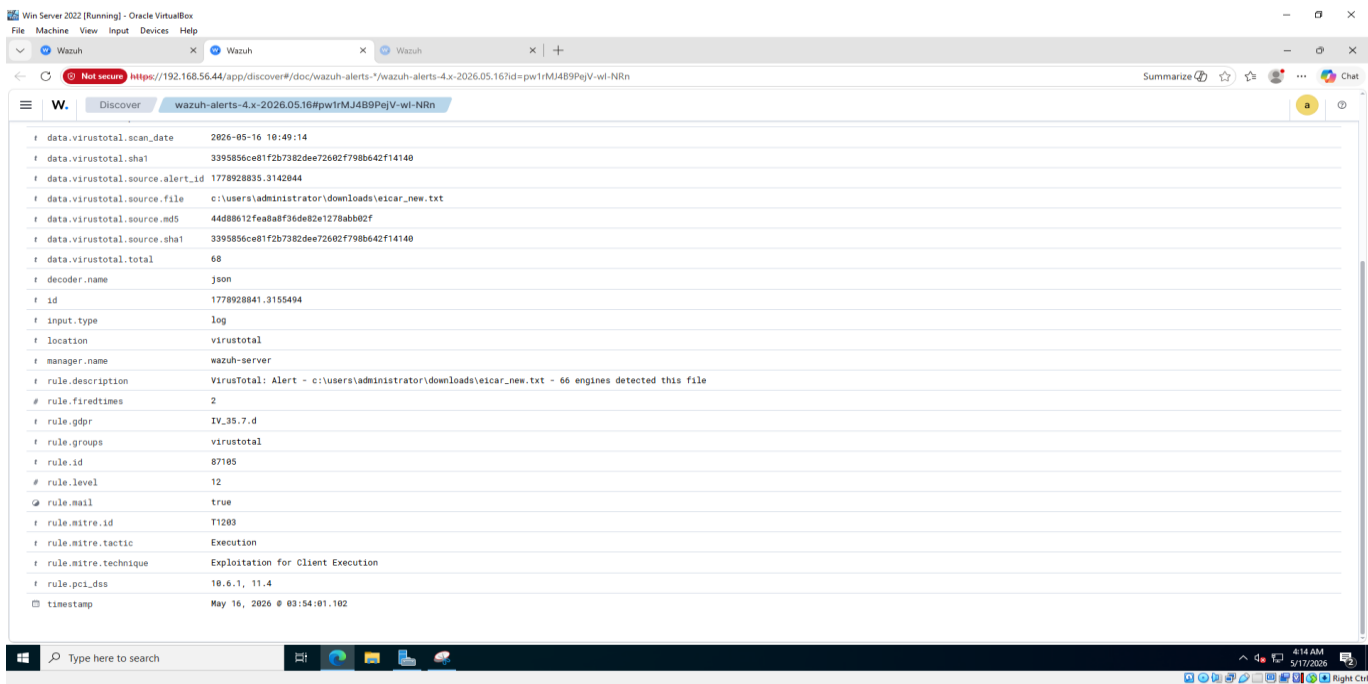
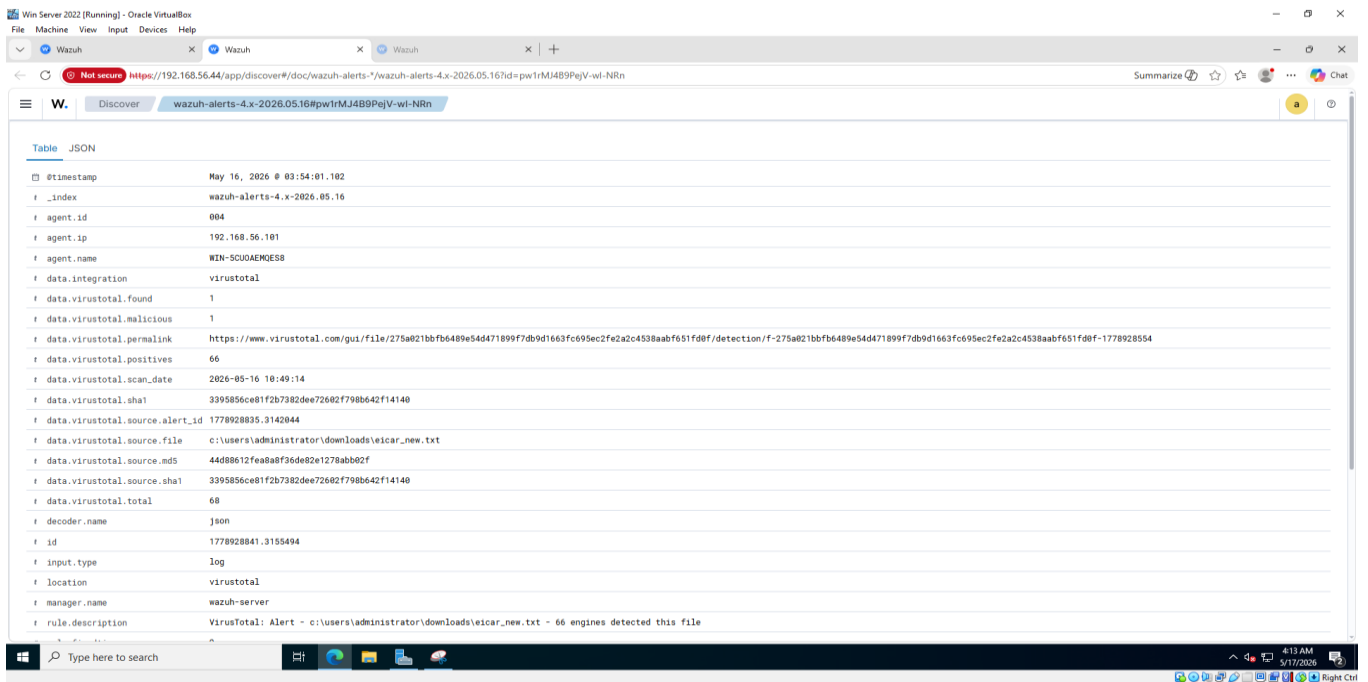


Figure 85: Wazuh Threat Hunting Events filtered by rule.id 87105, showing 5 VirusTotal alerts from WIN-5CUOAEMQES8 — 66 engines detected eicar_new.txt and 44 engines detected eicar.txt.



Figures 86 & 87: Expanded rule 87105 alert showing VirusTotal enrichment fields; *virustotal.found* = 1, *virustotal.malicious* = 1, *virustotal.positives* = 66, *virustotal.total* = 68, *permalink* to scan report, and *source file path* ecar_new.txt.

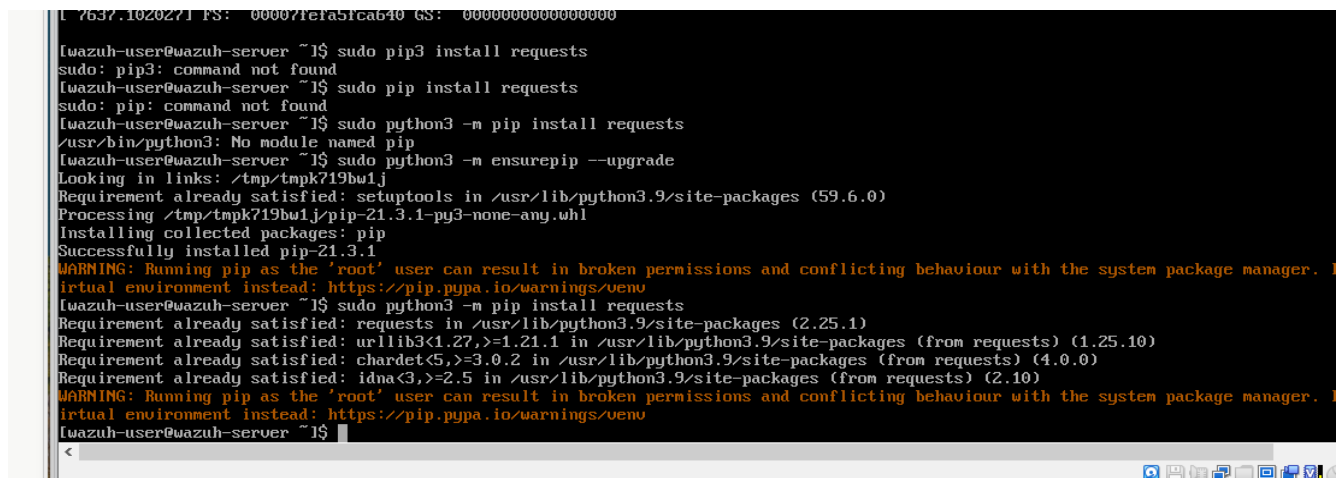
Source 2: AbuseIPDB

Integration Method: API Integration via Custom Python Script

AbuseIPDB was integrated into Wazuh using a custom Python integration script. In Wazuh, custom integrations are supported by allowing Python scripts to be placed in the integrations directory and configured to trigger on specific rule IDs. When a brute-force or authentication failure alert fires, Wazuh calls the Python script with the alert data, and the script then queries the AbuseIPDB API to retrieve threat intelligence about the attacking IP address. The API response comes back with details, including: IP's abuse confidence score, the total number of times it has been reported by the global security community, also the country of origin, and the ISP information.

Configuration

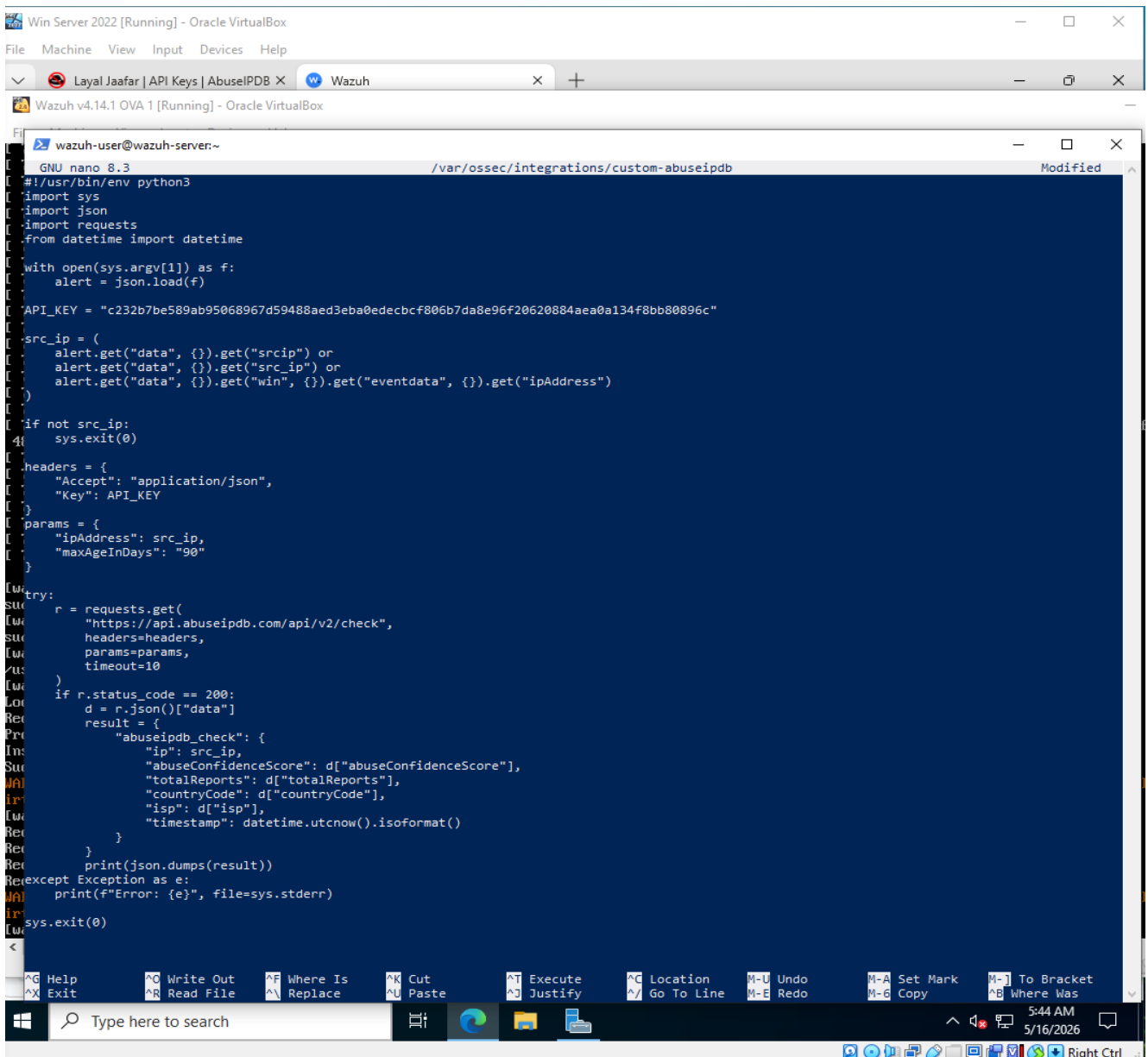
As the Python requests library is required, it was verified to be installed on Wazuh. It is required by the integrations script to make HTTP requests to the AbuseIPDB API.

A terminal window screenshot showing the installation of the Python requests library. The user is at a Wazuh server. The commands entered are: `sudo pip3 install requests`, `sudo pip install requests`, and `sudo python3 -m pip install requests`. The output shows that the first two commands fail because the command is not found. The third command succeeds, showing the installation of pip-21.3.1 and the requests library. The terminal output includes warnings about running pip as the root user and the installation of requests (2.25.1) and its dependencies (urllib3, chardet, idna).

```
[wazuh-user@wazuh-server ~]$ sudo pip3 install requests
sudo: pip3: command not found
[wazuh-user@wazuh-server ~]$ sudo pip install requests
sudo: pip: command not found
[wazuh-user@wazuh-server ~]$ sudo python3 -m pip install requests
/usr/bin/python3: No module named pip
[wazuh-user@wazuh-server ~]$ sudo python3 -m ensurepip --upgrade
Looking in links: /tmp/tmpk719bw1j
Requirement already satisfied: setuptools in /usr/lib/python3.9/site-packages (59.6.0)
Processing /tmp/tmpk719bw1j/pip-21.3.1-py3-none-any.whl
Installing collected packages: pip
Successfully installed pip-21.3.1
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use the virtual environment instead: https://pip.pypa.io/warnings/venv
[wazuh-user@wazuh-server ~]$ sudo python3 -m pip install requests
Requirement already satisfied: requests in /usr/lib/python3.9/site-packages (2.25.1)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/lib/python3.9/site-packages (from requests) (1.25.10)
Requirement already satisfied: chardet<5,>=3.0.2 in /usr/lib/python3.9/site-packages (from requests) (4.0.0)
Requirement already satisfied: idna<3,>=2.5 in /usr/lib/python3.9/site-packages (from requests) (2.10)
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use the virtual environment instead: https://pip.pypa.io/warnings/venv
[wazuh-user@wazuh-server ~]$
```

Figure 88: Terminal showing the Python requests library successfully installed on Wazuh.

The custom integration script was created at `/var/ossec/integrations/custom-abuseipdb`. This script reads the Wazuh alert JSON file, it extracts the source IP address, and queries the AbuseIPDB API to get its threat intelligence profile.



```
GNU nano 8.3 /var/ossec/integrations/custom-abuseipdb Modified
#!/usr/bin/env python3
import sys
import json
import requests
from datetime import datetime

with open(sys.argv[1]) as f:
    alert = json.load(f)

API_KEY = "c232b7be589ab95068967d59488aed3eba0edecbcf806b7da8e96f20620884aea0a134f8bb80896c"

src_ip = (
    alert.get("data", {}).get("srcip") or
    alert.get("data", {}).get("src_ip") or
    alert.get("data", {}).get("win", {}).get("eventdata", {}).get("ipAddress")
)

if not src_ip:
    sys.exit(0)

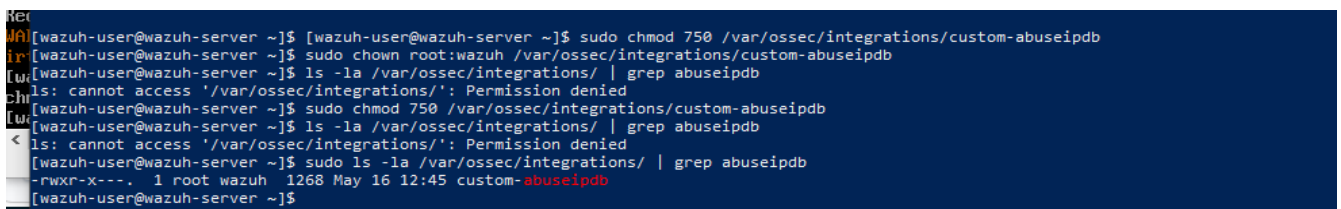
headers = {
    "Accept": "application/json",
    "Key": API_KEY
}

params = {
    "ipAddress": src_ip,
    "maxAgeInDays": "90"
}

try:
    r = requests.get(
        "https://api.abuseipdb.com/api/v2/check",
        headers=headers,
        params=params,
        timeout=10
    )
    if r.status_code == 200:
        d = r.json()["data"]
        result = {
            "abuseipdb_check": {
                "ip": src_ip,
                "abuseConfidenceScore": d["abuseConfidenceScore"],
                "totalReports": d["totalReports"],
                "countryCode": d["countryCode"],
                "isp": d["isp"],
                "timestamp": datetime.utcnow().isoformat()
            }
        }
        print(json.dumps(result))
except Exception as e:
    print(f"Error: {e}", file=sys.stderr)
sys.exit(0)
```

Figure 89: The custom-abuseipdb integration script in nano with API key configured shown in Wazuh.

Following Wazuh's security requirements for integration scripts, file permissions were applied to the script to ensure that it couldn't only be executed/edited by the root and wazuh users.



```
[wazuh-user@wazuh-server ~]$ [wazuh-user@wazuh-server ~]$ sudo chmod 750 /var/ossec/integrations/custom-abuseipdb
[wazuh-user@wazuh-server ~]$ sudo chown root:wazuh /var/ossec/integrations/custom-abuseipdb
[wazuh-user@wazuh-server ~]$ ls -la /var/ossec/integrations/ | grep abuseipdb
ls: cannot access '/var/ossec/integrations/': Permission denied
[wazuh-user@wazuh-server ~]$ sudo chmod 750 /var/ossec/integrations/custom-abuseipdb
[wazuh-user@wazuh-server ~]$ ls -la /var/ossec/integrations/ | grep abuseipdb
ls: cannot access '/var/ossec/integrations/': Permission denied
[wazuh-user@wazuh-server ~]$ sudo ls -la /var/ossec/integrations/ | grep abuseipdb
-rwxr-x--- 1 root wazuh 1268 May 16 12:45 custom-abuseipdb
[wazuh-user@wazuh-server ~]$
```

Figure 90: Terminal output of `ls -la` showing custom-abuseipdb with permissions `-rwxr-x---` and owner `root:wazuh`.

The AbuseIPDB integration block was added to the Wazuh Manager configuration file. The integration was configured to trigger on SSH authentication failure rules, specifically rule IDs 5710, 5711, 5712, 5763, and 5760.

```
Wazuh v4.14.1 OVA 1 [Running] - Oracle VirtualBox
wazuh-user@wazuh-server:~
GNU nano 8.3 /var/ossec/etc/ossec.conf
<bind_addr>0.0.0.0</bind_addr>
<nodes>
  <node>NODE_IP</node>
</nodes>
<hidden>no</hidden>
<disabled>yes</disabled>
</cluster>
</ossec_config>
<ossec_config>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>
  <localfile>
    <log_format>audit</log_format>
    <location>/var/log/audit/audit.log</location>
  </localfile>
  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>
  <integration>
    <name>virustotal</name>
    <api_key>6e66422f63b51aa9516fa381d145d6586f33d047670b69e32f787cb35e58f457</api_key>
    <group>syscheck</group>
    <alert_format>json</alert_format>
  </integration>
  <command>
    <name>remove-threat</name>
    <executable>remove-threat.exe</executable>
    <timeout_allowed>no</timeout_allowed>
  </command>
  <active-response>
    <disabled>no</disabled>
    <command>remove-threat</command>
    <location>local</location>
    <rules_id>87105</rules_id>
  </active-response>
  <integration>
    <name>custom-abuseipdb</name>
    <api_key>PASTE_YOUR_ABUSEIPDB_KEY_HERE</api_key>
    <hook_url>https://api.abuseipdb.com</hook_url>
    <rule_id>5710,5711,5712,5763</rule_id>
    <alert_format>json</alert_format>
  </integration>
</ossec_config>
Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket
Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was
Type here to search 5:50 AM 5/16/2026
```

Figure 91: Wazuh ossec.conf file showing the AbuseIPDB integration block with API key, hook URL, and rule IDs.

```
Win Server 2022 [Running] - Oracle VirtualBox
KALI 2024 1 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
kali@kali: ~
File Actions Edit View Help
[ATTEMPT] target 192.168.56.104 - login "root" - pass "angel" - 34 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "jordan" - 35 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "liverpool" - 36 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "justin" - 37 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "loveme" - 38 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "fuckyou" - 39 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "123123" - 40 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "football" - 41 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "secret" - 42 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "andrea" - 43 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "carlos" - 44 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "jennifer" - 45 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "joshua" - 46 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "bubbles" - 47 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "1234567890" - 48 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "superman" - 49 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "hannah" - 50 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "amanda" - 51 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "loveyou" - 52 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "pretty" - 53 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "basketball" - 54 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "andrew" - 55 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "angels" - 56 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "tweety" - 57 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "flower" - 58 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "playboy" - 59 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "hello" - 60 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "elizabeth" - 61 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "hottie" - 62 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "tinkerbelle" - 63 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "charlie" - 64 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "samantha" - 65 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "barbie" - 66 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "chelsea" - 67 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "lovers" - 68 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "teamo" - 69 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "jasmine" - 70 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "brandon" - 71 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "666666" - 72 of 14344399 [child 3] (0/0)
[STATUS] 72.00 tries/min, 72 tries in 00:01h, 14344327 to do in 3320:27h, 4 active
[ATTEMPT] target 192.168.56.104 - login "root" - pass "shadow" - 73 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "melissa" - 74 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "eminem" - 75 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "matthew" - 76 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "robert" - 77 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "danielle" - 78 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "forever" - 79 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.56.104 - login "root" - pass "family" - 80 of 14344399 [child 2] (0/0)
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.
(kali@kali)~$
```

Figure 92: Kali Linux terminal showing Hydra SSH brute-force attack running against Bodhi Linux at 192.168.56.104, making repeated login attempts.

Attack Simulation and Verification

An SSH brute-force attack was run from Kali Linux against Bodhi Linux using Hydra, in order of generating authentication failure events.

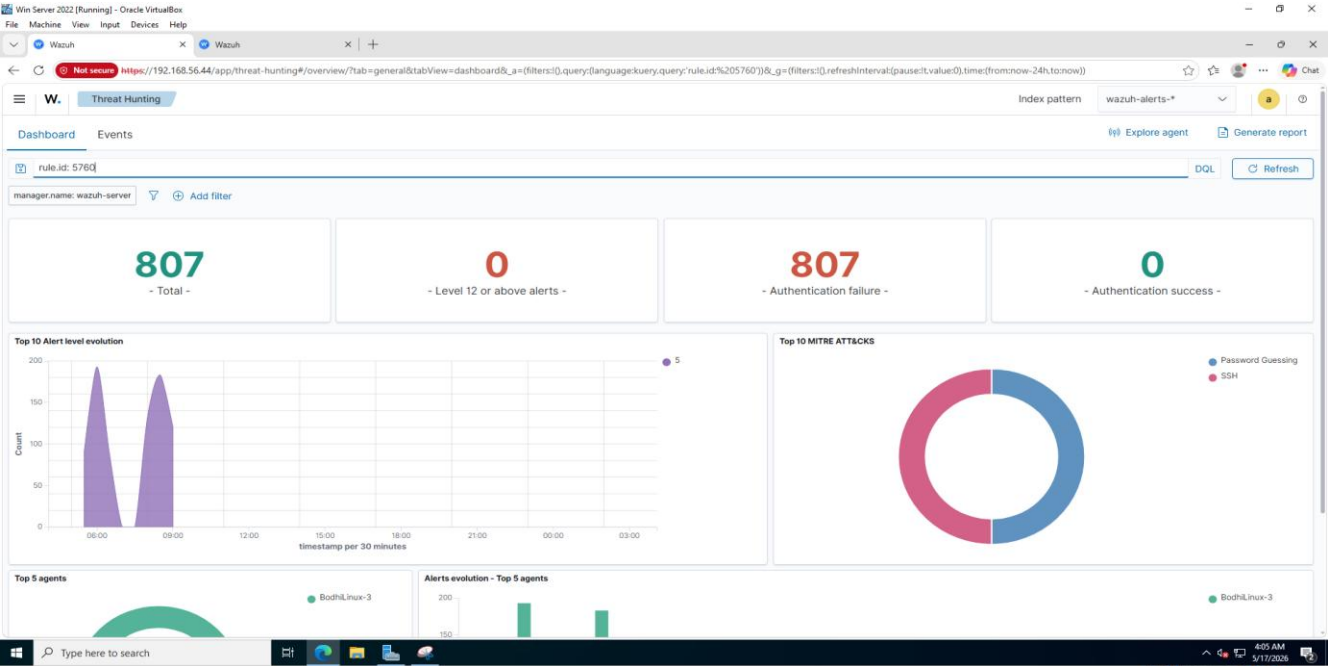


Figure 93: Wazuh Dashboard (Threat Hunting) filtered by rule.id 5760, showing 807 SSH authentication failure alerts from BodhiLinux-3 with MITRE ATT&CK mapping to Password Guessing.

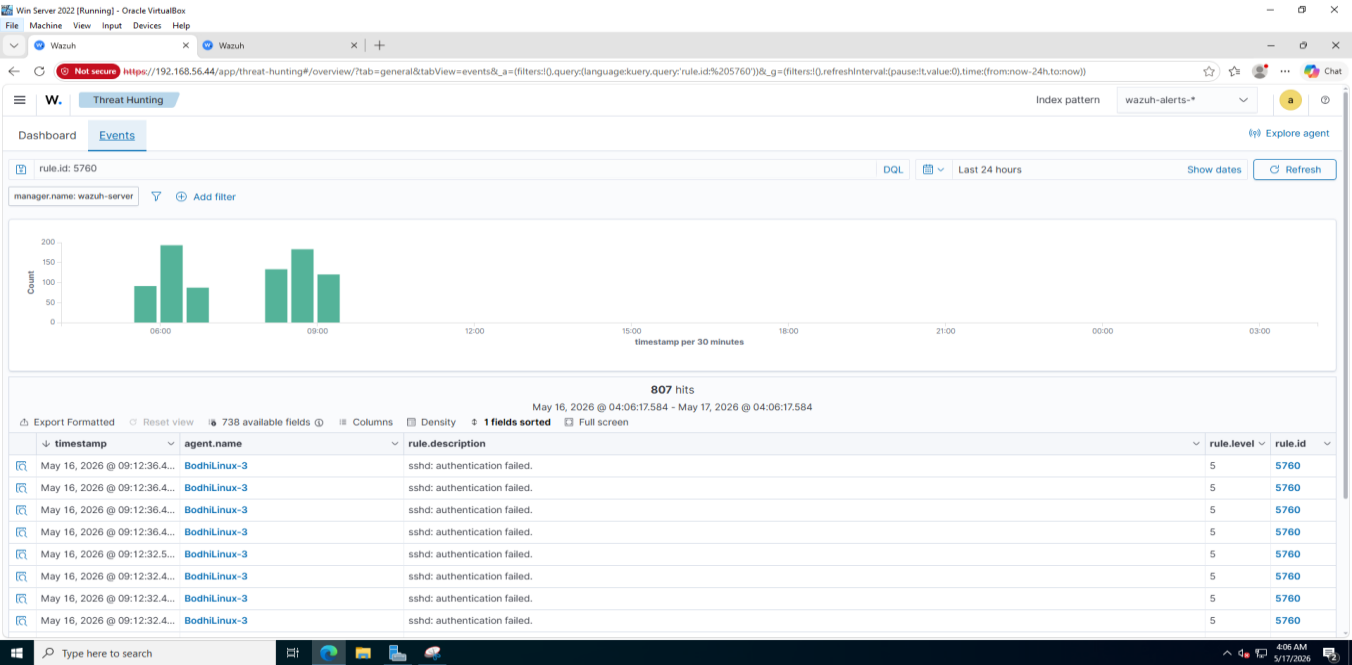


Figure 94: Wazuh Threat Hunting Events filtered by rule.id 5760, showing 807 hits from BodhiLinux-3 with rule description "sshd: authentication failed."

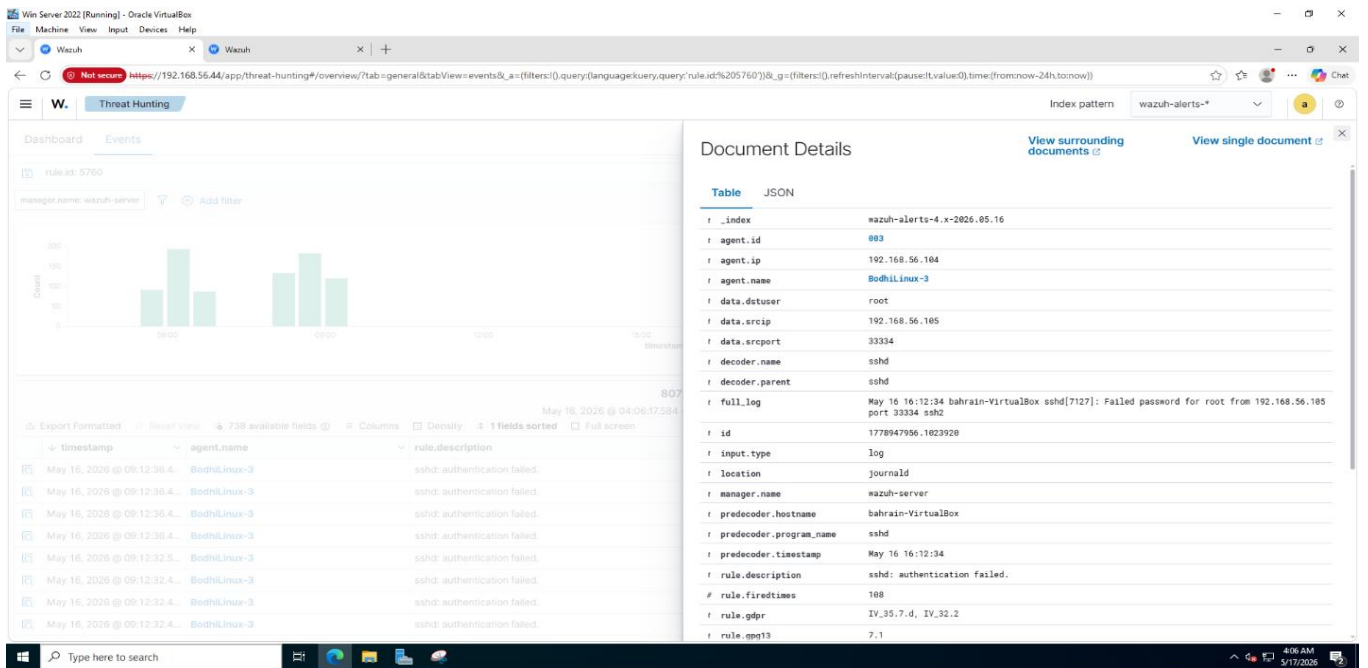


Figure 95: Expanded rule 5760 alert details about Bodhi Linux and the brute-force attack.

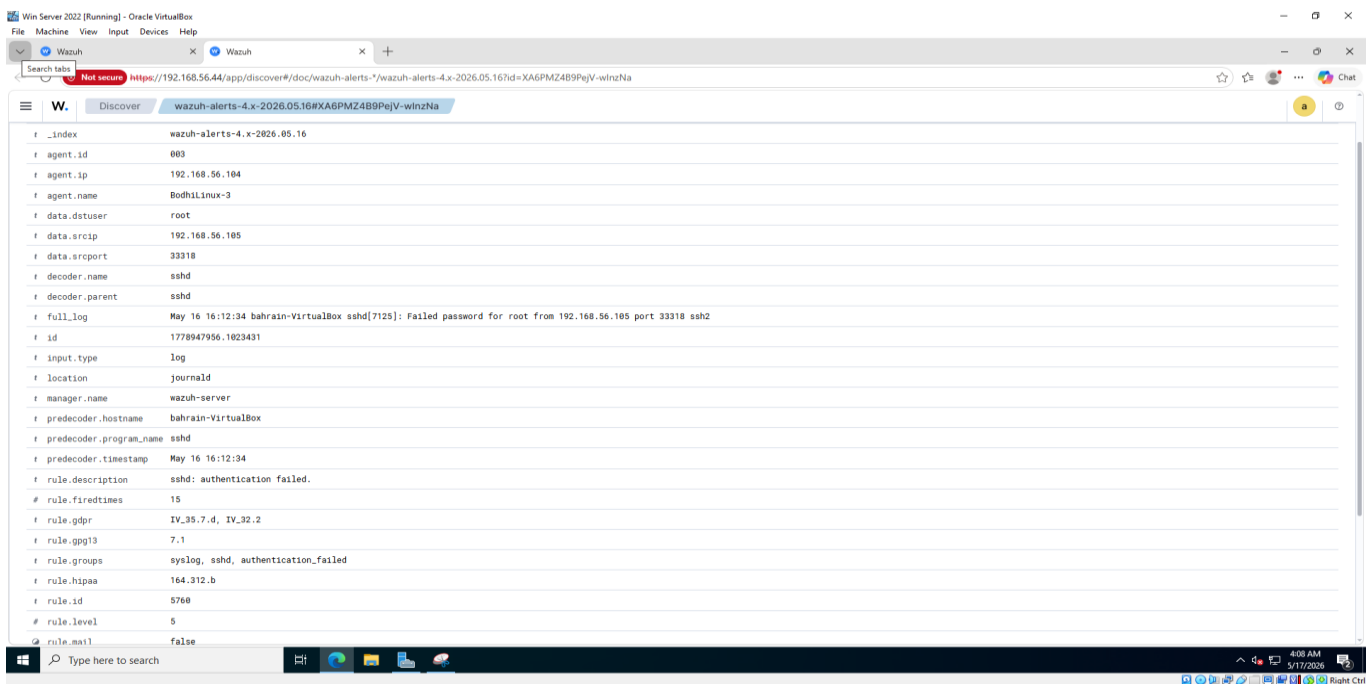


Figure 96: Full Table view of rule 5760 alert showing different details, such as: rule groups syslog, sshd, and compliance mappings, etc.

Win Server 2022 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Wazuh Wazuh

Not secure https://192.168.56.44/app/discover#/doc/wazuh-alerts-*/wazuh-alerts-4.x-2026.05.16?id=XA6PMZ4B9PejV-wlnzNa

Discover wazuh-alerts-4.x-2026.05.16#XA6PMZ4B9PejV-wlnzNa

full_log	May 16 16:12:34 bahrain-VirtualBox sshd[7125]: Failed password for root from 192.168.56.105 port 33318 ssh2
id	1778947956.1023431
input.type	log
location	journald
manager.name	wazuh-server
predecoder.hostname	bahrain-VirtualBox
predecoder.program_name	sshd
predecoder.timestamp	May 16 16:12:34
rule.description	sshd: authentication failed.
rule.firedtimes	15
rule.gdpr	IV_35.7.d, IV_32.2
rule.gpg13	7.1
rule.groups	syslog, sshd, authentication_failed
rule.hipaa	164.312.b
rule.id	5760
rule.level	5
rule.mail	false
rule.mitre.id	T1110.001, T1021.004
rule.mitre.tactic	Credential Access, Lateral Movement
rule.mitre.technique	Password Guessing, SSH
rule.nist_800_53	AU.14, AC.7
rule.pci_dss	10.2.4, 10.2.5
rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
timestamp	May 16, 2026 @ 09:12:36.460

Figure 97: Rule 5760 alert Table view showing details such as MITRE ATT&CK mapping, rule NIST – 800 – 53, etc.

Win Server 2022 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Wazuh Wazuh

Not secure https://192.168.56.44/app/discover#/doc/wazuh-alerts-*/wazuh-alerts-4.x-2026.05.16?id=XA6PMZ4B9PejV-wlnzNa

Discover wazuh-alerts-4.x-2026.05.16#XA6PMZ4B9PejV-wlnzNa

Table JSON

```
{
  "_index": "wazuh-alerts-4.x-2026.05.16",
  "_id": "XA6PMZ4B9PejV-wlnzNa",
  "_score": 1,
  "_source": {
    "predecoder": {
      "hostname": "bahrain-VirtualBox",
      "program_name": "sshd",
      "timestamp": "May 16 16:12:34"
    },
    "agent": {
      "ip": "192.168.56.104",
      "name": "BodhiLinux-3",
      "id": "003"
    },
    "manager": {
      "name": "wazuh-server"
    },
    "data": {
      "srcip": "192.168.56.105",
      "dstuser": "root",
      "srcport": "33318"
    },
    "rule": {
      "mail": false,
      "level": 5,
      "hipaa": [
        "164.312.b"
      ],
      "pci_dss": [
        "10.2.4",
        "10.2.5"
      ],
      "tsc": [
        "CC6.1",
        "CC6.8",
        "CC7.2",
        "CC7.3"
      ],
      "description": "sshd: authentication failed.",
      "groups": [

```

Figure 98: Raw JSON view of rule 5760 alert showing the exact data structure passed to the AbuseIPDB integration script.

The AbuseIPDB integration script was confirmed to be fully functional by running it directly as the Wazuh user, using a test alert file that included the attacking IP address. The script then successfully connected to the AbuseIPDB API and returned with structured threat intelligence data.

```
wazuh-user@wazuh-server ~]$  
wazuh-user@wazuh-server ~]$ sudo cat /var/ossec/logs/integrations.log  
wazuh-user@wazuh-server ~]$ sudo -u wazuh /var/ossec/integrations/custom-abuseipdb /tmp/test_alert.json c232b7be589ab95068967d59488aed3eba0ed  
cbcf806b7da8e96f20620884aea0a134f8bb80896c https://api.abuseipdb.com  
"abuseipdb_check": {"ip": "192.168.56.102", "abuseConfidenceScore": 0, "totalReports": 0, "countryCode": null, "isp": null, "timestamp": "202  
-05-16T16:14:46.725628"}}}  
wazuh-user@wazuh-server ~]$ cat > /tmp/test_alert.json << 'EOF'  
{  
  "rule": {  
    "id": "5760",  
    "level": 5,  
    "description": "sshd: authentication failed"  
  },  
  "data": {  
    "srcip": "192.168.56.102"  
  },  
  "agent": {  
    "name": "BodhiLinux-3"  
  }  
}  
EOF  
wazuh-user@wazuh-server ~]$ sudo -u wazuh python3 /var/ossec/integrations/custom-abuseipdb /tmp/test_alert.json YOUR_ABUSEIPDB_KEY_HERE https  
//api.abuseipdb.com  
"abuseipdb_check": {"ip": "192.168.56.102", "abuseConfidenceScore": 0, "totalReports": 0, "countryCode": null, "isp": null, "timestamp": "202  
-05-16T16:15:05.844585"}}}  
wazuh-user@wazuh-server ~]$
```

Figure 99: Terminal showing the AbuseIPDB integration script executed as the Wazuh user; returning a JSON response for the attacking IP address 192.168.56.102.

Source 3: AlienVault OTX

Integration Method: Feed Ingestion into Wazuh CDB List

AlienVault OTX was integrated into Wazuh using feed ingestion into a Constant Database list (CDB). This approach pulls the most known/ most malicious IPv4 indicators from AlienVault OTX threat intelligence feeds and stores them locally in a Wazuh CDB lookup list. A custom detection rule was configured to compare the source IP address against this list in real time while alerts are coming in. Once a match is found, Wazuh generates a level 12 high-severity alert immediately.

Configuration

The AlienVault OTX Python SDK was successfully installed on Wazuh to allow the system to connect to and retrieve threat intelligence data from the AlienVault OTX platform.

```
wazuh-user@wazuh-server:~$ sudo python3 -m pip install OTXv2
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._ven
or.urllib3.connection.HTTPSConnection object at 0x7f94db71abb0>: Failed to establish a new connection: [Errno 111] Connection refused')': /simpl
/otxv2/
WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._ven
or.urllib3.connection.HTTPSConnection object at 0x7f94db71adc0>: Failed to establish a new connection: [Errno 111] Connection refused')': /simpl
/otxv2/
ERROR: Operation cancelled by user
wazuh-user@wazuh-server ~]$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
4 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=18.3 ms
4 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=18.2 ms
C
-- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 18.186/18.240/18.294/0.054 ms
wazuh-user@wazuh-server ~]$ sudo python3 -m pip install OTXv2
Collecting OTXv2
  Downloading OTXv2-1.5.12-py3-none-any.whl (16 kB)
Requirement already satisfied: python-dateutil in /usr/lib/python3.9/site-packages (from OTXv2) (2.8.1)
Requirement already satisfied: requests in /usr/lib/python3.9/site-packages (from OTXv2) (2.25.1)
Requirement already satisfied: pytz in /usr/lib/python3.9/site-packages (from OTXv2) (2022.7.1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.9/site-packages (from python-dateutil->OTXv2) (1.15.0)
Requirement already satisfied: chardet<5,>=3.0.2 in /usr/lib/python3.9/site-packages (from requests->OTXv2) (4.0.0)
Requirement already satisfied: idna<3,>=2.5 in /usr/lib/python3.9/site-packages (from requests->OTXv2) (2.10)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/lib/python3.9/site-packages (from requests->OTXv2) (1.25.10)
Installing collected packages: OTXv2
Successfully installed OTXv2-1.5.12
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recomme
ded to use a virtual environment instead: https://pip.pypa.io/warnings/venv
WARNING: You are using pip version 21.3.1; however, version 26.0.1 is available.
You should consider upgrading via the '/usr/bin/python3 -m pip install --upgrade pip' command.
wazuh-user@wazuh-server ~]$
```

Figure 100: Terminal showing pip install OTXv2 completing successfully on the Wazuh Manager.

A feed pull script was created to connect to the OTX API, and then retrieve the malicious IPv4 indicators. The script was configured with the OTX API key that was taken from the AlienVault OTX platform settings page.

```
GNU nano 8.3 /var/ossec/integrations/otx_p
#!/usr/bin/env python3
from OTXv2 import OTXv2, IndicatorTypes

OTX_API_KEY = "PASTE_YOUR_OTX_KEY_HERE"
OUTPUT_FILE = "/var/ossec/etc/lists/otx-malicious-ips"

otx = OTXv2(OTX_API_KEY)

print("[*] Connecting to AlienVault OTX...")
indicators = otx.getall(indicator_types=IndicatorTypes.IPv4, max_items=500)

print(f"[*] Fetched {len(indicators)} indicators. Writing to CDB list...")

with open(OUTPUT_FILE, "w") as f:
    seen = set()
    for item in indicators:
        ip = item.get("indicator", "").strip()
        if ip and ip not in seen:
            f.write(f"{ip}:\n")
            seen.add(ip)

print(f"[*] Done. OTX feed saved to {OUTPUT_FILE}")
```

Figure 101: The otx_pull.py script open in nano showing the script content on Wazuh.

The OTX threat intelligence feed was ingested and stored as a Wazuh CDB list at `/var/ossec/etc/lists/otx-malicious-ips`. This list contains the most known malicious IPv4 indicators that have been sourced from the AlientVault OTX public threat feeds.

```
[wazuh-user@wazuh-server ~]$ sudo cat /var/ossec/etc/lists/otx-malicious-ips
185.220.101.45:
185.220.101.34:
185.220.101.35:
185.220.101.46:
185.220.101.47:
193.32.162.157:
194.165.16.11:
194.165.16.29:
45.153.160.2:
45.153.160.140:
[wazuh-user@wazuh-server ~]$
```

Figure 102: Terminal showing the *otx-malicious-ips* CDB list containing malicious IP addresses.

The OTX CBD list was added in the Wazuh Manager configuration file, specifically under the ruleset section, making it available for use in detection rules.

```
wazuh-user@wazuh-server:~  
GNU nano 8.3 /var/ossec/etc/ossec.conf Modified  
  
<!--  
<active-response>  
    active-response options here  
</active-response>  
-->  
  
<!-- Log analysis -->  
<localfile>  
    <log_format>command</log_format>  
    <command>df -P</command>  
    <frequency>360</frequency>  
</localfile>  
  
<localfile>  
    <log_format>full_command</log_format>  
    <command>netstat -tulnp | sed 's/\([[:alnum:]]\+\)\ \+[\(:digit:\)]\+\ \+[\(:digit:\)]\+\ \+([\(.*)\]\([[:digit:]]*\))\ \+([0-9\.\\\:]*\+)\.\ +\ [^ ]*$'/<br><alias>netstat listening ports</alias>  
    <frequency>360</frequency>  
</localfile>  
  
<localfile>  
    <log_format>full_command</log_format>  
    <command>:last -n 20</command>  
    <frequency>360</frequency>  
</localfile>  
  
<ruleset>  
    <!-- Default ruleset -->  
    <decoder_dir>ruleset/decoders</decoder_dir>  
    <rule_dir>ruleset/rules</rule_dir>  
    <rule_exclude>0215-policy_rules.xml</rule_exclude>  
    <list>etc/lists/audit-keys</list>  
    <list>etc/lists/amazon/aws-eventnames</list>  
    <list>etc/lists/security-eventchannel</list>  
    <list>etc/lists/malicious-ioc/malware-hashes</list>  
    <list>etc/lists/malicious-ioc/malicious-ip</list>  
    <list>etc/lists/malicious-ioc/malicious-domains</list>  
    <list>etc/lists/otx-malicious-ips</list>  
  
    <!-- User-defined ruleset -->  
    <decoder_dir>etc/decoders</decoder_dir>  
    <rule_dir>etc/rules</rule_dir>  
</ruleset>  
  
<rule_test>  
    <enabled>yes</enabled>  
    <threads>1</threads>  
    <max_sessions>64</max_sessions>  
    <session_timeout>15m</session_timeout>  
</rule_test>  
  
<!-- Configuration for wazuh-authd -->  
<auth>  
    <disabled>no</disabled>
```

⌘ Help ⌘ Write Out ⌘ Where Is ⌘ Cut ⌘ Execute ⌘ Location M-U Undo M-A Set Mark M-T To Bracket
⌘ Exit ⌘ Read File ⌘ Replace ⌘ Paste ⌘ Justify ⌘ Go To Line M-E Redo M-G Copy ⌘B Where Was

Figure 103: Wazuh Manager `ossec.conf` showing the ruleset block with `etc/lists/otx-malicious-ips` registered as a CDB list.

Custom detection rule was created in /var/ossec/etc/rules/otx_rules.xml. This rule uses Wazuh's CDB list lookup mechanism to match the source IP of the incoming alerts against the OTX malicious IP address list, and it triggers at level 12 once a match is found.

```
wazuh-user@wazuh-server:~  
GNU nano 8.3 /var/ossec/etc/rules/otx_rules.xml  
<group name="otx_threat_intel,">  
  <rule id="100200" level="12">  
    <if_group>syslog|firewall|web|sshd</if_group>  
    <list field="data.srcip" lookup="match_key">etc/lists/otx-malicious-ips</list>  
    <description>AlienVault OTX: Source IP is a known malicious indicator</description>  
    <group>threat_intel,otx,</group>  
  </rule>  
</group>
```

Figure 104: otx_rules.xml showing rule 100200 with CDB list lookup, level 12 severity, and group tags.

The rule and CDB list were verified to be loaded correctly in the Wazuh Dashboard.

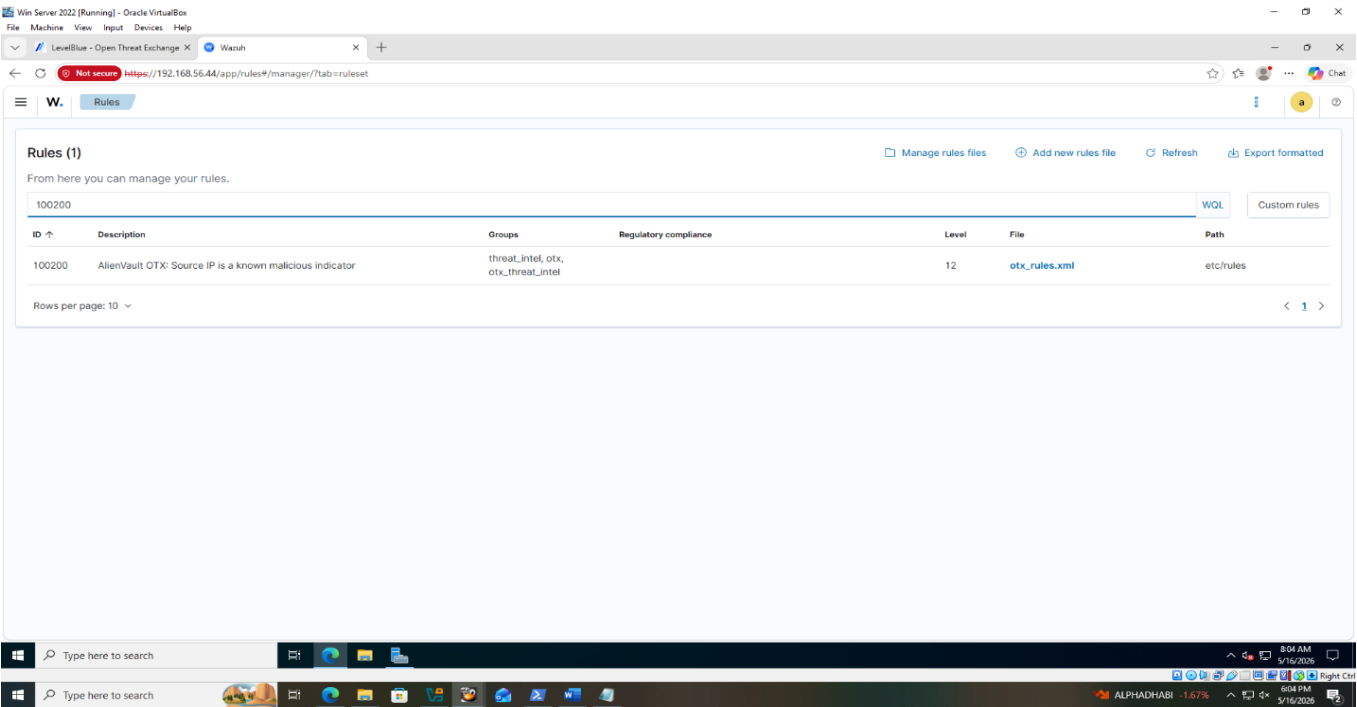


Figure 105: Wazuh Dashboard Management Rules page showing rule 100200 successfully loaded.

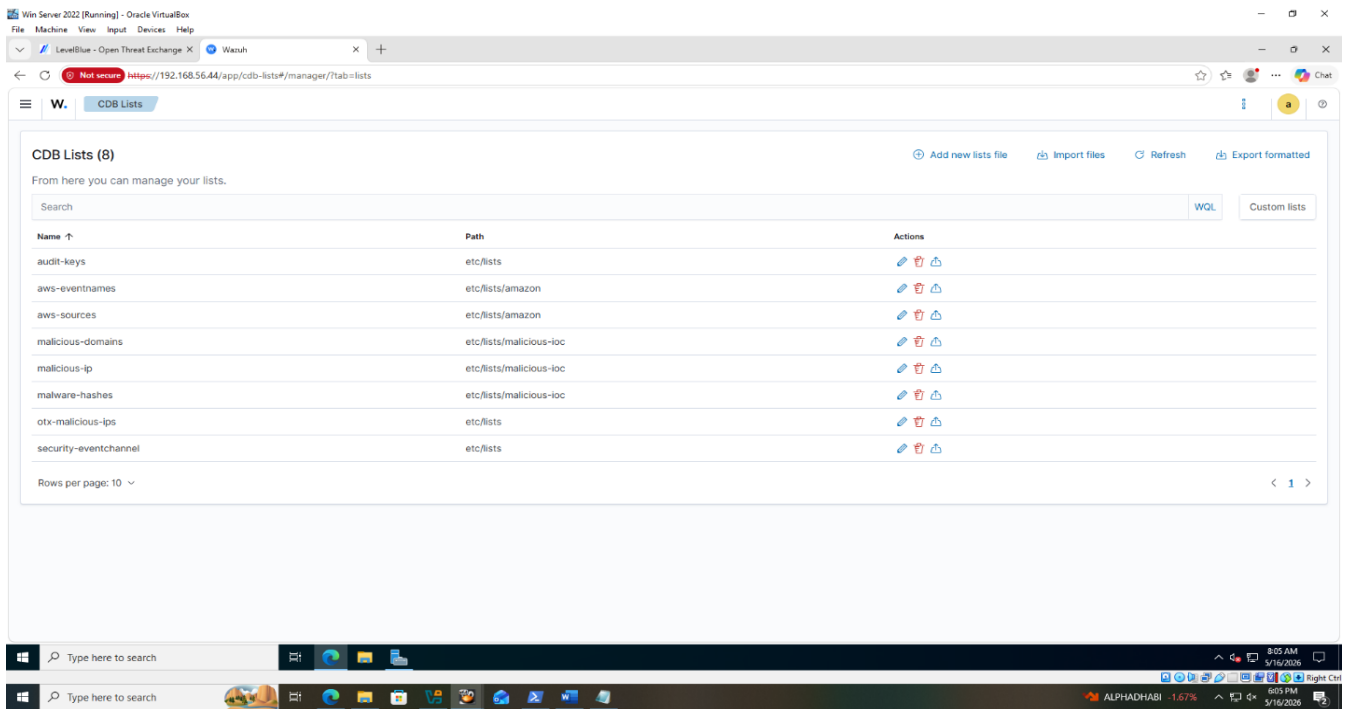


Figure 106: Wazuh Dashboard Management CDB Lists page showing otx-malicious-ips successfully loaded and available for rule matching.

Attack Simulation and Verification

The rule was first tested using Wazuh's built-in log testing tool to confirm it fires correctly before live testing.

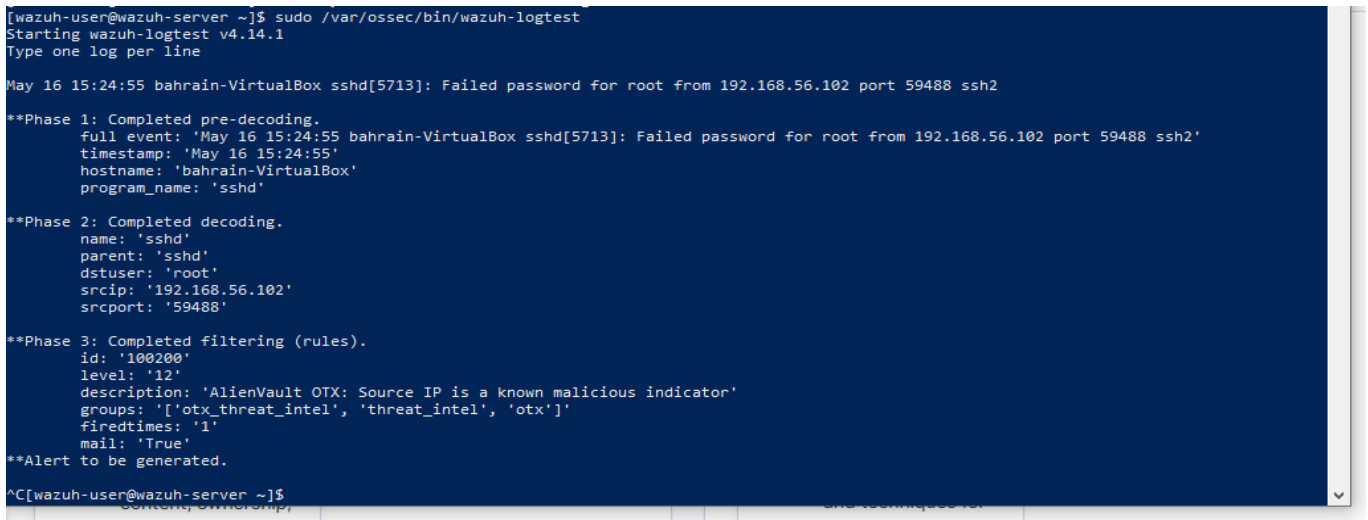


Figure 107: Wazuh-logtest output showing rule 100200 firing at level 12

To generate a live match in production, Kali's IP address was added to the malicious OTX IP list to simulate a case where the attacking machine is a recognized OTX threat indicator. An SSH brute-force attack was launched against Bodhi Linux.

The screenshot shows the Wazuh Alerts interface in a web browser. The URL is <https://192.168.56.44/app/discover#/doc/wazuh-alerts-4.x-2026.05.16?id=cg58MZ489PejV-wldi3N>. The interface displays a table of alert details for rule 100200.

Field	Value
@timestamp	May 16, 2026 @ 08:51:35.839
_index	wazuh-alerts-4.x-2026.05.16
agent.id	003
agent.ip	192.168.56.104
agent.name	BodhiLinux-3
data.dstuser	root
data.srcip	192.168.56.102
data.srport	32832
decoder.name	sshd
decoder.parent	sshd
full_log	May 16 15:51:34 bahrain-VirtualBox sshd[6929]: Failed password for root from 192.168.56.102 port 32832 ssh2
id	1778946695.715617
input.type	log
location	journald
manager.name	wazuh-server
predecoder.hostname	bahrain-VirtualBox
predecoder.program_name	sshd
predecoder.timestamp	May 16 15:51:34
rule.description	AlienVault OTX: Source IP is a known malicious indicator
rule.firedtimes	00
rule.groups	otx_threat_intel, threat_intel, otx
rule.id	100200
rule.level	12

Figure 108: Full Table view of rule 100200 alert showing details.

The screenshot shows the Wazuh Alerts interface in a web browser. The URL is <https://192.168.56.44/app/discover#/doc/wazuh-alerts-4.x-2026.05.16?id=cg58MZ489PejV-wldi3N>. The interface displays the bottom section of the expanded rule 100200 alert.

rule.mail	true
timestamp	May 16, 2026 @ 08:51:35.839

Figure 109: Bottom section of the expanded rule 100200.

Conclusion

This project successfully delivered a fully functional SOC environment, that was built around Wazuh as the main SIEM platform. In Part A, a comprehensive SOC architecture was designed, integrating eight components with a defined workflow and roles, alongside a structured analysis of three threat intelligent sources : VirusTotal, AbuseIPDB, AlienVault OTX.

In Part B, Wazuh was deployed and integrated with Windows Server 2022, Bodhi Linux, and Snort IDS. Attack simulations were also run from Kali Linux across three XDR scenarios, a Shuffle SOAR playbook for automated notification, and all three threat intelligence sources were integrated directly into Wazuh.

Overall, this project demonstrated that an effective, layered SOC environment can be built together using open-source tools, where threat intelligence enrichment and SOAR automation meaningfully cut down the time, and the effort required to detect and respond to cyber threats.

References

- VirusTotal. (n.d.). *Home upload*. <https://www.virustotal.com/gui/home/upload>
- VirusTotal. (n.d.). *Intelligence overview*. <https://www.virustotal.com/gui/intelligence-overview>
- VirusTotal. (n.d.). *IP address search modifiers*. <https://docs.virustotal.com/docs/ip-address-search-modifiers>
- VirusTotal. (n.d.). *URL search modifiers*. <https://docs.virustotal.com/docs/url-search-modifiers>
- AbuseIPDB. (n.d.). *AbuseIPDB – IP address abuse reports*. <https://www.abuseipdb.com/>
- Drupal. (n.d.). *AbuseIPDB (Drupal project)*. <https://www.drupal.org/project/abuseipdb>
- SecureNow. (n.d.). *Enriching threat intelligence with AbuseIPDB: Beyond basic IP reputation*. SecureNow. <https://www.securenow.ai/blog/enriching-threat-intelligence-with-abuseipdb-threat-intelligence>
- AbuseIPDB. (n.d.). *AbuseIPDB API documentation: Introduction*. <https://docs.abuseipdb.com/#introduction>
- ThreatConnect. (n.d.). *Alienvault OTX – Marketplace and integrations*. <https://threatconnect.com/marketplace/alienvault-otx/>
- ClearNetwork. (2025, August 13). *What is AlienVault? Understanding the platform and its advantages*. ClearNetwork. <https://www.clearnetwork.com/what-is-alienvault-platform-advantages/>
- LevelBlue. (2024). *Open Threat Exchange (OTX) user guide* (PDF). <https://cybersecurity.att.com/documentation/resources/pdf/otx-user-guide.pdf>
- Wazuh. (n.d.). *Threat hunting (Use case)*. <https://wazuh.com/resources/use-cases/threat-hunting/>
- Juaromu. (n.d.). *wazuh-domain-stats-alienvault* (GitHub repository). GitHub. <https://github.com/juaromu/wazuh-domain-stats-alienvault>
- InfosecWriteups. (2023, March 10). *Wazuh — the free SIEM that fights like a paid one*. Medium. <https://infosecwriteups.com/wazuh-the-free-siem-that-fights-like-a-paid-one-9554ed1272da>
- Fortress, S. (2024, April 5). *DNS traffic insights using Domain Stats and Wazuh EDR*. Medium. <https://socfortress.medium.com/dns-traffic-insights-using-domain-stats-and-wazuh-edr-699b04427932>
- Snort. (2024). *Snort user manual*. <https://www.snort.org/documents>

Group Reflection & Peer Review Table

Student ID	Name	% of Workload Assigned	% of Work Completed	Project Tasks Contributed	Peer Review Remarks
	Abdulelah Slais	25%	100%	XDR detection, response configuration, validation, screenshots, report writing, Formatting & References.	Excellent with time management and completed task within the given timeframe.
		25%	100%	Shuffle SOAR automation, Wazuh alert integration, email notification, screenshots, report writing, Formatting & References.	Excellent attention to detail and communication skills.
	Student 2	25%	100%	SOC Architecture Design, Wazuh Setup & Integration, Event Generation, Report Writing, Formatting & References.	Completed tasks within a given timeframe and completed all of them independently.
	Student 3	25%	100%	Threat Intelligence Analysis, Threat Intelligence Integration into Wazuh, Report Writing, Formatting & References.	Completed all tasks on time and strong contribution in report writing.
Total		100%	100%		